ใบอนุมัติโครงงานปริญญานิพนธ์
สาขาวิชา เทคโนโลยีสารสนเทศ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ชื่อปริญญานิพนธ์ แอปพลิเคชันบนโทรศัพท์มือถือสำหรับตรวจจับชื่อยาและจำแนกประเภทของยา
Mobile Application for detecting drug names and categorizing types of medications

ผู้จัดทำ นาย อภิภัทร รุ่งเรือง
นาย อริญชัย นิลทะวงษ์

ปีการศึกษา 2567

โครงงานปริญญานิพนธ์นี้ได้รับการอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิทยาศาสตร์บัณฑิต สาขาวิชาเทคโนโลยีสารสนเทศ
ภาควิชาคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

..... ประธานกรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร.สุนีย์ พงษ์พินิจภิญโญ)

..... กรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร.ทัศนวรรณ ศูนย์กลาง)

..... อาจารย์ที่ปรึกษาและกรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร.วีณาวดี ม่วงอ้น)

แอปพลิเคชันบนโทรศัพท์มือถือสำหรับตรวจจับชื่อยาและจำแนกประเภทของยา

Mobile Application for detecting drug names and categorizing types of medications

อภิภัทร รุ่งเรือง

อริญชัย นิลทะวงษ์

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาในหลักสูตรวิทยาศาสตร์บัณฑิต
สาขาวิชา เทคโนโลยีสารสนเทศ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร
ปีการศึกษา 2567

ชื่อปริญญานิพนธ์ แอปพลิเคชันบนโทรศัพท์มือถือสำหรับตรวจจับชื่อยาและจำแนกประเภทของยา
Mobile Application for detecting drug names and categorizing types of medications.

ผู้จัดทำ นาย อภิภัทร รุ่งเรือง, นาย อริญชัย นิลทะวงษ์

อาจารย์ที่ปรึกษา ผู้ช่วยศาสตราจารย์ ดร.วีณาวดี ม่วงอ้น

ชื่อปริญญา วิทยาศาสตร์บัณฑิต (เทคโนโลยีสารสนเทศ)

ปีการศึกษา 2567

บทคัดย่อ

โครงงานวิจัยนี้นำเสนอการพัฒนาแอปพลิเคชันบนโทรศัพท์มือถือสำหรับตรวจจับชื่อยาและจำแนกประเภทของยา โดยแบ่งการดำเนินงานออกเป็นหลายขั้นตอน เริ่มจากการเตรียมชุดข้อมูล (Data sets) ที่รวบรวมจากเว็บไซต์และจัดกลุ่มข้อมูลยาในรูปแบบของชื่อยา, โรคที่ยาสามารถรักษาได้, และสรรพคุณของยา ข้อมูลเหล่านี้จะถูกแบ่งออกเป็น 5 หมวดหมู่ ได้แก่ ยาแก้ปวด, ยาแก้ไอ, ยาฆ่าเชื้อ, ยาแก้แพ้ และยาระงับประสาท เพื่อใช้ในการฝึกโมเดล Deep Learning โดยใช้ไลบรารี Python เช่น NLTK และ Pandas สำหรับการประมวลผล Natural Language Processing และการวิเคราะห์ข้อมูลนอกจากนี้ แอปพลิเคชันยังใช้ API ของ Google Cloud Vision AI ในการสแกนและตรวจจับชื่อยาจากภาพถ่าย เมื่อสแกนสลากยาที่มีชื่อยาตรงกับข้อมูลในชุดข้อมูล แอปพลิเคชันจะแสดงข้อมูลที่สำคัญต่อการตัดสินใจในการใช้ยา เช่น ชื่อยา, โรคที่รักษาได้, สรรพคุณของยา, และประเภทของยา หากไม่พบชื่อยานั้นในชุดข้อมูล แอปพลิเคชันจะสามารถจำแนกประเภทของยาจากสรรพคุณโดยการสแกนข้อความจากสรรพคุณของยาและนำไปวิเคราะห์ด้วยโมเดล Deep Learning เพื่อแสดงผลลัพธ์ให้ผู้ใช้ทราบถึงประเภทและโรคที่ยาสามารถรักษาได้แอปพลิเคชันนี้มีประโยชน์ในการช่วยให้ผู้ใช้สามารถตัดสินใจเกี่ยวกับการใช้ยาได้อย่างมีข้อมูลและมั่นใจ โดยเฉพาะในกรณีที่ไม่มีข้อมูลยานั้นในฐานข้อมูล

งานวิจัยนี้นำเสนอการพัฒนาแอปพลิเคชันโดยใช้โมเดล LSTM และโมเดล Naive Bayes เพื่อ predict ประเภทของยาจากข้อความสรรพคุณจากกล่องยาหรือตัวซองยาที่ได้จาก OCR ในฐานข้อมูลยาจะแบ่งเป็นยาไทยและยาจากต่างประเทศ แต่จากการที่ยาไทยมีจำนวนน้อยที่คือ 120 ชื่อ และ ยาต่างประเทศมีจำนวน 198,517 ชื่อ จึงต้องแบ่งโมเดลในการใช้งานคือ โมเดล LSTM ใช้กับตัวยาภาษาอังกฤษ ตัวยาภาษาไทยนั้นใช้ Naive Bayes ความแม่นยำในการจำแนกประเภทยาของโมเดล LSTM (long short-term memory) อยู่ที่ 97% ส่วนโมเดล Naive Bayes อยู่ที่ 85% โดยค่าเหล่านี้คือค่าความถูกต้องในการจำแนกประเภทของยา

คำสำคัญ : ตรวจจับชื่อยา,จำแนกประเภทของยา,การสแกนสลากยา,ข้อมูลยา

กิตติกรรมประกาศ

การจัดทำรายงานฉบับนี้ สำเร็จลุล่วงตามวัตถุประสงค์ไปได้ด้วยดี เนื่องจากได้รับการสนับสนุนจากบุคลากรหลายฝ่ายดังนี้ ผู้ช่วยศาสตราจารย์ ดร.วีณาวดี ม่วงอ้น ผู้เป็นอาจารย์ที่ปรึกษาในการการจัดทำรายงานฉบับนี้ และได้สละเวลาอันมีค่าในการให้คำปรึกษา แนะนำแนวทาง ให้ความรู้ต่าง ๆ ตลอดระยะเวลาที่ผ่านมา ในการจัดทำรายงานเล่มนี้

ข้าพเจ้าขอขอบคุณคณะอาจารย์ภาควิชาคอมพิวเตอร์ และคณะกรรมการทุกท่านที่มอบความรู้ คำแนะนำ ในการทำงานตลอดระยะเวลาที่ผานมา รวมถึงผู้ที่มีส่วนร่วมในการให้ข้อมูล และเป็นที่ปรึกษาในการทำรายงานเล่มนี้จนกลายเป็นรายงานฉบับสมบูรณ์ ข้าพเจ้าขอขอบพระคุณทุกท่านไว้ ณ ที่นี้

อภิภัทร รุ่งเรือง

อริญชัย นิลทะวงษ์

สารบัญ

หน้า

บทคัดย่อ ..... ก
กิตติกรรมประกาศ ..... ข
สารบัญ ..... ค
สารบัญตาราง ..... จ
สารบัญรูป ..... ฉ
บทที่ 1 บทนำ 1
1.1 ที่มาและความสำคัญของปัญหา 1
1.2 วัตถุประสงค์ 1
1.3 ลักษณะและขอบเขต 1
1.4 อุปกรณ์และเครื่องมือที่ใช้ 2
1.4.1 ฮาร์ดแวร์ 2
1.4.2 ซอฟต์แวร์ 2
1.5 ผลที่คาดว่าจะได้รับ 3
1.6 ภาระหน้าที่ความรับผิดชอบ 3
บทที่ 2 ผลงานที่เกี่ยวข้อง 4
2.1 Mobile search engine for medicine information on smartphone [1] 4
2.2 Application Development for Searching Drugs Information [2] 5
2.3 Google Lens [3] 6
2.4 เปรียบเทียบความแตกต่างของผลงาน 6
บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง 7
3.1 OCR (Optical character recognition) [4] 7
3.2 Natural language processing (NLP) [5] 9
3.3 Text classification [8] 10
3.4 Long Short-Term Memory [9] 11
3.5 Naive Bayes [10] 12
บทที่ 4 ขั้นตอนและแผนการดำเนินงาน 13
4.1 ขั้นตอนการดำเนินงาน 13
4.1.1 คิดหัวข้อโครงงานและรวบรวมข้อมูล 13
4.1.2 รวบรวมข้อมูลและกำหนดขอบเขตของโครงงาน 13
4.1.3 ศึกษาทฤษฎีและความรู้ที่เกี่ยวข้อง 13
4.1.4 การออกแบบระบบและพัฒนาโปรแกรม 13
4.1.5 ทดสอบและปรับปรุง 13
4.1.6 จัดทำเอกสารประกอบโครงงาน 13
4.2 แผนการดำเนินงาน 14

บทที่ 5 การวิเคราะห์และออกแบบระบบ 15
5.1 โครงสร้างของระบบ 15
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ 20
6.1 ผลการดำเนินงาน 20
6.1.1 ผลการดำเนินงานของโมเดล Naïve Bayes 20
6.1.2 ผลการดำเนินงานของโมเดล Deep learning 27
6.1.3 การเปรียบเทียบประสิทธิภาพของโมเดล Naïve Bayes และ Deep learning 36
6.2 ข้อจำกัด 37
6.3 ข้อเสนอแนะ 37
ภาคผนวก ก ตัวอย่างโค้ดและวิธีการติดตั้ง 39
ภาคผนวก ข คู่มือการใช้งาน 57

สารบัญตาราง

หน้า

ตารางที่ 2.4 ตารางเปรียบเทียบความแตกต่างของผลงาน.....6
ตารางที่ 4.3 แผนการดำเนินงาน..... 14

สารบัญรูป

หน้า

รูปที่ 1 ตัวอย่างของ Mobile search engine for medicine 4
รูปที่ 2 ตัวอย่างการทำงานของ Application for Searching Drugs Information 5
รูปที่ 3 การทำงานของ Optical Character Recognition 8
รูปที่ 4 การทำงานของการประมวลผลภาษาธรรมชาติ (NLP) 9
รูปที่ 5 Long Short-Term Memory 11
รูปที่ 6 โครงสร้างระบบ 16
รูปที่ 7 โครงสร้างระบบ แสดงการนำรูปผ่านกระบวนการ OCR 16
รูปที่ 8 โครงสร้างระบบ แสดงการตรวจจับและจำแนกข้อมูล จะแบ่งแยกเป็น 2 กรณี คือ เมื่อพบข้อมูลตรงกับ Data set และไม่พบข้อมูลตรงกับ Data set 17
รูปที่ 9 โครงสร้างระบบ แสดงการทำงานเมื่อ Text มีชื่อตรงกับ Data set 18
รูปที่ 10 โครงสร้างระบบ แสดงขั้นตอนการทำงานในกรณีที่เมื่อ Text ไม่ตรงกับ Data set 19
รูปที่ 11 กราฟแสดงค่า Precision, Recall และ F1-Score 20
รูปที่ 12 กราฟ Confusion Matrix ของกลุ่มตัวยาภาษาไทย 22
รูปที่ 13 กราฟแสดงค่า Accuracy ของสรรพคุณภาษาไทยใน Deep learning 24
รูปที่ 14 กราฟแสดงค่า Loss ของสรรพคุณภาษาไทยใน Deep learning 26
รูปที่ 15 กราฟแสดงค่า Accuracy ของสรรพคุณภาษอังกฤษใน Deep learning 27
รูปที่ 16 กราฟแสดงค่า Loss ของสรรพคุณภาษอังกฤษใน Deep learning 29
รูปที่ 17 กราฟ Confusion Matrix ของกลุ่มตัวยาภาษาอังกฤษ 31
รูปที่ 18 ผล Precision, Recall, และ F1-score ของแต่ละคลาส 33
รูปที่ 19 รูปเปรียบเทียบประสิทธิภาพของโมเดล Naïve Bayes และ Deep learning 36

รูปผนวกที่ ก.1 ไลบรารีสำหรับโมเดล machine learning 42
รูปผนวกที่ ก.2 ข้อมูลของตัวยาภาษาไทย 43
รูปผนวกที่ ก.3 ผลลัพธ์การ predict ของโมเดล naive bayes 45
รูปผนวกที่ ก.4 ไลบรารีสำหรับโมเดล Long short-term memmory 45
รูปผนวกที่ ก.5 ข้อมูลของยาภาษาอังกฤษ 47
รูปผนวกที่ ก.6 ข้อมูลของยาภาษาอังกฤษที่แบ่งคำแล้ว 47
รูปผนวกที่ ก.7 โค้ดนี้เป็นการฝึกโมเดลโดยใช้ EarlyStopping เพื่อหยุดการฝึกโมเดลหากไม่เกิดการพัฒนาเพิ่มเติม 53
รูปผนวกที่ ก.8 รูป Download Zip ไฟล์ของ Application 55
รูปผนวกที่ ก.9 รูปเปิดเมนู File เลือก Settings ใน Android Studio 55
รูปผนวกที่ ก.10 รูปการตั้งค่า Android Version 56
รูปผนวกที่ ก.11 รูปเลือก devices และกดปุ่ม Run Application เป็นการติดตั้ง Application 56
รูปผนวกที่ ข.1 หน้าเลือกภาษาของแอปพลิเคชัน 57

รูปผนวกที่ ข.2 หน้าเลือกฟังก์ชันของแอปพลิเคชัน กดปุ่มเปิดกล้องเพื่อถ่ายภาพ 58
รูปผนวกที่ ข.3 ฟังก์ชันเปิดกล้องถ่ายรูป ให้ทำการถ่ายรูปหน้าฉลากยาหรือกล่องยา 59
รูปผนวกที่ ข.4 หน้าแสดงผลของแอปพลิเคชันเมื่อพบข้อมูล 60
รูปผนวกที่ ข.5 หน้าเลือกฟังก์ชันของแอปพลิเคชัน กดปุ่มเลือกรูปภาพจาก Gallery 61
รูปผนวกที่ ข.6 ฟังก์ชันเปิด Gallery ในโทรศัพท์มือถือ เพื่อทำการเลือกรูปหน้าฉลากยา 62
รูปผนวกที่ ข.7 หน้าแสดงผลของแอปพลิเคชันเมื่อพบข้อมูล 63
รูปผนวกที่ ข.8 หน้าเลือกฟังก์ชันของแอปพลิเคชัน ในกรณีที่ไม่พบชื่อยี่ห้อยาใน Data sets จะจำแนกจากสรรพคุณ 64
รูปผนวกที่ ข.9 หน้าเลือกเมนูเปิดกล้องเพื่อถ่ายภาพ ในกรณีจำแนกจากสรรพคุณ 65
รูปผนวกที่ ข.10 หน้าแสดงผลของแอปพลิเคชันในกรณีจำแนกจากสรรพคุณ 66
รูปผนวกที่ ข.11 หน้าเลือกฟังก์ชันของแอปพลิเคชัน กดปุ่มเลือกรูปภาพจาก Gallery 67
รูปผนวกที่ ข.12 หน้าเมนูเลือกรูปภาพจาก Gallery ในกรณีจำแนกจากสรรพคุณ 68
รูปผนวกที่ ข.13 หน้าแสดงผลของแอปพลิเคชันในกรณีจำแนกจากสรรพคุณ 69
รูปผนวกที่ ข.14 หน้าแสดงผลของแอปพลิเคชันในกรณีไม่พบข้อมูลยา 70

คำอธิบายสัญลักษณ์และคำย่อ

NLTK = Natural Language Toolkit.

OCR = Optical Character Recognition.

NLP = Natural Language Processing.

ML = Machine Learning

LSTM = Long Short-Term Memory

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

ด้านสุขภาพเป็นหนึ่งในกลุ่มด้านสำคัญและเติบโตอย่างรวดเร็วในโลกปัจจุบัน กับการเปลี่ยนแปลงพฤติกรรมชีวิตคน คนต้องการยามากขึ้นเพื่อรับมือกับสภาวะความเครียดหรือโรคอื่น ๆ ผลที่เกิดขึ้นคือการใช้จ่ายเงินกับยามีมากขึ้น นอกจากนี้ขยะทางการแพทย์ที่เกิดขึ้นมีมากมาย ตามรายงานจากองค์การอนามัยโลก (WHO) อัตราการสร้างขยะทางการแพทย์ในโลกคือ 2.5 กิโลกรัมต่อผู้ป่วยหนึ่งเตียงใน 1 วัน ในประเทศอินเดีย อัตราขยะทางการแพทย์คือ 1.55 กิโลกรัมต่อผู้ป่วยหนึ่งเตียงใน 1 วัน ส่วนใหญ่ขยะทางการแพทย์นี้คือ ยา เหตุผลที่ทิ้งเพราะไม่มีข้อมูลเกี่ยวกับยาและไม่รู้ถึงกลุ่มของยานั้น ๆ ขยะทางการแพทย์เพิ่มขึ้นเรื่อย ๆ ทุกวันที่ผ่านไปเป็นการเป็นอันตรายต่อโลก หลังจากวิกฤติโควิด-19 สถานการณ์เศรษฐกิจกลายเป็นอันตรายมากขึ้นทุกวัน คนต้องหาวิธีที่จะประหยัดเงินเพื่อแก้ไขสถานการณ์เศรษฐกิจของตนเอง

จากปัญหาที่กล่าวมาโครงงานวิจัยนี้ เรามุ่งเน้นแก้ไขปัญหาเดียวกัน โดยการพัฒนา mobile application ที่จะตรวจจับยาด้วยความช่วยเหลือจากเทคโนโลยีการรับรู้ตัวอักษร (OCR) แอปพลิเคชันนี้จะให้ข้อมูล เช่น ชื่อของยา, โรคที่ยาสามารถใช้รักษาได้, สรรพคุณยา, ประเภทของยา เพื่อให้เป็นประโยชน์ต่อการใช้งาน กระบวนการในการทำงาน mobile application สแกนกล่องยาเพื่อดึงชื่อของยี่ห้อยาจากกล่องยาให้แสดงข้อมูลจาก Data sets และในกรณีที่ไม่มีชื่อยี่ห้อยาใน Data sets ก็ยังสามารถที่จะจำแนกประเภทของยาแต่ละชนิดออกมาได้เช่นกัน

1.2 วัตถุประสงค์

1) พัฒนา mobile application ที่จับข้อความชื่อของยี่ห้อยาและบอกสพรรคุณของยาชนิดนั้นจากการตรวจจับโดยจำแนกจากชื่อยี่ห้อของยา
2) พัฒนาระบบเพื่อบอกว่าสรรพคุณของยานั้นเหมาะกับผู้ป่วยประเภทใด

1.3 ลักษณะและขอบเขต

1) การเตรียม Data sets เพื่อนำมาใช้ในกระบวนการ Deep Learning โดยใช้ไลบรารี Python เริ่มต้นด้วยการรวบรวมข้อมูลจากเว็บไซต์และจัดข้อมูลของยาทั้งภาษาไทยและภาษาอังกฤษ โดยโดยเก็บข้อมูลดังนี้ ชื่อยา, โรคที่ยาสามารถใช้รักษาได้, สรรพคุณของยา, ประเภทของยา, ผลข้างเคียง เพื่อนำมาสร้างเป็น Data sets ซึ่งถูกแบ่งเป็นหมวดหมู่ จากนั้นจะใช้ไลบรารี Python เช่น NLTK, Scikit-learn และ Pandas เพื่อนำข้อมูลที่เตรียมไว้มาใช้ในกระบวนการ Deep Learning โดยทำการฝึกโมเดลโดยใช้เทคโนโลยี Natural Language Processing (NLP) จาก NLTK และ Scikit-learn เพื่อจำแนกประเภทข้อมูลและจัดกลุ่มข้อมูลจากข้อมูลที่ไม่มีอยู่ใน Data sets ในการจัดกลุ่มของยาจากสพรรคุณ รวมถึงใช้ Pandas เพื่อวิเคราะห์ข้อมูลที่นำเข้ามา
2) การนำชื่อของยาออกมาจากภาพถ่ายโดยการนำ API ของ Google cloud - Vision AI ใช้เป็นเครื่องมือในการประมวลผลภาพและแสดงออกมาเป็น Text เพื่อนำไปวิเคราะห์ต่อไป

3) ในแอปพลิเคชันนี้ผู้ใช้สามารถใช้ฟังก์ชันเปิดกล้องเพื่อสแกนสลากยาเพื่อให้สามารถนำ Text ที่ได้จากการสแกนชื่อยาเมื่อเจอชื่อยาที่ตรงกับข้อมูลยาใน Data sets จะนำข้อมูลที่มีประโยชน์และมีความสำคัญต่อผู้ใช้ในการตัดสินใจเกี่ยวกับการใช้ยาต่าง ๆ เช่น ชื่อยา, โรคที่ยาสามารถใช้รักษาได้, สรรพคุณของยา, และประเภทของยา ออกมาเป็นผลลัพธ์จาก Data sets
4) ในกรณีที่ไม่มีชื่อยี่ห้อยาใน Data sets แอปพลิเคชันยังสามารถทำการจำแนกประเภทของยาจากสรรพคุณโดยการสแกน Text จากสรรพคุณของยาไปใช้ในกระบวนการ Deep Learning ตัวอย่างเช่น ยา TYLENOL 500 ไม่มีข้อมูลอยู่ใน Data sets กรณีนี้จะต้องสแกน Text จากสรรพคุณยา และจะนำ Text เหล่านี้ไปเข้าสู่โมเดลในการจำแนกประเภทของยาจากสรรพคุณ ซึ่งผลลัพธ์ที่ได้จะแสดงให้ผู้ใช้ทราบถึงประเภทของและ โรคที่ยาสามารถใช้รักษาได้
5) มีฟังก์ชัน Text-to-Speech ในการอ่านข้อความจากผลลัพธ์ เพื่อความสะดวกในการใช้งาน

1.4 อุปกรณ์และเครื่องมือที่ใช้

1.4.1 ฮาร์ดแวร์

- คอมพิวเตอร์ประกอบตั้งโต๊ะ
    - หน่วยประมวลผลกลาง 12 th Gen Intel® Core™ i5-12400F 4.40 GHz
    - หน่วยความจำ ขนาด 16 GB
    - การ์ดจอ NVIDIA GeForce RTX3060
- คอมพิวเตอร์ประกอบตั้งโต๊ะ
    - หน่วยประมวลผลกลาง 10 th Gen Intel® Core™ i5-10400F 4.30 GHz
    - หน่วยความจำ ขนาด 16 GB
    - การ์ดจอ NVIDIA GeForce GTX1660 Super

1.4.2 ซอฟต์แวร์

- Visual Studio Code
- Android Studio
- ภาษาที่ใช้ คือ Dart , Python
- Framework ที่ใช้ คือ Flutter , Fast
- API ที่ใช้ คือ Fast API
- ไลบรารี คือ Scikit-learn, NLTK, Pandas
- Model คือ Deep Learning Models (Convolutional Neural Networks (CNNs))

1.5 ผลที่คาดว่าจะได้รับ

1) ได้แอปพลิเคชันที่สามารถบ่งบอกได้ว่ายาชนิดนั้นเป็นยาชนิดไหน ใช้รักษาโรคอะไร มีสรรพคุณและประโยชน์ของยาอย่างไร
2) ผู้ใช้ได้รับผลประโยชน์จากการใช้แอปพลิเคชันนี้เพื่อระบุได้ว่ายาชนิดที่ต้องการ ใช้รักษาโรคอะไร มีสรรพคุณหรือรายละเอียดของยาอย่างไร
3) ผู้จัดทำได้รับความรู้เรื่อง Deep Learning เพื่อนำไปพัฒนาแอปพลิเคชันที่สามารถตอบสนองความต้องการของผู้ใช้ในเรื่องการระบุประเภทของยาและสรรพคุณของยา

1.6 ภาระหน้าที่ความรับผิดชอบ

1) นาย อภิภัทร รุ่งเรือง

   <u>รับผิดชอบ</u> – รวบรวมข้อมูลยาทั้งภาษาไทยและภาษาอังกฤษ โดยเก็บข้อมูลดังนี้ ชื่อยา, โรคที่ยาสามารถใช้รักษาได้, สรรพคุณของยา, ประเภทของยา, ผลข้างเคียงยา นำมาสร้างเป็น Data sets ซึ่งถูกแบ่งเป็นหมวดหมู่

   - สร้างและออกแบบหน้า UI ของ Mobile Application

2) นาย อริญชัย นิลทะวงษ์

   <u>รับผิดชอบ</u> - นำ API ของ Google cloud - Vision AI ทำการ OCR ภาพออกมาเป็นตัวอักษร

   - สร้างเขื่อนไขในการ matches ชื่อของยาที่ผ่านกระบวนการ OCR แล้วมาเปรียบเทียบกับ Data sets
   - สร้างเขื่อนไขในการ ตรวจจับสรรพคุณของยา เพื่อนำมาจำแนกประเภทยา

3) ความรับผิดชอบร่วม

   <u>รับผิดชอบ</u> – ทำการแปลง Script Python เป็น Fast API เพื่อใช้กับ Mobile Application

   - การส่งข้อมูลเป็นรูปภาพ ผ่าน Fast API เพื่อส่งข้อมูลไปสู่กระบวนการ OCR เพื่อตรวจจับชื่อยาและจำแนกประเภทของยา
   - หน้าแสดงผลของ Mobile Application นำผลลัพธ์ที่ผ่านการตรวจจับและจำแนกยาแล้วมาแสดงผลในรูปแบบดังนี้ ชื่อยา, โรคที่ยาสามารถใช้รักษาได้, สรรพคุณของยา ,ประเภทของยา

บทที่ 2 ผลงานที่เกี่ยวข้อง

ในบทนี้จะแสดงถึงผลงานที่มีความเกี่ยวข้องหรือมีลักษณะคล้ายคลึงกับโครงการทั้งวิจัยและบทความเป็นต้นโดยจะทำการวิเคราะห์และทฤษฎีการทำงานของผลงานนั้น ๆ รวมทั้งข้อดีและข้อเสีย

2.1 Mobile search engine for medicine information on smartphone [1]

ในปัจจุบัน คนไทยส่วนใหญ่กำลังประสบปัญหาการใช้ยาด้วยตนเองโดยไม่มีข้อมูลที่ถูกต้อง เนื่องจากในประเทศไทย ผู้คนมักใช้ยาด้วยตนเองและซื้อยาที่ไม่ต้องมีใบสั่งแพทย์ การเลือกใช้ยาแบบสุ่มสี่สุ่มห้าอาจไม่เหมาะสมกับอาการป่วย บทความนี้นำเสนอเครื่องมือค้นหาข้อมูลยาบนสมาร์ทโฟน ซึ่งเป็นแอปพลิเคชันที่พัฒนาขึ้นเพื่อให้ข้อมูลเกี่ยวกับยาแก่ผู้ใช้งานทั่วไป ฟังก์ชันของแอปพลิเคชันถูกแบ่งออกเป็นสองเทคนิคการค้นหา ได้แก่ การค้นหาจากประเภทของยาและการค้นหาจากอาการป่วย ประเภทของยาสามารถแบ่งออกเป็นสี่ประเภท ได้แก่ ยาอันตราย ยาควบคุมพิเศษ ยาสามัญประจำบ้าน และยาทั่วไป บทความนี้นำเสนอการออกแบบและการพัฒนาแอปพลิเคชันโดยใช้ Android Studio ผลการทดลองแสดงให้เห็นว่าแอปพลิเคชันสามารถค้นหาชื่อยา รวมถึงชื่ออาการป่วย สรรพคุณ คำเตือน และวิธีการใช้ยาได้

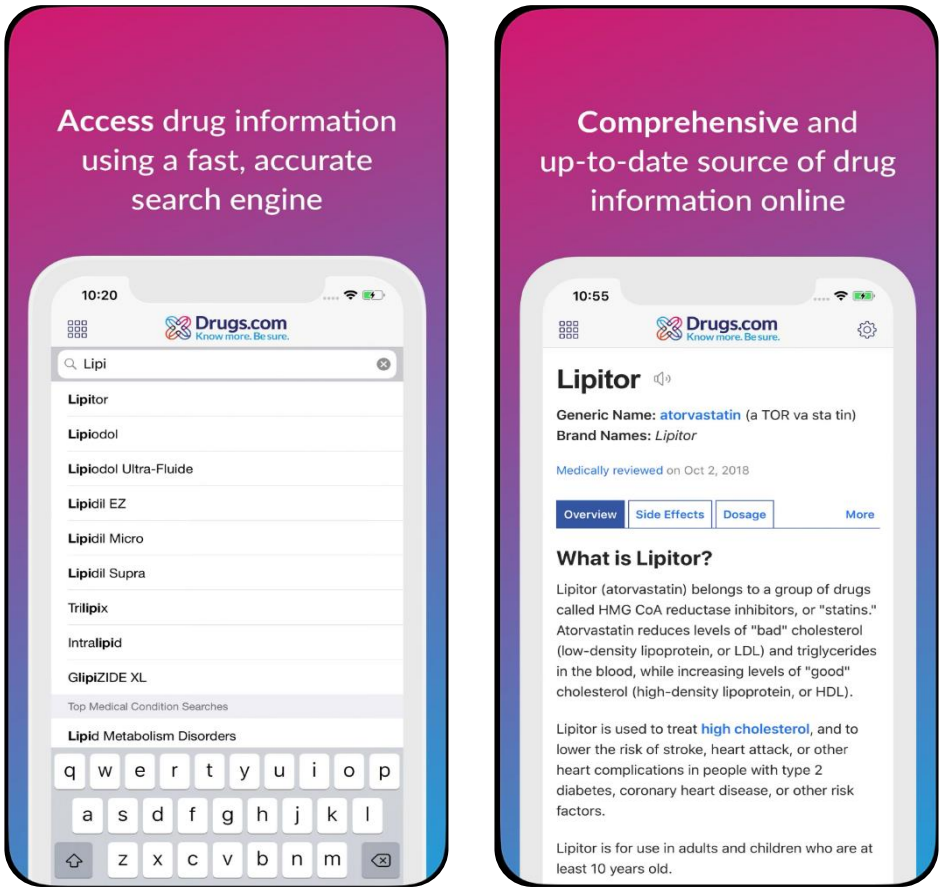
รูปที่ 1 ตัวอย่างของ Mobile search engine for medicine

จากรูปที่ 1 เป็นตัวอย่างของ Mobile Search Engine for Medicine ซึ่งแสดงถึงการออกแบบแอปพลิเคชันบนอุปกรณ์พกพาที่ใช้สำหรับการค้นหาข้อมูลทางการแพทย์ให้เข้าถึงข้อมูลที่สำคัญได้สะดวก เช่น การค้นหาข้อมูลยา

2.2 Application Development for Searching Drugs Information [2]

ความต้องการยาสูงเพื่อรักษาโรคต่าง ๆ ทำให้การพัฒนายาเติบโตอย่างรวดเร็วและก่อให้เกิดการสะสมข้อมูลยา การสะสมข้อมูลยาจำนวนมากนี้ทำให้การค้นหายากขึ้น ดังนั้นจึงจำเป็นต้องมีแอปพลิเคชันค้นหาที่ช่วยในการค้นหาข้อมูลยา ในการศึกษานี้ ได้พัฒนาแอปพลิเคชันค้นหาที่สามารถจัดการกับข้อมูลยาจำนวนมากได้อย่างง่ายดาย แอปพลิเคชันค้นหาถูกพัฒนาบนแพลตฟอร์มมือถือที่ใช้ระบบปฏิบัติการ Android โดยใช้เอกสารยารูปแบบ PDF เป็นแหล่งข้อมูล ข้อมูลจากเอกสารถูกดึงมา จัดทำดัชนี และเก็บไว้ในฐานข้อมูล กระบวนการค้นหาข้อมูลยาสามารถทำได้โดยการป้อนคำสำคัญ เช่น ชื่อยา ชื่อสามัญของยา สรรพคุณ และส่วนประกอบของยา

หลังการทดสอบพบว่า วิธีการเก็บข้อมูลโดยไม่จัดหมวดหมู่เหมาะสมกว่าการจัดหมวดหมู่ โดยพิสูจน์ได้จากเวลาที่ใช้ในการดำเนินการซึ่งรวดเร็วกว่าอยู่ที่ประมาณ 0.45-0.56 มิลลิวินาที สำหรับวิธีการค้นหาข้อมูลพบว่า การค้นหาด้วยคำสั่ง SQL เหมาะสมกว่าการใช้ FTS3 โดยพิสูจน์ได้จากค่า F1-Score ซึ่งอยู่ที่ 0.882 สำหรับการค้นหาด้วยคำสั่ง SQL และ 0.832 สำหรับการค้นหาด้วย FTS3



รูปที่ 2 ตัวอย่างการทำงานของ Application for Searching Drugs Information

จากรูปที่ 2 เป็นตัวอย่างของ Application for Searching Drugs Information มุ่งเน้นการพัฒนาระบบหรือแพลตฟอร์มที่ช่วยให้ผู้ใช้งาน ทั้งบุคลากรทางการแพทย์และประชาชนทั่วไป สามารถเข้าถึงข้อมูลเกี่ยวกับยาได้สะดวกและรวดเร็ว เช่น รายละเอียดเกี่ยวกับชื่อยาทั่วไป , ชื่อการค้า , กลไกการออกฤทธิ์, ข้อบ่งใช้, ผลข้างเคียง, และปฏิกิริยาของยา

2.3 Google Lens [3]

Google Lens เป็นแอปพลิเคชันที่ให้ข้อมูลเกี่ยวกับวัตถุและสถานที่ในโลกแห่งความเป็นจริงรอบตัวคุณ โดยGoogle Lens ใช้การรู้ จำภาพและปัญญาประดิษฐ์เพื่อค้นหาข้อมูลที่มีประโยชน์เกี่ยวกับภาพที่คุณแสดงให้มันดู ตัวอย่างเช่น คุณสามารถถ่ายรูปพืชหรือต้นไม้แล้วใช้ Google Lens เพื่อระบุชื่อของมันหรือถ่ายรูปป้ายร้านอาหารแล้วรับข้อมูลเช่นเวลาเปิดทำการและเมนูได้ มันยังสามารถแปลข้อความภาษาต่างประเทศให้คุณได้อีกด้วย สรุปได้ว่า Google Lens เหมือนกับการนำพลังของเครื่องมือค้นหาของ Google มาใช้กับโลกที่อยู่รอบตัวของคุณ

2.4 เปรียบเทียบความแตกต่างของผลงาน

ตารางที่ 2.4 ตารางเปรียบเทียบความแตกต่างของผลงาน

	ใช้กล้องในการแสกนหาชื่อยา	ให้ข้อมูลของยาครบในการใช้งาน	ใช้ในการแยกประเภทของยากจากการสแกนจากภาพ
Application Development for Searching Drugs Information	✗	✓	✗
Google Lens	✗	✗	✗
Mobile search engine for medicine information on smartphone	✗	✓	✗
แอปพลิเคชันบนโทรศัพท์มือถือสำหรับตรวจจับชื่อยาและจำแนกประเภทของยา	✓	✓	✓

จากตารางที่ 2.4 ผลเปรียบเทียบจะเห็นได้ว่า แอปพลิเคชันบนโทรศัพท์มือถือสำหรับตรวจจับชื่อยาและจำแนกประเภทของยา มีความสามารถใช้กล้องในการสแกนหา ชื่อยา,ให้ข้อมูลของยาครบถ้วนในการใช้งาน,ใช้ในการแยกประเภทของยาจากการสแกนภาพ ในขณะที่ผลงานอื่น ๆ นั้นมีข้อจำกัดที่ไม่สามารถใช้กล้องหรือแยกประเภทของยาได้จากภาพ

บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง

ในบทนี้จะแสดงรายละเอียดของทฤษฎีและความรู้ที่เกี่ยวข้องที่จะใช้ในการศึกษาเพิ่มเติมเพื่อเป็นแนวทางในการพัฒนาโครงงานวิจัยนี้ โดยมีเนื้อหารายละเอียดดังนี้

3.1 OCR (Optical character recognition) [4]

OCR คืออะไร เป็นกระบวนการที่แปลงภาพข้อความให้เป็นรูปแบบข้อความที่เครื่องอ่านได้ตัวอย่างเช่น หากคุณสแกนแบบฟอร์มหรือใบเสร็จ คอมพิวเตอร์ของคุณจะบันทึกการสแกนดังกล่าวเป็นไฟล์รูปภาพ คุณไม่สามารถใช้ตัวแก้ไขข้อความเพื่อแก้ไข ค้นหา หรือนับคำในไฟล์รูปภาพได้ อย่างไรก็ตาม คุณสามารถใช้ OCR เพื่อแปลงรูปภาพเป็นเอกสารข้อความที่มีการจัดเก็บเนื้อหาเป็นข้อมูลตัวอักษรได้

OCR ทำงานอย่างไร ซอฟต์แวร์ OCR ทำงานโดยมีขั้นตอนดังต่อไปนี้

การรับภาพ

ตัวสแกนจะอ่านเอกสารและแปลงเป็นข้อมูลไบนารี จากนั้นซอฟต์แวร์ OCR จะวิเคราะห์ภาพที่สแกนแลระบุส่วนที่สว่างเป็นพื้นหลัง และส่วนที่มืดเป็นข้อความ

กระบวนการก่อนการประมวลผล

ซอฟต์แวร์ OCR จะทำการปรับปรุงคุณภาพของภาพโดยการลบข้อผิดพลาดและสิ่งรบกวนออก เพื่อเตรียมภาพให้พร้อมสำหรับการอ่านข้อมูล

การรู้จำข้อความ

อัลกอริทึมหรือกระบวนการด้านซอฟต์แวร์ OCR หลักสองประเภทที่ซอฟต์แวร์ OCR ใช้ในการรู้จำข้อความเรียกว่าการจับคู่รูปแบบและการแยกลักษณะ

การจับคู่รูปแบบ

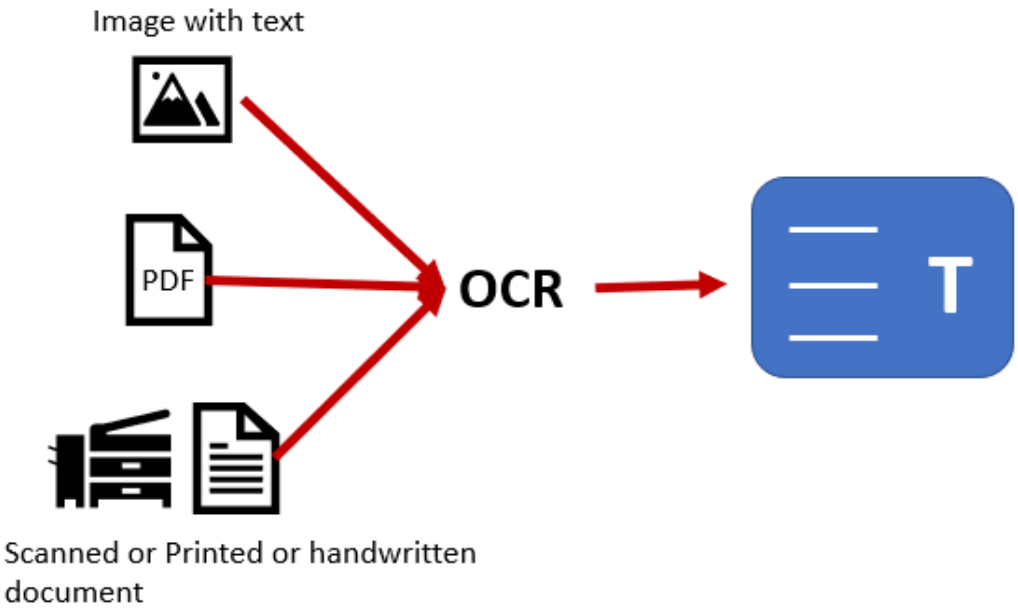
การจับคู่รูปแบบทำงานโดยการแยกภาพอักขระที่เรียกว่ารูปอักษร และเปรียบเทียบกับรูปอักษรที่จัดเก็บไว้ในลักษณะเดียวกัน การรู้จำรูปแบบจะทำงานได้ก็ต่อเมื่อรูปอักษรที่จัดเก็บไว้มีแบบอักษรและมาตราส่วนใกล้เคียงกับรูปอักษรที่ใช้ โดยวิธีการนี้ใช้ได้ดีกับรูปภาพที่สแกนของเอกสารที่พิมพ์ด้วยแบบอักษรที่เป็นที่รู้จัก

การแยกลักษณะ

การแยกลักษณะจะแบ่งหรือแยกย่อยรูปอักษรออกเป็นคุณสมบัติต่าง ๆ เช่น เส้น วงปิด ทิศทางของเส้น และจุดตัดของเส้น จากนั้นจึงใช้คุณสมบัติเหล่านี้เพื่อค้นหาคู่ที่เหมาะสมที่สุดหรือตำแหน่งข้างเคียงที่ใกล้ที่สุดในบรรดารูปอักษรต่าง ๆ ที่จัดเก็บไว้

กระบวนการหลังการประมวลผล

หลังจากการวิเคราะห์ ระบบจะแปลงข้อมูลตัวอักษรที่แยกออกมาเป็นไฟล์ที่ใช้ระบบคอมพิวเตอร์ ระบบ OCR บางส่วนสามารถสร้างไฟล์ PDF ที่มีคำอธิบายประกอบซึ่งมีทั้งเวอร์ชันก่อนและหลังของเอกสารที่สแกนได้



รูปที่ 3 การทำงานของ Optical Character Recognition

จากรูปที่ 3 แสดงให้เห็นถึงขั้นตอนการทำงานของระบบ OCR (Optical Character Recognition) ซึ่งเป็นเทคโนโลยีที่ใช้ในการแปลงข้อมูลในรูปแบบของภาพหรือเอกสารเป็นข้อความ OCR ทำหน้าที่ประมวลผลรูปภาพหรือเอกสารเพื่อตรวจจับตัวอักษรและคำที่อยู่ในข้อมูลภาพหรือไฟล์นั้น

3.2 Natural language processing (NLP) [5]

การประมวลผลภาษาธรรมชาติ (NLP) เป็นเทคโนโลยี Machine Learning ช่วยให้คอมพิวเตอร์สามารถตีความ จัดการ และทำความเข้าใจภาษามนุษย์ได้ องค์กรในปัจจุบันมีข้อมูลเสียงและข้อความจำนวนมากจากช่องทางการสื่อสารต่าง ๆ เช่น อีเมล ข้อความ ฟีดข่าวโซเชียลมีเดีย วิดีโอ เสียง และอื่น ๆ พวกเขาใช้ซอฟต์แวร์ NLP เพื่อประมวลผลข้อมูลนี้โดยอัตโนมัติ วิเคราะห์เจตนาหรือความเชื่อมั่นในข้อความ และตอบสนองการสื่อสารของมนุษย์แบบเรียลไทม์ [5]

NLP ทำงานอย่างไร

การประมวลผลภาษาธรรมชาติ (NLP) นำภาษาการคำนวณ Machine Learning และโมเดล Deep learning มารวมกันในการประมวลผลภาษามนุษย์

ภาษาศาสตร์เชิงคำนวณ

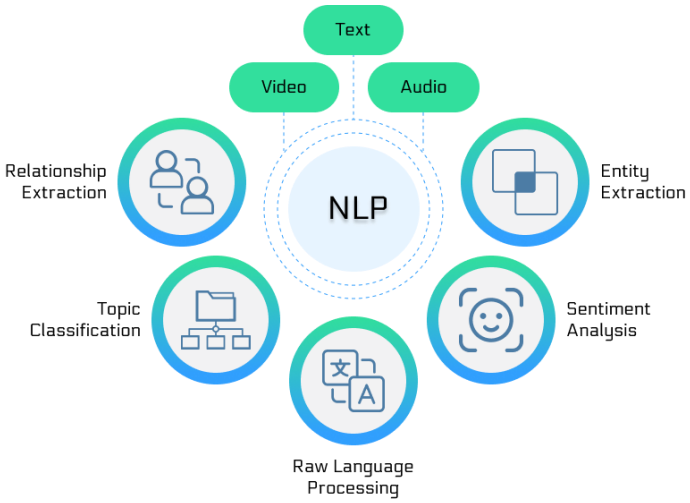
เป็นศาสตร์แห่งการทำความเข้าใจและสร้างโมเดลภาษามนุษย์ด้วยคอมพิวเตอร์และเครื่องมือซอฟต์แวร์ นักวิจัยใช้วิธีการทางภาษาศาสตร์เชิงคำนวณ เช่น การวิเคราะห์ในเชิงโครงสร้างและความหมาย เพื่อสร้าง Framework ที่ช่วยให้เครื่องเข้าใจภาษามนุษย์ในการสนทนา เครื่องมือต่าง ๆ เช่น ตัวแปลภาษา เทคโนโลยีสังเคราะห์เสียงจากข้อความ และซอฟต์แวร์สำหรับจำเสียงพูดล้วนมีพื้นฐานมาจากภาษาศาสตร์เชิงคำนวณ

Machine Learning [6]

เป็นเทคโนโลยีที่ฝึกคอมพิวเตอร์ด้วยข้อมูลตัวอย่างเพื่อปรับปรุงประสิทธิภาพ ภาษามนุษย์มีคุณลักษณะหลายอย่าง เช่น คำเสียดสี คำอุปมา โครงสร้างประโยคที่แปรผัน บวกกับข้อยกเว้นด้านไวยากรณ์และการใช้งานที่มนุษย์ต้องใช้เวลาหลายปีในการเรียนรู้ โปรแกรมเมอร์ใช้ Machine Learning เพื่อสอนแอปพลิเคชัน NLP ให้รู้จักและเข้าใจฟีเจอร์เหล่านี้อย่างถูกต้องตั้งแต่เริ่มต้น

Deep learning [7]

เป็นสาขาวิชาเฉพาะของ Machine Learning ซึ่งสอนคอมพิวเตอร์ให้เรียนรู้และคิดเหมือนมนุษย์ โดยเกี่ยวข้องกับ Neural Networks ที่ประกอบด้วย Node ประมวลผลข้อมูลที่มีโครงสร้างคล้ายกับสมองมนุษย์ ด้วย Deep learning คอมพิวเตอร์จะรับรู้ จำแนกและเชื่อมโยงรูปแบบที่ซับซ้อนในข้อมูลอินพุต



รูปที่ 4 การทำงานของการประมวลผลภาษาธรรมชาติ (NLP)

จากรูปที่ 4 แสดงคือแผนภาพที่อธิบายถึงการทำงานของ Natural Language Processing โดยเน้นไปที่กระบวนการทำงานหลักและการใช้งาน Natural Language Processing สามารถประมวลผลข้อมูลจากแหล่งข้อมูลหลากหลายรูปแบบ

3.3 Text classification [8]

Text classification เป็นกระบวนการกำหนดหมวดหมู่ที่มีการกำหนดไว้ล่วงหน้าให้กับเอกสารข้อความปลายเปิด โดยใช้ระบบ AI และ Machine Learning (AI/ML) องค์กรหลายแห่งมีคลังเอกสารขนาดใหญ่และระบบ workflow ทางธุรกิจที่สร้างเอกสารในปริมาณมากอย่างต่อเนื่อง เช่น เอกสารทางกฎหมาย สัญญา เอกสารการวิจัย ข้อมูลที่ผู้ใช้สร้างขึ้น และอีเมล การจัดประเภทข้อความคือขั้นตอนแรกในการจัดระเบียบ จัดโครงสร้าง และจัดหมวดหมู่ข้อมูลนี้เพื่อการวิเคราะห์เพิ่มเติม ซึ่งช่วยให้สามารถติดป้ายกำกับและติดแท็กเอกสารได้โดยอัตโนมัติ วิธีนี้จะช่วยประหยัดเวลาในองค์กรของคุณได้หลายพันชั่วโมง มิฉะนั้นคุณจะต้องอ่าน ทำความเข้าใจ และจัดประเภทเอกสารด้วยตนเอง

ประโยชน์ของ Text classification มีอะไรบ้าง

ปรับปรุงความถูกต้อง

โมเดลการจำแนกประเภทข้อความจัดหมวดหมู่ข้อความได้อย่างถูกต้องโดยมีการฝึกเพิ่มเติมเล็กน้อยหรือแทบจะไม่มีเลย โมเดลเหล่านี้จะช่วยให้องค์กรเอาชนะข้อผิดพลาดที่มนุษย์อาจทำให้เกิดขึ้นได้เมื่อจำแนกข้อมูลที่เป็นข้อความด้วยตนเอง นอกจากนี้ ระบบการจำแนกข้อความที่มีความสอดคล้องมากกว่ามนุษย์เมื่อกำหนดแท็กให้กับข้อมูลข้อความในหัวข้อต่าง ๆ

มอบการวิเคราะห์แบบเรียลไทม์

องค์กรต่าง ๆ เผชิญกับแรงกดดันด้านเวลาเมื่อต้องประมวลผลข้อมูลข้อความแบบเรียลไทม์ เมื่อใช้อัลกอริทึมการจัดประเภทข้อความ คุณจะสามารถดึงข้อมูลเชิงลึกที่นำไปปฏิบัติได้จากข้อมูลดิบและกำหนดการตอบสนองได้ทันที ตัวอย่างเช่น องค์กรสามารถใช้ระบบการจำแนกประเภทข้อความเพื่อวิเคราะห์ความคิดเห็นของลูกค้าและตอบสนองต่อคำขอเร่งด่วนได้ทันที

ปรับขนาดงานการจำแนกประเภทข้อความ

ก่อนหน้านี้องค์กรต่าง ๆ อาศัยระบบที่ดำเนินการด้วยตนเองหรือดำเนินการตามกฎเพื่อจัดประเภทเอกสาร วิธีการเหล่านี้มีความล่าช้าและใช้ทรัพยากรมากเกินไป เมื่อใช้การจำแนกประเภทข้อความของ Machine Learning คุณจะสามารถขยายความพยายามในการจัดหมวดหมู่เอกสารข้ามแผนกต่าง ๆ ได้อย่างมีประสิทธิภาพมากขึ้นเพื่อรองรับการเติบโตขององค์กร

แปลภาษา

องค์กรสามารถใช้ตัวจำแนกประเภทข้อความเพื่อตรวจหาภาษาได้ โมเดลการจัดประเภทข้อความสามารถตรวจจับภาษาต้นทางในการสนทนาหรือคำขอบริการ และส่งไปยังทีมงานที่เกี่ยวข้อง

กรณีการใช้งานของ Text classification

การจัดการเอกสาร

หลายองค์กรเผชิญกับความท้าทายในการประมวลผลและจัดเรียงเอกสารในการสนับสนุนการดำเนินธุรกิจ ตัวจำแนกประเภทข้อความสามารถตรวจจับข้อมูลที่ขาดหายไป แยกคำหลักแบบเฉพาะเจาะจง และระบุความสัมพันธ์ทางความหมายได้ คุณสามารถใช้ระบบการจำแนกประเภทข้อความเพื่อติดป้ายกำกับและจัดเรียงเอกสาร เช่น ข้อความ บทวิจารณ์ และสัญญาตามหมวดหมู่ต่าง ๆ ได้

3.4 Long Short-Term Memory [9]

LSTM ย่อจาก Long Short-Term Memory ถือเป็นประเภทหนึ่งของสถาปัตยกรรมแบบ Recurrent Neural Network (RNN) อยู่ในกลุ่มของ Deep Learning ถูกออกแบบให้จดจำ Patterns ในช่วงเวลานาน ๆ มีประสิทธิภาพสำหรับปัญหาการทำนายที่เป็น Sequential เนื่องจากสามารถเก็บข้อมูลก่อนหน้าและนำมาร่วมใช้ในการประมวลผลได้ สามารถแก้ปัญหา Long-term Dependency ได้ โดย RNN แบบดั้งเดิมจะเผชิญกับ ความท้าทายในเรื่อง Long-range Dependency และมีปัญหา Vanishing Gradient

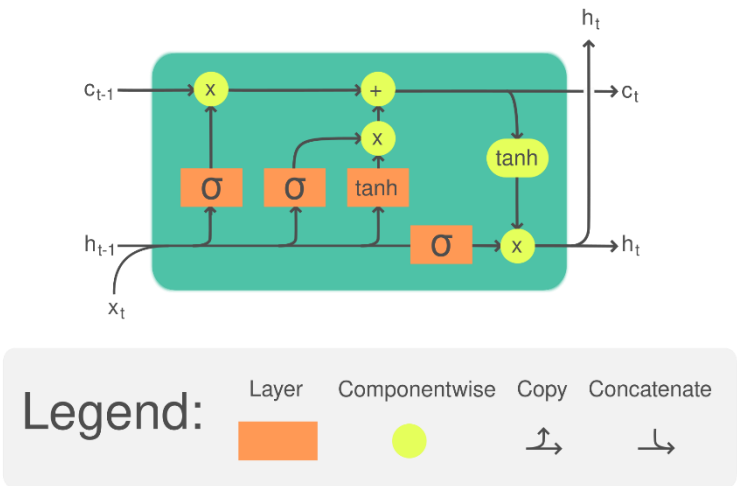
LSTM ถูกออกแบบมาให้จดจำ Long-term Information โดยใช้ Gating Mechanisms ที่ออกแบบมาเฉพาะมีความยืดหยุ่นที่ดี LSTM สามารใช้ Model ได้ทั้ง Long-term และ Short-term Temporal Sequences

ข้อดี

- ความสามารถในการจดจำ การจดจำ Patterns จาก Long Sequences ทำให้เหมาะกับ Applications ส่วนใหญ่ในทางปฏิบัติ
- มีการนำไปใช้งานอย่างแพร่หลาย
- มี Libraries ต่าง ๆ รองรับ เช่น TensorFlow และ Keras ทำให้ง่ายต่อการ Implementation

ข้อเสีย

- ความซับซ้อนในการคำนวณ การ Train LSTMs ใช้เวลาและทรัพยากรในการคำนวณสูง
- ต้องใช้ Dataset จำนวนมาก สำหรับการ Train Model ที่มีประสิทธิภาพ และหลีกเลี่ยง Overfitting จำเป็นต้องใช้ Data จำนวนมาก
- Hyperparameter Tuning เหมือนกับ Neural Network อื่น ๆ หากต้องการได้ประสิทธิภาพที่ดี จำเป็นต้องใช้ความพยายามกับการทำ Hyperparameter Tuning



รูปที่ 5 Long Short-Term Memory

จากรูปที่ 5 แสดงถึงโครงสร้างของ Long Short-Term Memory หรือ LSTM ซึ่งเป็นส่วนหนึ่งของโครงข่ายประสาทเทียมชนิด Recurrent Neural Network ที่ถูกออกแบบมาเพื่อแก้ปัญหาการเรียนรู้ลำดับข้อมูลในระยะยาว โดยสามารถอธิบายส่วนต่าง ๆ ในรูปได้ดังนี้ องค์ประกอบหลักของ LSTM, กระบวนการทำงานภายใน LSTM, คำอธิบายสัญลักษณ์ในรูป (Legend), บทบาทของ LSTM

3.5 Naive Bayes [10]

Naive Bayes Classifiers คือตัวจำแนกประเภทแบบ Naive Bayes คือชุดของอัลกอริทึมการจำแนกประเภทที่อิงตามทฤษฎีบทของเบย์สมันไม่ใช่อัลกอริทึมเดี่ยวแต่เป็นกลุ่มอัลกอริทึมที่ทุกอัลกอริทึมมีหลักการร่วมกันนั่นคือคุณลักษณะแต่ละคู่ที่ถูกจำแนกประเภทนั้นเป็นอิสระจากกัน ในการเริ่มต้น ให้เรามาพิจารณาชุดข้อมูลกันก่อน

ตัวจำแนกแบบ Naive Bayes เป็นหนึ่งในอัลกอริทึมการจำแนกที่เรียบง่ายและมีประสิทธิภาพที่สุด ซึ่งช่วยในการพัฒนาโมเดลการเรียนรู้ของเครื่องอย่างรวดเร็วโดยมีความสามารถในการทำนายอย่างรวดเร็ว

อัลกอริทึม Naive Bayes ใช้สำหรับปัญหาการจำแนกประเภท โดยมักใช้ในการจำแนกประเภทข้อความ ในงานจำแนกประเภทข้อความ ข้อมูลจะมีมิติสูง (เนื่องจากแต่ละคำแสดงถึงคุณลักษณะหนึ่งในข้อมูล) อัลกอริทึมนี้ใช้ในการกรองสแปม การตรวจจับความรู้สึก การจำแนกประเภทเรตติ้ง เป็นต้น ข้อดีของการใช้ Naïve Bayes คือความเร็วรวดเร็ว และทำนายได้ง่ายเมื่อมีข้อมูลที่มีมิติสูง

แบบจำลองนี้ทำนายความน่าจะเป็นที่อินสแตนซ์จะอยู่ในคลาสที่มีค่าคุณลักษณะชุดหนึ่งที่กำหนดไว้ แบบจำลองนี้เป็นตัวจำแนกประเภทแบบความน่าจะเป็น เนื่องจากแบบจำลองนี้ถือว่าคุณลักษณะหนึ่งในแบบจำลองนั้นไม่ขึ้นอยู่กับการมีอยู่ของคุณลักษณะอื่น กล่าวอีกนัยหนึ่ง คุณลักษณะแต่ละประการมีส่วนสนับสนุนการทำนายโดยไม่มีความสัมพันธ์ระหว่างกัน ในโลกแห่งความเป็นจริง เงื่อนไขนี้ไม่ค่อยเป็นไปตามนั้น แบบจำลองนี้ใช้ทฤษฎีบทเบย์สในอัลกอริทึมสำหรับการฝึกและทำนาย

บทที่ 4 ขั้นตอนและแผนการดำเนินงาน

สำหรับขั้นตอนและแผนการดำเนินงาน จะกำหนดการวางแผนตามขั้นตอนดังนี้

4.1 ขั้นตอนการดำเนินงาน

4.1.1 คิดหัวข้อโครงงานและรวบรวมข้อมูล

- เลือกหัวข้อที่สนใจ และรวบรวมข้อมูลที่เกี่ยวข้องโครงงาน

4.1.2 รวบรวมข้อมูลและกำหนดขอบเขตของโครงงาน

- กำหนดขอบเขตของโครงงานตามความเหมาะสม
- รวบรวมข้อมูล Data set จากเว็บไซต์ที่หน้าเชื่อถือ
- รวบรวมข้อมูลที่มีความเกี่ยวข้องกับโครงงานนี้ได้แก่ผลงาน งานวิจัยและทฤษฎี
- วางแผนการดำเนินงานและระยะเวลาในการดำเนินงาน

4.1.3 ศึกษาทฤษฎีและความรู้ที่เกี่ยวข้อง

- ศึกษาการทำ OCR
- ศึกษาเรื่อง NLP
- ศึกษาเรื่อง Text classification
- ศึกษาเรื่อง machine learning
- ศึกษาเรื่อง ภาษา dart
- ศึกษาการทำ Mobile application
- ศึกษาการดึง Api ของ Text-to-speech มาใช้

4.1.4 การออกแบบระบบและพัฒนาโปรแกรม

- ออกแบบการทำโมเดล
- ออกแบบ UX และUI
- ออกแบบระบบการทำงานของ Mobile application
- การเชื่อมต่อระบบหลังบ้านเข้ากับตัว Mobile application

4.1.5 ทดสอบและปรับปรุง

- ทดสอบการทำ OCR ของโมเดล
- ทดสอบการทำจัดประเภทของความของโมเดล
- ทดสอบการใช้งานแอพพลิเคชั่น
- แก้ไขข้อผิดพลาดของแอพพลิเคชั่น

4.1.6 จัดทำเอกสารประกอบโครงงาน

- จัดทำรายละเอียดของโครงงานทั้งหมด ตั้งแต่ขั้นตอนเริ่มต้นจนถึงขั้นตอนสุดท้ายของโครงงาน

4.2 แผนการดำเนินงาน

ตารางที่ 4.1 แผนการดำเนินงาน

แผนการดำเนินงาน	เดือน						
	เมษายน 2567	พฤษภาคม 2567	มิถุนายน 2567	กรกฎาคม 2567	สิงหาคม 2567	กันยายน 2567	ตุลาคม 2567
1.คิดหัวข้อโครงงานและรวบรวมข้อมูล							
2.รวบรวมข้อมูลและกำหนดขอบเขตของโครงงาน							
3.ศึกษาทฤษฎีและความรู้ที่เกี่ยวข้อง							
4.การออกแบบระบบและพัฒนาโปรแกรม							
5.ทดสอบและปรับปรุง							
6.จัดทำเอกสารประกอบโครงงาน							

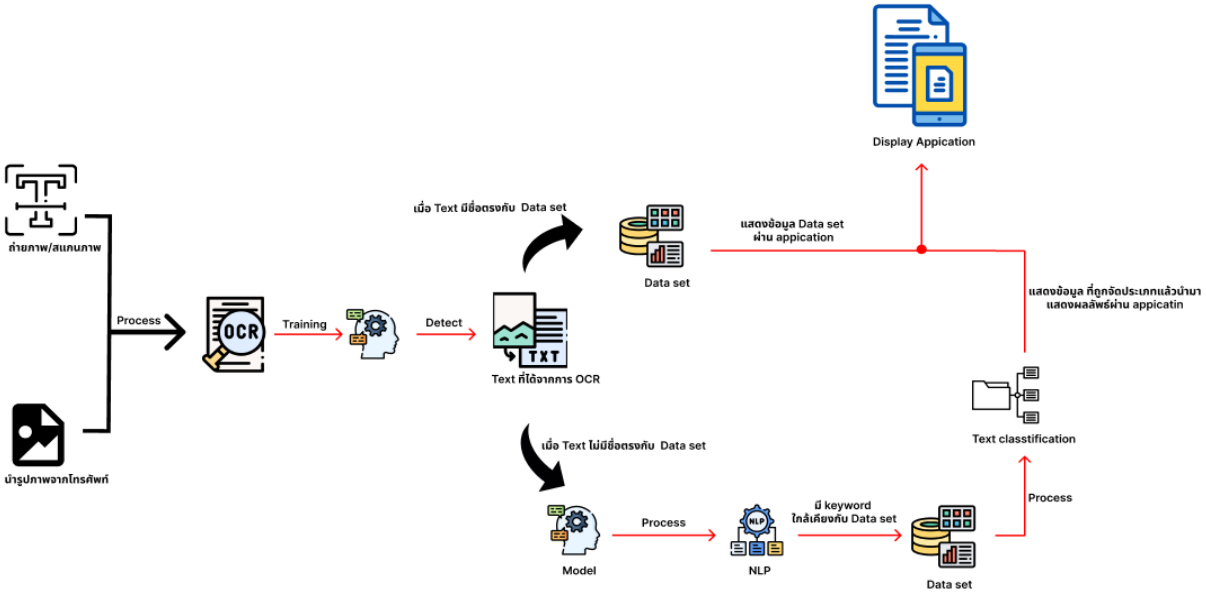
จากตารางที่ 4.1 เป็นการแสดงระยะเวลาในการทำงานวิจัยนี้ว่าขั้นตอนการพัฒนาอย่างไรบ้างและเกิดขึ้นในช่วงเวลาใด

บทที่ 5 การวิเคราะห์และออกแบบระบบ

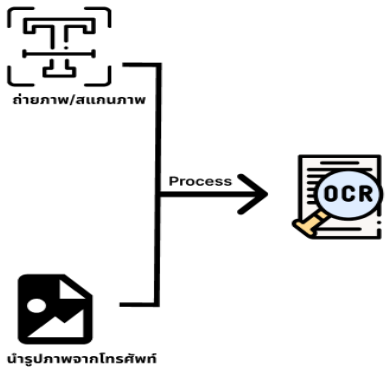
5.1 โครงสร้างของระบบ

ระบบนี้จะแบ่งออกเป็น 2 ส่วนที่สำคัญคือ

1. ส่วนของการทำโมเดลระบบ Optical Character Recognition (OCR) จะวิธีการดำเนินงานดังนี้
   1.1 การทำระบบโดยใช้ข้อมูลรูปภาพกล่องยาหรือฉลากยาจากฐานข้อมูล Data set เข้าสู่โมเดลในการดึง Text ชื่อยาออกมาจากภาพ โดยการนำ API ของ Google cloud - Vision AI ใช้เป็นเครื่องมือในการประมวลผลภาพและแสดงออกมาเป็น Text เพื่อนำไปวิเคราะห์ต่อไป
   1.2 เทรนโมเดลโดยใช้วิธี adaptive thresholding เพื่อเพิ่มความแม่นยำในการดึงข้อความที่เป็นจุดของชื่อยา
   1.3 เมื่อทำการดึงชื่อยาออกมาแล้วระบบจะตรวจสอบกับฐานข้อมูล Data set ว่าชื่อตรงกับตัวยาที่มีใน Data set หรือถ้ามีระบบจะแสดงผลลัพธ์ทันที โดยผลลัพธ์จะอยู่ในรูปแบบดังนี้ ชื่อยา, ประเภทของยา, สรรพคุณของยา, ผลข้างเคียงยา
2. ส่วนนี้จะเป็นกรณีที่ไม่มีชื่อยาในฐานข้อมูลจะนำตัวยานั้นที่ไม่มีในฐานข้อมูลมาสแกนตัวสรรพคุณของตัวฉลากยาข้างกล่องมีวิธีการดำเนินงานดังนี้
   2.1 ใช้ข้อมูลจากสรรพคุณที่ได้จากการ OCR มาเทรนโมเดลโดยใช้เทคนิค NLP ในการจับคำจากสรรพคุณและแยกประเภทของคลาสโดยใช้ไลบรารี่ NLTK ในการจัดประเภทของคำจากนั้นระบบจัดประเภทของข้อคำนั้นว่ายาชนิดนี้เป็นยาประเภทไหนและสามารถใช้กับโรคอะไรได้บ้างจึงจะแสดงผลลัพธ์ออกมา
3. ในส่วนของการจำแนกประเภทนั้นใช้ 2 โมเดลนั้นคือ Naïve Bayes และ Long Short-Term Memory ในการจำแนกประของยาจากสรรพคุณและซึ่งในการใช้โมเดล Naïve Bayes ในการจำแนกสรรพคุณยาภาษาไทยเนื่องจากข้อมูลที่มีจำนวน 120 ตัวยาดังนั้นจึงเลือกใช้ Naïve Bayes ในการจำแนกมีจำนวน 5 คลาสแต่ละคลาสคือมีประเภทยาดังนี้ ยาฆ่าเชื้อ, ยาระงับประสาท, ยาแก้ปวด, ยาแก้แพ้, ยาแก้ไอ ขับเสมหะ ในส่วนโมเดล Long Short-Term Memory นั้นจะจะใช้ในการจำแนกสรรพคุณยาภาษาอังกฤษเนื่องจากข้อมูลที่มีจำนวน 191517 ตัวยาดังนั้นจึงเลือกใช้ Long Short-Term Memory ในการจำแนกมีจำนวน 8 คลาสประด้วยประเภทยาดังนี้ ANTI INFECTIVES, CARDIAC , DERMA , GASTRO INTESTINAL, NEURO CNS, OTHERS, PAIN ANALGESICS , RESPIRATORY



รูปที่ 6 โครงสร้างระบบ



รูปที่ 7 โครงสร้างระบบ แสดงการนำรูปผ่านกระบวนการ OCR

จากรูปที่ 7 นี้แสดงกระบวนการ OCR (Optical Character Recognition) จากได้ครับภาก่อนนำไป OCR นั้นคือการถ่ายภาพหรือการเลือกรูปจากโทรเราเพื่อเข้าสู่กระบวนกการ OCR

ตั้งค่าคีย์ API ของ Google Cloud

```
os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = r'project-service.json'
client = vision_v1.ImageAnnotatorClient()
```

ตั้งค่าพาธของไฟล์รูปภาพ

```
file_name = 'p.jpg'
image_folder = 'C:/Users/User/google_vision_ai_demo/'
image_path = os.path.join(image_folder, file_name)
```

อ่านข้อมูลรูปภาพ

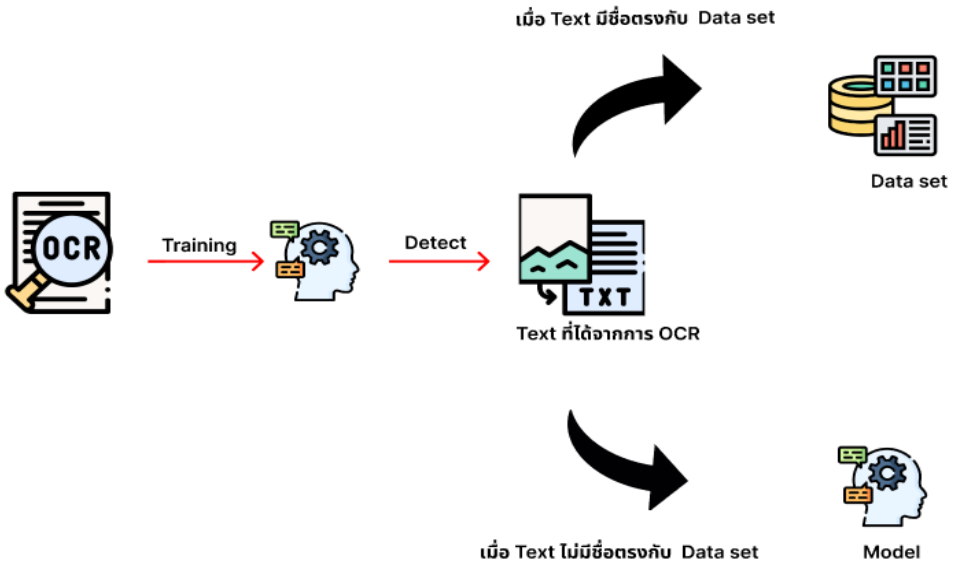
```
with io.open(image_path, 'rb') as image_file:
content = image_file.read()
image = vision_v1.types.Image(content=content)
```

ระบุคำใบ้ภาษา (ภาษาไทยและภาษาอังกฤษ)

```
image_context = vision_v1.types.ImageContext(
    language_hints=['th', 'en']
)
```

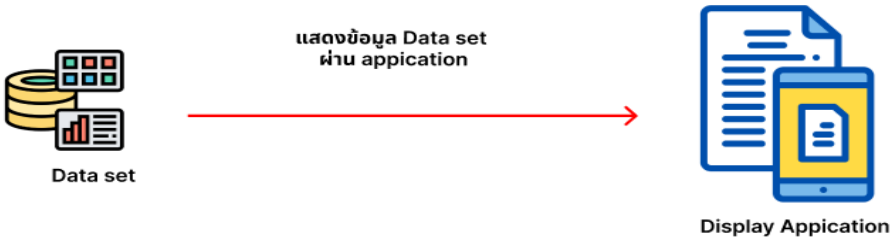
ทำการตรวจจับข้อความในรูปภาพ

```
response = client.text_detection(image=image, image_context=image_context)
texts = response.text_annotations
```



รูปที่ 8 โครงสร้างระบบ แสดงการตรวจจับและจำแนกข้อมูล
จะแบ่งแยกเป็น 2 กรณี คือ เมื่อพบข้อมูลตรงกับ Data set และไม่พบข้อมูลตรงกับ Data set

จากรูปที่ 8 นี้แสดงถึงการนำตัว Text ที่ได้จากการ OCR มาเข้ากระบวนการตรวจโดยแยกออกเป็น 2 แบบโดยที่ได้



รูปที่ 9 โครงสร้างระบบ แสดงการทำงานเมื่อ Text มีชื่อตรงกับ Data set

จากรูปที่ 9 นี้จะแสดงให้เห็นต่อจาก flowที่แล้วว่าเมื่อเกิดว่าตรวจจับชื่อยาได้ที่มีขอมูลใน Data set ตัวแอปนั้นจะพาไปสู่หน้า output ซึ่งจะแสดงข้อมูลขอตัวยานั้น

กรองและรวมข้อความจาก OCR

```
ocr_texts = [filter_alnum(text.description) for text in texts if filter_alnum(text.description)]
```

แปลงข้อความเป็น vectors โดยใช้ TfidfVectorizer

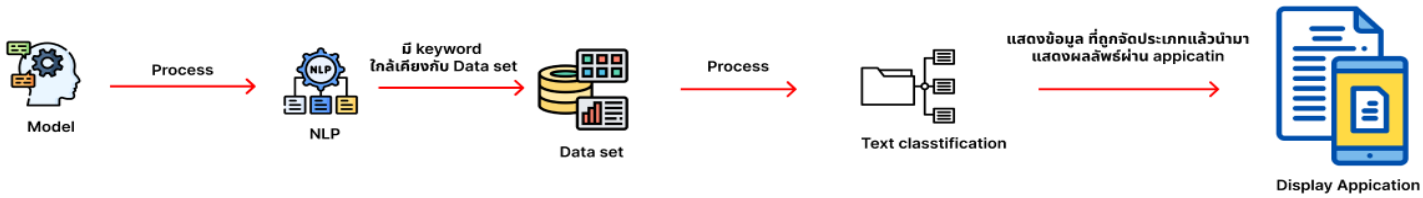
```
vectorizer = TfidfVectorizer()
vectors = vectorizer.fit_transform(data['name'].dropna().unique())
ocr_vector = vectorizer.transform([' '.join(ocr_texts)])
```

คำนวณค่า cosine similarity ระหว่าง vectors

```
cosine_similarities = cosine_similarity(ocr_vector, vectors).flatten()
highest_similarity_index = cosine_similarities.argmax()
highest_similarity_value = cosine_similarities[highest_similarity_index]
```

กำหนดค่า similarity ที่เหมาะสม

```
similarity_threshold = 0.5
if highest_similarity_value >= similarity_threshold:
  best_match = data['name'].dropna().unique()[highest_similarity_index]
  matches = data[data['name'].str.contains(best_match, case=False, na=False)]
  if not matches.empty:
        print(matches.iloc[0])
```



รูปที่ 10 โครงสร้างระบบ แสดงขั้นตอนการทำงานในกรณีที่เมื่อ Text ไม่ตรงกับ Data set
จะทำการตรวจจับสรรพคุณที่ มี keyword ใกล้เคียงกับ Data set และทำการ Classification

จากรูปที่ 10 นี้จะแสดงให้เห็นว่าตรวจจับชื่อยาไม่ได้นั้นตัวแอปจะทำการนำตัว Data set และตัว text ทำได้จากการ OCR มาเทรนโมเดลเพื่อที่จะให้ตัวแอปนั้นจำแนกประเภทของยาแล้วนำข้อมูลเมื่อจำแนกแล้วไปแสดงหน้า output ให้แก่ผู้ใช้งาน

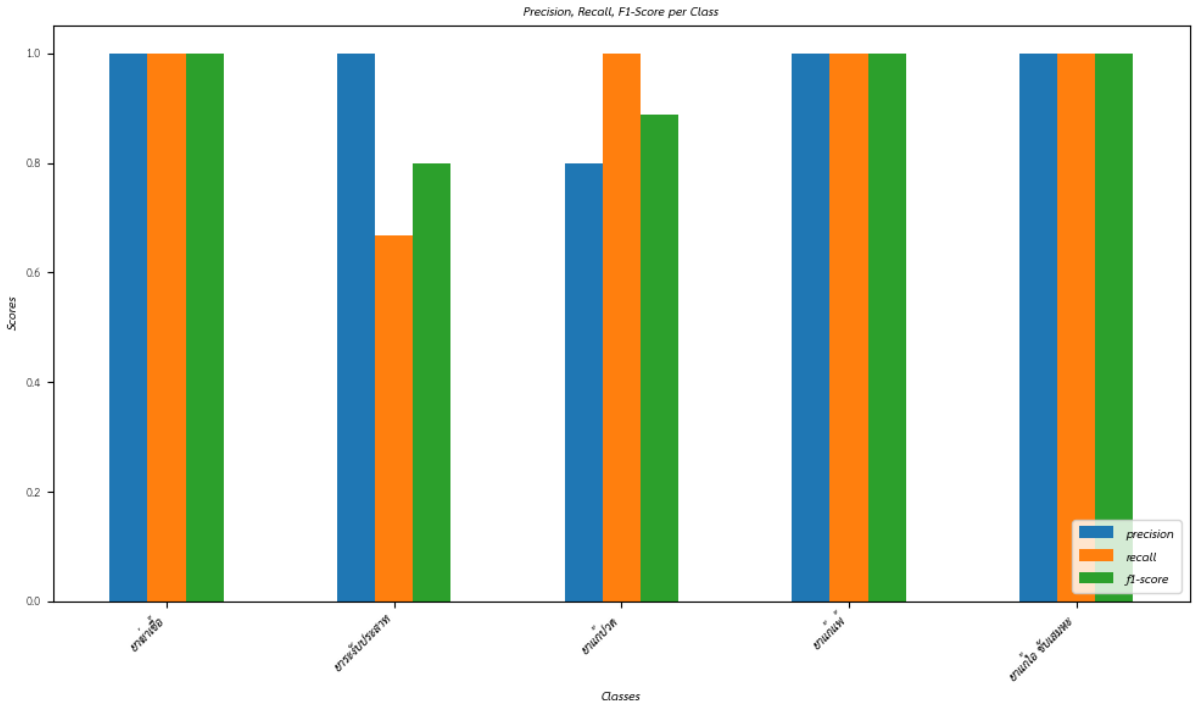
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ

6.1 ผลการดำเนินงาน

แอปพลิเคชันบนมือถือในการตรวจจับชื่อยาและจำแนกประเภทของยาโดยแบ่งการดำเนินงานออกเป็นหลายขั้นตอน เริ่มจากการเตรียมชุดข้อมูล (Data sets) ที่รวบรวมข้อมูลจากเว็บไซต์และจัดข้อมูลของยาทั้งภาษาไทยและภาษาอังกฤษ โดยโดยเก็บข้อมูลดังนี้ ชื่อยา, โรคที่ยาสามารถใช้รักษาได้, สรรพคุณของยา, ประเภทของยา, ผลข้างเคียง เพื่อนำมาสร้างเป็น Data sets ซึ่งถูกแบ่งเป็นหมวดหมู่ และยังมีกลุ่มยาที่เป็นยาจากต่างประเทศคือ Anti-Infectives, Pain Analgesics, Derma, Gastro Intestinal, Respiratory แอปพลิเคชันยังใช้ API ของ Google Cloud Vision AI ในการสแกนและตรวจจับชื่อยาจากภาพถ่าย เมื่อสแกนสลากยาที่มีชื่อยาตรงกับข้อมูลในชุดข้อมูล แอปพลิเคชันจะแสดงข้อมูลที่สำคัญต่อการตัดสินใจในการใช้ยา เช่น ชื่อยา, โรคที่รักษาได้, สรรพคุณของยา, และประเภทของยา หากไม่พบชื่อยานั้นในชุดข้อมูลแอปพลิเคชันจะสามารถจำแนกประเภทของยาจากสรรพคุณโดยการสแกนข้อความจากสรรพคุณของยาและนำไปวิเคราะห์ด้วยโมเดล Deep Learning เพื่อแสดงผลลัพธ์ให้ผู้ใช้ทราบถึงประเภทและผู้ที่สามารสามารถใช้งานตัวนั้นได้

6.1.1 ผลการดำเนินงานของโมเดล Naïve Bayes

ตัวอย่างผลการทดสอบในการตรวจจับจากสรรพคุณของยาแล้วนำไปทำการ Text classification



รูปที่ 11 กราฟแสดงค่า Precision, Recall และ F1-Score

จากรูปที่ 11 ในกราฟนี้เป็นตัวกราฟของโมเดล Naive Bayes แสดงถึงค่าของ **Precision**, **Recall** และ **F1-Score** ของแต่ละประเภท (Class) ซึ่งมีดังนี้ ยาฆ่าเชื้อ, ยาระงับประสาท, ยาแก้ปวด, ยาแก้แพ้, ยาแก้ไอ ขับเสมหะ ซึ่งผลของกราฟนี้คือ

1. Precision (สีน้ำเงิน): แสดงถึงความแม่นยำของโมเดลในกรณีที่โมเดลทำนายว่าตัวอย่างเป็นของประเภทนั้น ๆ Precision มีค่าที่สูงแสดงถึงโมเดลมีแนวโน้มทำนายถูกต้องในแต่ละประเภท

2. Recall (สีส้ม): แสดงถึงความสามารถในการค้นหาตัวอย่างที่เป็นของประเภทนั้นจริง ๆ Recall มีค่าที่สูงโมเดลสามารถค้นหาตัวอย่างที่เป็นของประเภทนั้นได้ดี

3. F1-Score (สีเขียว): เป็นค่าเฉลี่ยของ Precision และ Recall ซึ่งจะให้คะแนนสมดุลสำหรับโมเดลในกรณีที่ต้องการประสิทธิภาพของทั้ง Precision และ Recall ในระดับที่ดี F1-Score มีค่าที่สูงโมเดลมีทั้งความแม่นยำ (Precision) และความสามารถในการค้นหาตัวอย่างจริง (Recall) ที่ดี

ผลของแต่ละประเภท:

- **ยาฆ่าเชื้อ**: Precision, Recall และ F1-Score ของประเภทนี้อยู่ที่ระดับสูงอยู่ที่ 0.97 เท่ากันนั้นคือหมายความว่าโมเดลมีประสิทธิภาพที่ดีมากในการทำนายตัวอย่างในประเภทนี้ได้ถูกต้องและสามารถค้นหาตัวอย่างที่เป็นของประเภทนี้ได้ดี
- **ยาระงับประสาท**: Precision ของประเภทนี้ยังคงอยู่ในระดับสูง แต่ Recall มีค่าต่ำกว่า ซึ่งบ่งบอกว่าโมเดลสามารถทำนายตัวอย่างที่เป็นของประเภทนี้ได้แม่นยำ แต่ยังพลาดตัวอย่างบางส่วนที่เป็นของประเภทนี้ไป ทำให้ Recall ต่ำกว่า Precision
- **ยาแก้ปวด**: Precision, Recall และ F1-Score อยู่ในระดับสูงใกล้เคียงกัน หมายความว่าโมเดลมีความสมดุลที่ดีทั้งในการทำนายและการค้นหาตัวอย่างที่เป็นของประเภทนี้
- **ยาแก้แพ้**: ค่า Precision, Recall และ F1-Score ของประเภทนี้ก็อยู่ในระดับสูงอยู่ที่ 0.97 เท่ากัน แสดงว่าโมเดลมีประสิทธิภาพที่ดีทั้งด้านความแม่นยำและความสามารถในการค้นหาตัวอย่างของประเภทนี้
- **ยาแก้ไอ ขับเสมหะ**: Precision, Recall และ F1-Score ของประเภทนี้ยังอยู่ในระดับสูงอยู่ที่ 0.97 เท่ากัน ซึ่งบ่งบอกถึงประสิทธิภาพที่ดีในภาพรวมสำหรับประเภทนี้เช่นกัน

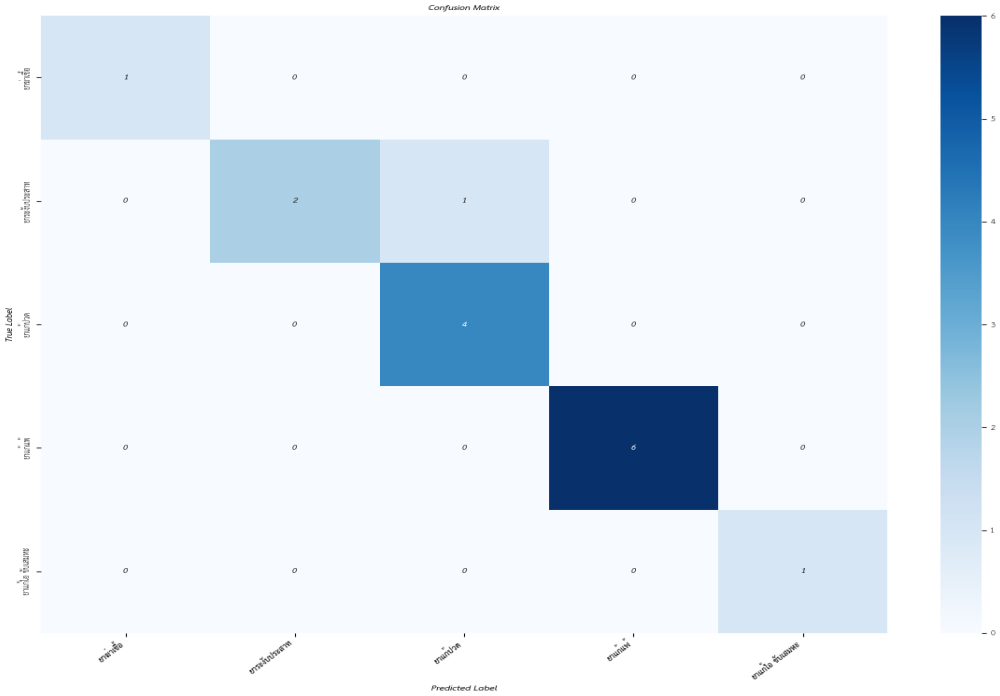
สรุปผลของกราฟ

กราฟนี้แสดงให้เห็นถึงประสิทธิภาพของโมเดลในการทำนายประเภทต่าง ๆ ของยา โดยใช้ค่าชี้วัด Precision, Recall และ **F1-Score** ซึ่งบ่งบอกถึงความแม่นยำและความสามารถในการค้นหาตัวอย่างของโมเดลในแต่ละประเภทดังนี้

1. **ประสิทธิภาพสูงโดยรวม**: โมเดลมีค่า Precision, Recall, และ F1-Score สูงในเกือบทุกประเภท <u>ยกเว้นใน</u> ยาระงับประสาท ซึ่ง Recall ต่ำกว่าค่าชี้วัดอื่น ๆ เล็กน้อย
2. **ความแม่นยำดีเยี่ยม (Precision สูง)**: โมเดลสามารถทำนายได้แม่นยำมากในเกือบทุกประเภท โดยเฉพาะ ยาฆ่าเชื้อ, ยาแก้ปวด, ยาแก้แพ้, และ ยาแก้ไอ ขับเสมหะ มีค่า Precision สูงถึง 0.97
3. **ความสามารถในการค้นหาตัวอย่าง**: Recall ของโมเดลก็สูงในเกือบทุกประเภท <u>ยกเว้นใน</u> ยาระงับประสาท ซึ่งค่า Recall ต่ำกว่าเล็กน้อย บ่งบอกว่ามีบางตัวอย่างที่โมเดลพลาดการทำนาย
4. **สมดุลระหว่าง Precision และ Recall (F1-Score สูง)**: F1-Score ซึ่งเป็นค่าเฉลี่ยของ Precision และ Recall ก็สูงในทุกประเภท แสดงว่าโมเดลมีความสมดุลที่ดีในการทำนายประเภทของยาได้อย่างแม่นยำและครอบคลุม

ดังนั้นโมเดลนี้มีประสิทธิภาพที่ดีในการทำนายทุกประเภทของยา โดยเฉพาะในประเภท ยาฆ่าเชื้อ, ยาแก้ปวด, ยาแก้แพ้ และ ยาแก้ไอ ขับเสมหะ <u>ยกเว้น</u> ยาระงับประสาท ที่อาจพลาดตัวอย่างบางส่วนไปเล็กน้อย แต่โดยรวมถือว่าโมเดลทำงานได้ดีและมีความน่าเชื่อถือสูง

Confusion Matrix



รูปที่ 12 กราฟ Confusion Matrix ของกลุ่มตัวยาภาษาไทย

จากรูปที่ 12 Confusion Matrix แสดงจำนวนตัวอย่างในแต่ละคลาสมีเพียงไม่กี่ตัวอย่าง (เช่น 1-6 ตัวอย่างต่อคลาส) เนื่องจาก data ของกลุ่มตัวยาภาษาไทยนั้นมีน้อยทำให้กราฟที่แสดงผลออกมามีค่าที่น้อยตาม กราฟนี้แสดงผลการทำนายของโมเดลสำหรับการจำแนกประเภทของยาตามคลาสต่าง ๆ ซึ่งใช้ในการวิเคราะห์ประสิทธิภาพของโมเดลว่ามีความถูกต้องเพียงใดในการจำแนกประเภท โดยในกราฟนี้

- (Predicted Label) แสดงประเภทของยาที่โมเดลทำนายว่าเป็น
- (True Label) แสดงประเภทของยาที่เป็นจริง
- ตัวเลขในแต่ละช่องแสดงจำนวนตัวอย่างที่อยู่ในแต่ละกลุ่ม

ผลการวิเคราะห์ Confusion Matrix

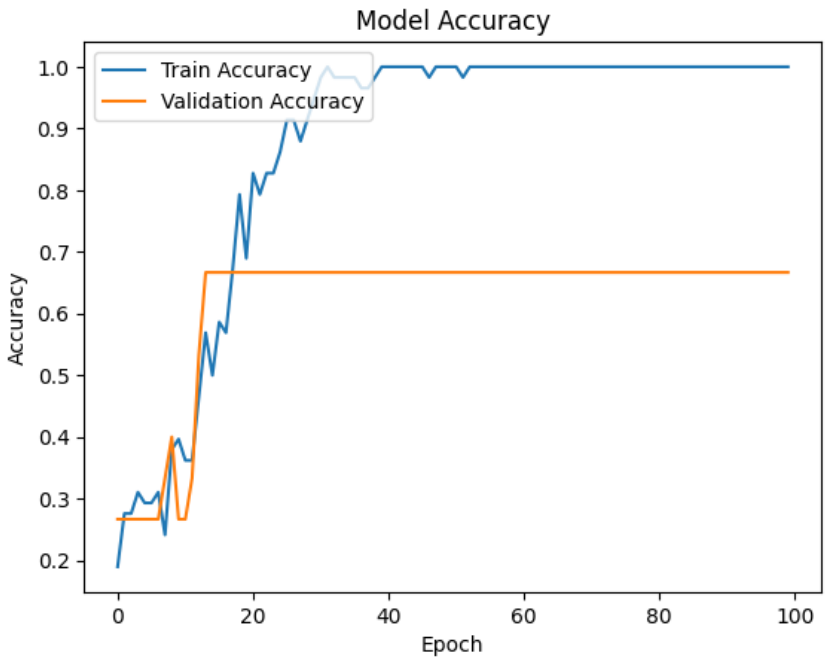
จาก Confusion Matrix แสดงให้เห็นถึงประสิทธิภาพการทำนายของโมเดลในการจำแนกประเภทของยาตามคลาสต่าง ๆ ดังนี้:

1. **ยาฆ่าเชื้อ**: โมเดลสามารถทำนายได้อย่างถูกต้องทั้งหมด จำนวน 1 ตัวอย่าง โดยไม่มีการทำนายผิดเป็นประเภทอื่นเลย ซึ่งแสดงให้เห็นว่าโมเดลมีความแม่นยำสูงสำหรับการจำแนกประเภทนี้
2. **ยาระงับประสาท**: โมเดลทำนายได้ถูกต้อง 2 ตัวอย่าง แต่มีการทำนายผิดจำนวน 1 ตัวอย่าง ซึ่งทำนายผิดเป็นยาแก้ปวด ซึ่งแสดงให้เห็นว่าโมเดลยังมีการสับสนเล็กน้อยระหว่างคลาส ยาระงับประสาท และ ยาแก้ปวด

3. **ยาแก้ปวด**: โมเดลสามารถทำนายได้ถูกต้องทั้งหมด จำนวน 4 ตัวอย่าง โดยไม่มีการทำนายผิดเป็นคลาสอื่น แสดงถึงความแม่นยำที่สูงในการจำแนกประเภท ยาแก้ปวด
4. **ยาแก้แพ้**: โมเดลสามารถทำนายได้ถูกต้องทั้งหมด จำนวน 6 ตัวอย่าง โดยไม่มีการทำนายผิดไปเป็นคลาสอื่น ซึ่งสะท้อนให้เห็นถึงความแม่นยำที่สูงในการจำแนกประเภท ยาแก้แพ้
5. **ยาแก้ไอ ขับเสมหะ**: โมเดลสามารถทำนายได้ถูกต้อง 1 ตัวอย่าง โดยไม่มีการทำนายผิดไปเป็นคลาสอื่นเช่นกัน แสดงถึงความแม่นยำที่สูงในการจำแนกประเภทนี้

ดังนั้นผลสรุปของโมเดลนี้คือ มีความแม่นยำสูงในการจำแนกประเภทของยาในกลุ่ม ยาฆ่าเชื้อ, ยาแก้ปวด, ยาแก้แพ้, และ ยาแก้ไอ ขับเสมหะ โดยไม่มีการทำนายผิดสำหรับกลุ่มเหล่านี้เลย อย่างไรก็ตาม โมเดลมีการสับสนเล็กน้อยในกลุ่ม ยาระงับประสาท ซึ่งมีการทำนายผิดเป็นกลุ่ม ยาแก้ปวด จำนวน 1 ตัวอย่าง

ผลการวิเคราะห์ชี้ให้เห็นว่าโมเดลมีประสิทธิภาพสูงในการจำแนกประเภทของยาส่วนใหญ่ แต่ยังคงมีข้อผิดพลาดเล็กน้อยที่อาจต้องการการปรับปรุงเพื่อเพิ่มความแม่นยำในการจำแนกประเภทที่มีความใกล้เคียงกัน



รูปที่ 13 กราฟแสดงค่า Accuracy ของสรรพคุณภาษาไทยใน Deep learning

จากรูปที่ 13 นำตัว data ยาภาษาไทยมาใช้กับโมเดล Long short-term memory นั้นจะได้ผลลัพธ์ของค่า accuracy ดังนี้

กราฟนี้แสดงถึงการเปรียบเทียบค่าความแม่นยำ (accuracy) ระหว่างข้อมูลฝึก (Train Accuracy) และข้อมูลทดสอบ (Validation Accuracy) ของโมเดลในแต่ละ epoch ซึ่งแสดงผลได้ดังนี้

รายละเอียดกราฟ

- แกน X (Epoch): จำนวนครั้งที่โมเดลได้ฝึกกับชุดข้อมูลฝึกทั้งหมด (หนึ่งรอบที่โมเดลผ่านข้อมูลทั้งหมดเรียกว่า 1 epoch) โดยในกราฟนี้ฝึกทั้งหมด 100 epochs
- แกน Y (Accuracy): ระดับความแม่นยำของโมเดลต่อข้อมูลฝึกและข้อมูลทดสอบ มีค่าอยู่ในช่วง 0 ถึง 1 ซึ่งค่าใกล้ 1 หมายถึงความแม่นยำที่สูงขึ้น

ลักษณะของกราฟ

- Train Accuracy (เส้นสีน้ำเงิน): ความแม่นยำของโมเดลต่อชุดข้อมูลฝึก พบว่าในช่วงแรก ความแม่นยำค่อย ๆ เพิ่มขึ้นอย่างต่อเนื่อง และเมื่อผ่านไปประมาณ 40 epochs ค่าความแม่นยำจะคงที่และมีค่าใกล้ 1 ซึ่งหมายความว่าโมเดลสามารถทำนายข้อมูลฝึกได้อย่างแม่นยำ
- Validation Accuracy (เส้นสีส้ม): ความแม่นยำของโมเดลต่อชุดข้อมูลทดสอบ พบว่าความแม่นยำเพิ่มขึ้นอย่างรวดเร็วในช่วงต้นและคงที่อยู่ที่ประมาณ 0.6 หลังจาก epoch ที่ 10 แล้วไม่มีการเปลี่ยนแปลงเพิ่มเติม

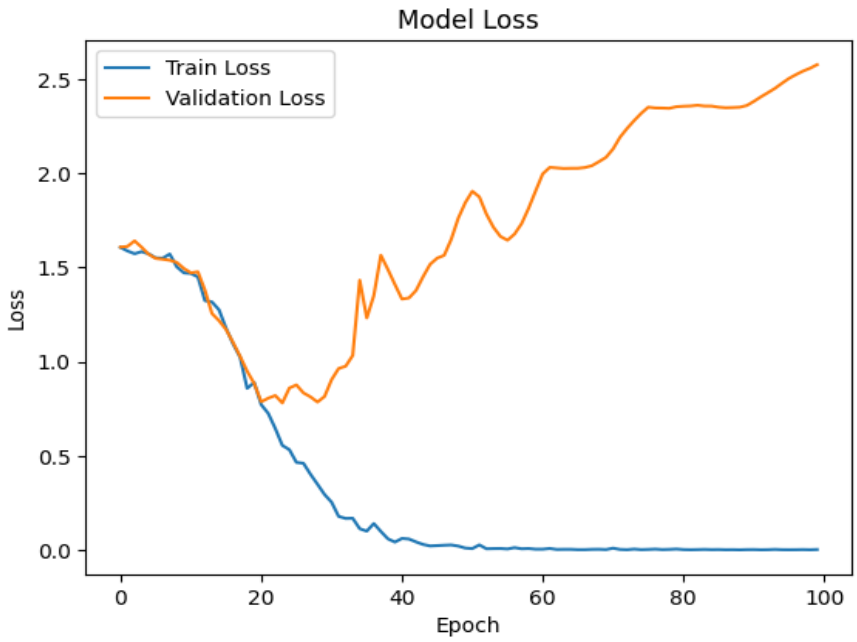
สรุปผลจากกราฟแสดงความแม่นยำของแบบจำลองนี้สามารถอธิบายได้ดังนี้

1.**ช่วงเริ่มต้น (Epoch 0 - 20)**: ในช่วงแรกของการฝึกอบรม (ประมาณ 20 รอบแรก) ความแม่นยำของแบบจำลองทั้งจากชุดข้อมูลการฝึกอบรม (Train Accuracy) และชุดข้อมูลทดสอบ (Validation Accuracy) เพิ่มขึ้นอย่างรวดเร็ว แต่มีการผันผวนเล็กน้อย ซึ่งเป็นปกติในกระบวนการฝึกแบบจำลอง เนื่องจากตัวแบบยังอยู่ในช่วงปรับน้ำหนักและพารามิเตอร์ต่าง ๆ เพื่อให้ได้ผลลัพธ์ที่ดีขึ้น

2.**การพัฒนาในช่วงกลาง (Epoch 20 - 40)**: หลังจากรอบที่ 20 ความแม่นยำของชุดการฝึกอบรมเพิ่มขึ้นต่อเนื่อง จนใกล้เคียงกับค่า 1.0 ซึ่งหมายถึงแบบจำลองสามารถเรียนรู้จากข้อมูลการฝึกอบรมได้ดีมาก ในทางกลับกัน ความแม่นยำของชุดทดสอบกลับคงที่ที่ประมาณ 0.7 และไม่เพิ่มขึ้นอีกหลังจากนั้น ซึ่งแสดงว่าแบบจำลองอาจไม่สามารถเรียนรู้จากชุดข้อมูลทดสอบได้เพิ่มเติม

3.**ช่วงหลังของการฝึกอบรม (Epoch 40 เป็นต้นไป)**: ความแม่นยำของชุดการฝึกอบรมยังคงใกล้ 1.0 ตลอดระยะเวลาที่เหลือ แต่ความแม่นยำของชุดทดสอบยังคงค้างอยู่ที่ค่าเดิม (ประมาณ 0.7) ซึ่งบ่งบอกถึงการเกิด Overfitting โดยแบบจำลองสามารถจดจำรายละเอียดของข้อมูลการฝึกอบรมได้ดีมาก แต่ไม่สามารถนำความรู้ที่ได้ไปใช้กับข้อมูลใหม่ในชุดทดสอบได้อย่างมีประสิทธิภาพ

ดังนั้นสรุปคือโมเดลมีความแม่นยำต่อชุดข้อมูลฝึกสูงมาก (ใกล้ 1) ซึ่งอาจบ่งบอกว่าโมเดลเรียนรู้ลักษณะของข้อมูลฝึกได้ดี แต่เนื่องจากโมเดลทำนายชุดข้อมูลทดสอบได้เพียง 60% ซึ่งต่างจากชุดข้อมูลฝึกอย่างชัดเจน แสดงถึงอาการ Overfitting



รูปที่ 14 กราฟแสดงค่า Loss ของสรรพคุณภาษาไทยใน Deep learning

จากรูปที่ 14 นำตัว data ยาภาษาไทยมาใช้กับโมเดล Long short-term memory นั้นจะได้ผลลัพธ์ของค่า Loss ดังนี้

กราฟนี้แสดงถึงการเปลี่ยนแปลงของค่าการสูญเสีย (Loss) ของแบบจำลองทั้งจากชุดข้อมูลการฝึกอบรม (Train Loss) และชุดข้อมูลทดสอบ (Validation Loss) ตามจำนวนรอบการฝึกอบรม (Epoch) โดยค่าการสูญเสียเป็นตัววัดความแตกต่างระหว่างค่าที่คาดการณ์โดยแบบจำลองและค่าจริง ซึ่งค่าการสูญเสียที่ต่ำลงไปแสดงถึงประสิทธิภาพที่ดีขึ้นของแบบจำลอง

รายละเอียดการวิเคราะห์กราฟ

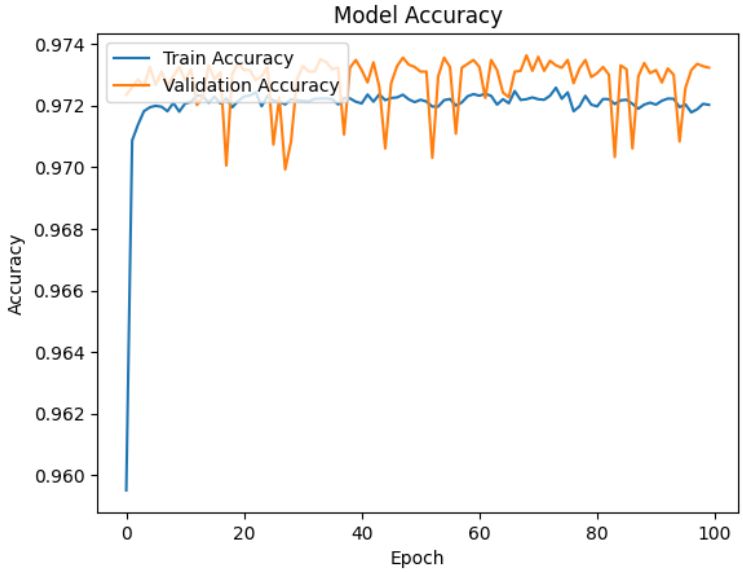
1. **แกน X (Epoch)**: แทนจำนวนรอบการฝึกอบรม (Epoch) ตั้งแต่ 0 ถึง 100 รอบ
2. **แกน Y (Loss)**: แทนค่าการสูญเสีย (Loss) ของแบบจำลอง มีค่าตั้งแต่ 0 ขึ้นไป โดยค่าที่ต่ำกว่าแสดงถึงความแม่นยำที่สูงขึ้นของแบบจำลอง
3. **เส้นสีน้ำเงิน (Train Loss)**: แสดงค่าการสูญเสียของแบบจำลองเมื่อใช้ข้อมูลการฝึกอบรม
4. **เส้นสีส้ม (Validation Loss)**: แสดงค่าการสูญเสียของแบบจำลองเมื่อใช้ข้อมูลทดสอบ

การวิเคราะห์ผล

ในช่วงเริ่มต้นของการฝึกอบรม (Epoch 0 - 20) ค่าการสูญเสียทั้งจากชุดข้อมูลการฝึกอบรมและชุดข้อมูลทดสอบมีแนวโน้มลดลง ซึ่งแสดงถึงการเรียนรู้ที่มีประสิทธิภาพของแบบจำลอง ตั้งแต่ Epoch 20 เป็นต้นไป ค่าการสูญเสียของชุดข้อมูลการฝึกอบรม (Train Loss) ลดลงอย่างรวดเร็วและคงที่อยู่ใกล้ 0 แสดงถึงการที่แบบจำลองสามารถจดจำข้อมูลการฝึกอบรมได้ดีมาก ในขณะเดียวกันค่าการสูญเสียของชุดข้อมูลทดสอบ (Validation Loss) เริ่มมีแนวโน้มเพิ่มขึ้นและมีการผันผวน ซึ่งแสดงถึงปัญหาการ Overfitting กล่าวคือ แบบจำลองสามารถจดจำข้อมูลการฝึกอบรมได้ดี แต่ไม่สามารถทั่วไปผลลัพธ์ได้ดีกับข้อมูลที่ไม่เคยเห็นมาก่อนในชุดทดสอบ

ดังนั้นผลสรุปคือแบบจำลองนี้เกิดปัญหาการ Overfitting โดยค่าการสูญเสียของชุดข้อมูลการฝึกอบรมลดลงใกล้ 0 ในขณะที่ค่าการสูญเสียของชุดข้อมูลทดสอบเพิ่มขึ้นเมื่อผ่าน Epoch ไปสู่ระดับสูง ซึ่งแสดงถึงการที่แบบจำลองไม่สามารถทำนายผลได้อย่างแม่นยำกับข้อมูลที่ไม่เคยเห็นมาก่อน วิธีการแก้ไขอาจใช้เทคนิคการป้องกันการ Overfitting เช่น การใช้ Regularization, การลดความซับซ้อนของแบบจำลอง, หรือการใช้เทคนิค Early Stopping เพื่อหยุดการฝึกอบรมเมื่อค่าการสูญเสียของชุดทดสอบเริ่มเพิ่มขึ้น

6.1.2ผลการดำเนินงานของโมเดล Deep learning



รูปที่ 15 กราฟแสดงค่า Accuracy ของสรรพคุณภาษอังกฤษใน Deep learning

จากรูปที่ 15 กราฟนี้เป็นการเป็นการนำกลุ่มตัวยาภาษาอังกฤษมาจำแนกประเภทโดยใช้ deep learning Long short-term memory จากกราฟนี้แสดงถึงการเปลี่ยนแปลงของความแม่นยำ (Accuracy) ของแบบจำลองในการทำนายผล โดยพิจารณาจากค่าความแม่นยำของชุดข้อมูลการฝึกอบรม (Train Accuracy) และชุดข้อมูลทดสอบ (Validation Accuracy) ตลอดช่วงจำนวนรอบการฝึกอบรม (Epoch) ตั้งแต่ 0 ถึง 100 รอบ ซึ่งสามารถสรุปและวิเคราะห์ได้ดังนี้

รายละเอียดการวิเคราะห์กราฟ

1. **แกน X (Epoch)**: แทนจำนวนรอบการฝึกอบรม (Epoch) โดยเริ่มตั้งแต่ 0 ไปจนถึง 100 รอบ
2. **แกน Y (Accuracy)**: แทนค่าความแม่นยำ (Accuracy) ของแบบจำลอง โดยมีช่วงค่าอยู่ระหว่าง 0.960 ถึง 0.974 ซึ่งสะท้อนถึงประสิทธิภาพของแบบจำลองในการทำนายผลได้อย่างถูกต้องเมื่อเทียบกับค่าจริง
3. **(Train Accuracy)**: แสดงความแม่นยำของแบบจำลองเมื่อใช้ข้อมูลการฝึกอบรม (Training Data) ซึ่งแสดงถึงความสามารถในการเรียนรู้จากข้อมูลที่ป้อนให้ระหว่างการฝึกอบรม

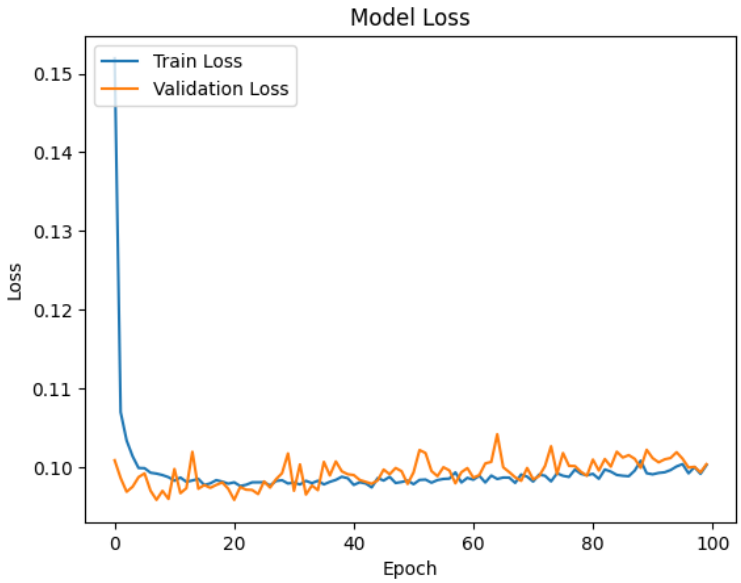
4. **(Validation Accuracy)**: แสดงความแม่นยำของแบบจำลองเมื่อใช้ข้อมูลทดสอบ (Validation Data) ซึ่งแสดงถึงความสามารถในการทำนายข้อมูลใหม่ที่ไม่เคยเห็นมาก่อนในชุดการฝึก

การวิเคราะห์ผล

- **ช่วงเริ่มต้นของการฝึกอบรม (Epoch 0 - 10)**: ในช่วงเริ่มต้น ความแม่นยำของแบบจำลองเพิ่มขึ้นอย่างรวดเร็วในรอบการฝึกอบรมแรก ๆ โดยเฉพาะในรอบแรกที่มีการกระโดดของค่าความแม่นยำอย่างมีนัยสำคัญ หลังจากนั้นค่าความแม่นยำค่อย ๆ เข้าใกล้ค่าสูงสุดในช่วงประมาณ 0.972 แสดงให้เห็นถึงการที่แบบจำลองสามารถเรียนรู้จากข้อมูลการฝึกอบรมได้อย่างรวดเร็ว

- **ช่วงกลางและช่วงหลังของการฝึกอบรม (Epoch 10 - 100)**: ความแม่นยำของทั้งชุดข้อมูลการฝึกอบรมและชุดข้อมูลทดสอบมีแนวโน้มคงที่อยู่ที่ประมาณ 0.972 ตลอดช่วงนี้ ซึ่งแสดงถึงความเสถียรของแบบจำลอง โดยเส้นกราฟของค่าความแม่นยำในชุดข้อมูลการฝึกอบรมมีการเปลี่ยนแปลงเพียงเล็กน้อยเท่านั้น ในขณะที่เส้นกราฟของค่าความแม่นยำในชุดข้อมูลทดสอบมีการผันผวนขึ้นลงเล็กน้อย แต่ยังคงรักษาค่าเฉลี่ยใกล้เคียงกับค่าความแม่นยำของชุดการฝึกอบรม ซึ่งเป็นสัญญาณที่ดีว่าแบบจำลองสามารถทำงานได้ดีในสภาวะที่มีข้อมูลใหม่และไม่มีการ Overfitting อย่างชัดเจน

ดังนั้นผลสรุปคือแบบจำลองนี้แสดงให้เห็นถึงประสิทธิภาพที่ดีและมีความเสถียรในความแม่นยำทั้งในชุดข้อมูลการฝึกอบรมและชุดข้อมูลทดสอบ โดยมีค่าความแม่นยำคงที่ที่ประมาณ 0.972 ตลอดระยะเวลาการฝึกอบรม ซึ่งบ่งบอกว่าแบบจำลองสามารถเรียนรู้ได้อย่างดีและไม่มีการเกิด Overfitting อย่างชัดเจน อย่างไรก็ตาม การผันผวนเล็กน้อยในค่าความแม่นยำของชุดข้อมูลทดสอบอาจสะท้อนถึงความไวของแบบจำลองต่อความแตกต่างเล็กน้อยในข้อมูลทดสอบ ซึ่งอาจปรับปรุงได้ด้วยการใช้เทคนิคเพิ่มเติม เช่น การปรับปรุงชุดข้อมูลทดสอบให้มีความครอบคลุมมากขึ้น หรือการปรับแต่งพารามิเตอร์ของแบบจำลองเพิ่มเติมเพื่อเพิ่มความแม่นยำ



รูปที่ 16 กราฟแสดงค่า Loss ของสรรพคุณภาษอังกฤษใน Deep learning

จากรูปที่ 16 นำตัว data ยาภาษาไทยมาใช้กับโมเดลLong short-term memory นั้นจะได้ผลลัพธ์ของค่า Loss ดังนี้

กราฟนี้แสดงถึงการสูญเสียของแบบจำลอง (Model Loss) ในระหว่างการฝึกอบรมและการทดสอบ โดยมีรายละเอียดดังนี้

1. **(Epoch)** แกน X แสดงจำนวน epoch หรือรอบการฝึกอบรมแบบจำลอง แต่ละ epoch คือการฝึกอบรมข้อมูลหนึ่งรอบผ่านแบบจำลอง
2. **(Loss)** แกน Y แสดงค่าการสูญเสีย (Loss) ซึ่งเป็นตัวชี้วัดความคลาดเคลื่อนระหว่างค่าที่แบบจำลองทำนายกับค่าจริง ยิ่งค่านี้ยิ่งต่ำ แสดงว่าแบบจำลองทำการพยากรณ์ได้แม่นยำมากขึ้น
3. **(Train Loss)** – แสดงค่าการสูญเสียระหว่างการฝึกอบรม ในช่วงแรกของการฝึก ค่าการสูญเสียลดลงอย่างรวดเร็ว และเริ่มคงที่เมื่อถึงประมาณ epoch ที่ 10
4. **(Validation Loss)** – แสดงค่าการสูญเสียในการทดสอบ ซึ่งใช้ข้อมูลที่ไม่ถูกใช้ในการฝึกอบรม เพื่อประเมินความสามารถของแบบจำลองในการพยากรณ์ข้อมูลใหม่

การวิเคราะห์กราฟ Loss

กราฟแสดงถึงพฤติกรรมของค่าการสูญเสีย (Loss) สำหรับชุดข้อมูลการฝึก (Train Loss) และชุดข้อมูลการตรวจสอบ (Validation Loss) ในการฝึกอบรมแบบจำลอง โดยมีการพิจารณาในรายละเอียดดังนี้:

1. **ช่วงเริ่มต้นของการฝึกอบรม (Epoch 0-10):**

ในช่วงเริ่มต้น ค่าการสูญเสียของชุดข้อมูลการฝึก (เส้นสีน้ำเงิน) มีการลดลงอย่างรวดเร็ว แสดงให้เห็นว่าแบบจำลองสามารถเรียนรู้และปรับปรุงการคาดการณ์ได้อย่างมีประสิทธิภาพในช่วงแรก ซึ่งบ่งชี้ถึงความสามารถในการจับรูปแบบและความสัมพันธ์ของข้อมูลที่มีในชุดข้อมูลการฝึกได้อย่างรวดเร็ว

สำหรับค่าการสูญเสียของชุดข้อมูลตรวจสอบ (เส้นสีส้ม) ในช่วงเดียวกัน พบว่ามีความผันผวนเล็กน้อยและลดลงในแนวโน้มที่ใกล้เคียงกับชุดข้อมูลการฝึก แสดงว่าแบบจำลองสามารถปรับให้เข้ากับข้อมูลได้อย่างเหมาะสม และยังคงสามารถทำงานได้ดีเมื่อทดสอบกับข้อมูลที่ไม่ได้ถูกใช้ในการฝึก

2. ช่วงการฝึกอบรมในระยะกลางถึงระยะปลาย (Epoch 10-100):

หลังจาก Epoch ที่ 10 เป็นต้นไป ค่าการสูญเสียของชุดข้อมูลการฝึกจะมีแนวโน้มที่คงที่ โดยค่าจะลดลงอย่างช้า ๆ จนกระทั่งเข้าสู่ระดับที่ค่อนข้างคงที่ที่ประมาณ 0.10 ซึ่งสะท้อนให้เห็นว่าแบบจำลองมีความเสถียร (Stability) ในการเรียนรู้และไม่ปรับปรุงอย่างเกินจำเป็น

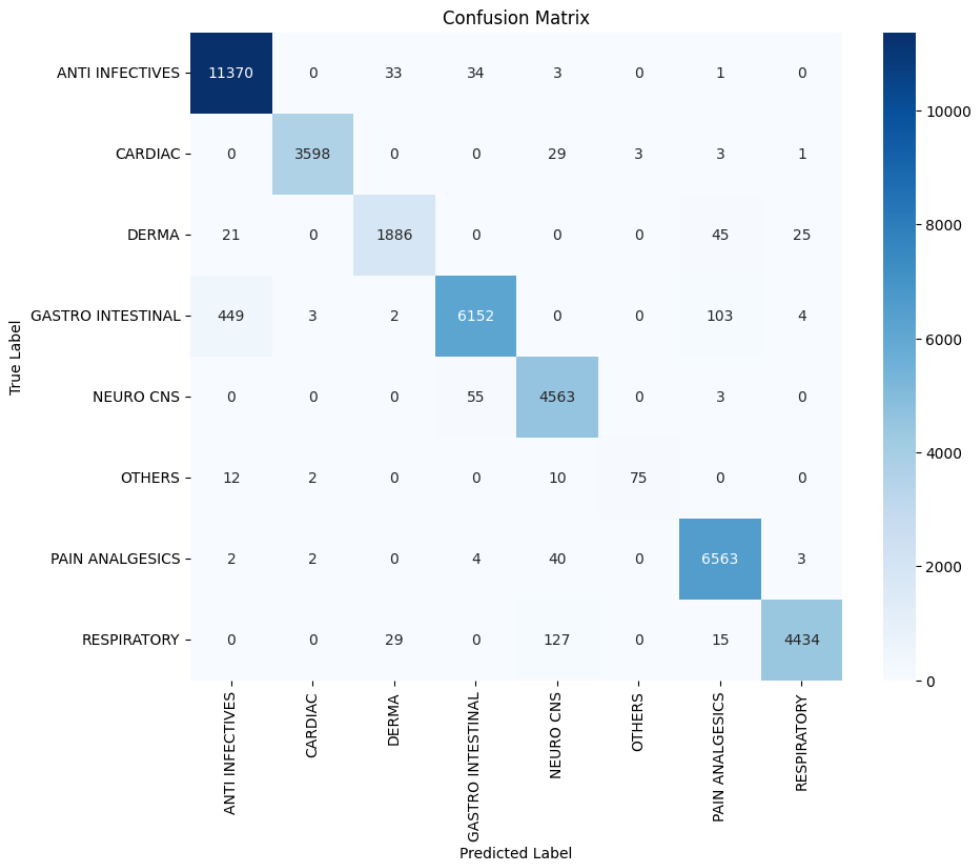
ค่าการสูญเสียของชุดข้อมูลตรวจสอบยังคงมีการผันผวนเล็กน้อยในบางช่วงของ Epoch แต่มีแนวโน้มที่คงตัวในระดับใกล้เคียงกับค่าของชุดข้อมูลการฝึก ซึ่งสะท้อนให้เห็นว่าแบบจำลองสามารถทำนายผลลัพธ์ได้อย่างมีประสิทธิภาพกับข้อมูลที่ไม่เคยเห็นมาก่อนและไม่มีปัญหา overfitting (การเรียนรู้เฉพาะเจาะจงกับข้อมูลฝึกมากเกินไป) หรือ underfitting (การเรียนรู้ไม่เพียงพอ)

3. การเปรียบเทียบระหว่าง Train Loss และ Validation Loss:

ค่าการสูญเสียของชุดข้อมูลการฝึกและชุดข้อมูลตรวจสอบมีแนวโน้มเคลื่อนที่ในทิศทางใกล้เคียงกันโดยตลอด ซึ่งบ่งบอกถึงการเรียนรู้ที่สมดุลของแบบจำลอง กล่าวคือ แบบจำลองสามารถเข้าใจข้อมูลได้โดยไม่เกิดการปรับตัวมากเกินไปในชุดข้อมูลการฝึก ซึ่งเป็นสัญญาณที่ดีในเชิงการพยากรณ์
ระดับค่าการสูญเสียที่ใกล้เคียงกันระหว่าง Train Loss และ Validation Loss สะท้อนถึงการพยากรณ์ที่มีความแม่นยำสูงและมีความสามารถในการทำนายข้อมูลใหม่ได้อย่างมีประสิทธิภาพ

สรุปผลของโมเดล

จากการวิเคราะห์ข้างต้น สามารถสรุปได้ว่าแบบจำลองมีความสามารถในการเรียนรู้รูปแบบจากข้อมูลได้ดีและนำไปใช้ในการทำนายข้อมูลใหม่ได้อย่างมีประสิทธิภาพ โดยไม่เกิดปัญหา overfitting หรือ underfitting กล่าวคือ ค่าการสูญเสียของชุดข้อมูลการฝึกและชุดข้อมูลตรวจสอบมีแนวโน้มเคลื่อนที่ในทิศทางเดียวกันและอยู่ในระดับต่ำ ซึ่งบ่งชี้ถึงความแม่นยำและความเสถียรของแบบจำลองนี้



รูปที่ 17 กราฟ Confusion Matrix ของกลุ่มตัวยาภาษาอังกฤษ

จากรูปที่ 17 กราฟนี้เป็น Confusion Matrix ซึ่งแสดงถึงผลลัพธ์ของการจำแนกประเภทของแบบจำลองในการทำนายประเภทของยาในแต่ละกลุ่ม โดยรายละเอียดของ Confusion Matrix ดังกล่าวมีดังนี้

การอธิบาย Confusion Matrix

(Predicted Label) แสดงประเภทที่แบบจำลองคาดการณ์ (True Label) แสดงประเภทที่ถูกต้องตามจริง ค่าในแต่ละช่องแสดงจำนวนตัวอย่างในแต่ละกลุ่มที่แบบจำลองคาดการณ์

ผลการวิเคราะห์ Confusion Matrix

Confusion Matrix นี้แสดงผลการจำแนกประเภทของแบบจำลองที่ใช้ในการทำนายกลุ่มข้อมูลทางการแพทย์ต่าง ๆ โดยประกอบด้วยข้อมูลเกี่ยวกับจำนวนการคาดการณ์ที่ถูกต้องและคาดการณ์ผิดในแต่ละกลุ่ม สามารถอธิบายได้ตามรายละเอียดดังนี้

ANTI INFECTIVES:

แบบจำลองสามารถจำแนกประเภทนี้ได้ถูกต้องทั้งหมด 11,370 ตัวอย่าง ซึ่งแสดงถึงความแม่นยำสูงในการจำแนกประเภทนี้มีข้อผิดพลาดในการคาดการณ์ โดยแบบจำลองจำแนกกลุ่ม ANTI INFECTIVES ผิดเป็นกลุ่มอื่น ๆ เล็กน้อย ได้แก่ GASTRO INTESTINAL จำนวน 449 ครั้ง และ DERMA จำนวน 21 ครั้ง ซึ่งอาจเกิดจากคุณลักษณะที่คล้ายคลึงกันของข้อมูลบางส่วนระหว่างกลุ่มเหล่านี้

CARDIAC:

จำนวนการคาดการณ์ที่ถูกต้องสำหรับกลุ่มนี้อยู่ที่ 3,598 ตัวอย่างมีการคาดการณ์ผิดเป็นกลุ่ม OTHERS 10 ตัวอย่าง และเป็น GASTRO INTESTINAL 29 ตัวอย่าง ซึ่งบ่งชี้ว่าแบบจำลองมีประสิทธิภาพดีในการจำแนกประเภทนี้ อย่างไรก็ตาม ยังคงมีข้อผิดพลาดในระดับเล็กน้อยที่อาจมาจากความไม่ชัดเจนของลักษณะเฉพาะในกลุ่มนี้

DERMA:

จำนวนการคาดการณ์ที่ถูกต้องในกลุ่ม DERMA คือ 1,886 ตัวอย่างการคาดการณ์ผิดส่วนใหญ่เป็นกลุ่ม ANTI INFECTIVES จำนวน 21 ตัวอย่าง และ RESPIRATORY 25 ตัวอย่าง ซึ่งอาจบ่งชี้ถึงความคล้ายคลึงกันของข้อมูลในบางลักษณะ ทำให้เกิดข้อผิดพลาดในการจำแนก

GASTRO INTESTINAL:

มีการคาดการณ์ที่ถูกต้องในกลุ่มนี้ทั้งหมด 6,152 ตัวอย่างอย่างไรก็ตาม ยังพบข้อผิดพลาดบางประการ โดยกลุ่มนี้ถูกจำแนกผิดเป็น ANTI INFECTIVES จำนวน 449 ตัวอย่าง และ OTHERS 103 ตัวอย่าง ซึ่งอาจชี้ถึงการมีคุณลักษณะร่วมบางประการระหว่างกลุ่ม GASTRO INTESTINAL และ ANTI INFECTIVES ทำให้แบบจำลองมีความสับสนระหว่างกลุ่มเหล่านี้

NEURO CNS:

แบบจำลองสามารถคาดการณ์ได้ถูกต้องในกลุ่มนี้ทั้งหมด 4,563 ตัวอย่างมีการคาดการณ์ผิดในจำนวนเล็กน้อย เช่น ถูกจำแนกผิดเป็น OTHERS 3 ตัวอย่าง และ GASTRO INTESTINAL 55 ตัวอย่าง ซึ่งแสดงให้เห็นว่าประสิทธิภาพการจำแนกกลุ่มนี้อยู่ในเกณฑ์ที่ดีมาก

OTHERS:

แบบจำลองสามารถจำแนกประเภทนี้ได้ถูกต้อง 75 ตัวอย่างอย่างไรก็ตาม เนื่องจากจำนวนตัวอย่างในกลุ่มนี้มีน้อย จึงอาจทำให้แบบจำลองมีปัญหาในการแยกแยะ ส่งผลให้บางตัวอย่างในกลุ่มนี้ถูกจำแนกผิดเป็นกลุ่มอื่น เช่น ANTI INFECTIVES และ NEURO CNS

PAIN ANALGESICS:

แบบจำลองสามารถคาดการณ์ได้ถูกต้องในกลุ่มนี้ 6,563 ตัวอย่างมีข้อผิดพลาดในการจำแนกเป็นกลุ่ม RESPIRATORY 3 ตัวอย่าง และเป็น ANTI INFECTIVES 2 ตัวอย่าง ซึ่งบ่งชี้ว่าการจำแนกกลุ่ม PAIN ANALGESICS เป็นไปอย่างมีประสิทธิภาพสูง

RESPIRATORY:

จำนวนการคาดการณ์ที่ถูกต้องสำหรับกลุ่มนี้คือ 4,434 ตัวอย่างมีการคาดการณ์ผิดในจำนวนเล็กน้อย โดยกลุ่มนี้ถูกจำแนกเป็น GASTRO INTESTINAL 127 ตัวอย่าง และ ANTI INFECTIVES 29 ตัวอย่าง ซึ่งแสดงถึงการมีคุณลักษณะที่คล้ายคลึงกันระหว่างกลุ่มเหล่านี้บางประการ

สรุปผลการวิเคราะห์

จากการวิเคราะห์ Confusion Matrix นี้ พบว่าแบบจำลองมีความแม่นยำสูงในการจำแนกประเภทข้อมูล โดยเฉพาะในกลุ่มที่มีตัวอย่างจำนวนมาก เช่น ANTI INFECTIVES, GASTRO INTESTINAL, และ PAIN ANALGESICS ซึ่งสามารถจำแนกได้อย่างถูกต้องเกือบทั้งหมด อย่างไรก็ตาม ยังคงพบข้อผิดพลาดในบางกลุ่มที่มีลักษณะใกล้เคียงกัน เช่น กลุ่ม ANTI INFECTIVES กับ GASTRO INTESTINAL และกลุ่ม DERMA กับ RESPIRATORY ซึ่งอาจเกิดจากความคล้ายคลึงของคุณลักษณะของข้อมูลในกลุ่มเหล่านี้

	precision	recall	f1-score	support
ANTI INFECTIVES	0.96	0.99	0.98	11441
CARDIAC	1.00	0.99	0.99	3634
DERMA	0.97	0.95	0.96	1977
GASTRO INTESTINAL	0.99	0.92	0.95	6713
NEURO CNS	0.96	0.99	0.97	4621
OTHERS	0.96	0.76	0.85	99
PAIN ANALGESICS	0.97	0.99	0.98	6614
RESPIRATORY	0.99	0.96	0.98	4605
accuracy			0.97	39704
macro avg	0.97	0.94	0.96	39704
weighted avg	0.97	0.97	0.97	39704

รูปที่ 18 ผล Precision, Recall, และ F1-score ของแต่ละคลาส

ผลการประเมินประสิทธิภาพของแบบจำลองโดยละเอียด

จากรูปที่ 19 ตารางนี้แสดงผลการวัดประสิทธิภาพของแบบจำลองในการจำแนกประเภทข้อมูลทางการแพทย์ โดยอิงจากเกณฑ์วัด **Precision**, **Recall**, และ **F1-score** รวมถึงจำนวนตัวอย่าง (**Support**) ของแต่ละประเภท โดยคำนิยามและความสำคัญของแต่ละเกณฑ์สามารถอธิบายได้ดังนี้:

- **Precision** (ค่าความแม่นยำ): แสดงถึงความแม่นยำของแบบจำลองในการจำแนกประเภทข้อมูลที่คาดการณ์ว่าถูกต้องว่ามีความถูกต้องจริง ๆ เพียงใด โดยเป็นอัตราส่วนระหว่าง True Positive และผลรวมของ True Positive และ False Positive ค่ายิ่งสูงแสดงถึงความแม่นยำในการระบุตัวอย่างที่เป็นจริงในแต่ละกลุ่ม
- **Recall** (ค่าการดึงกลับ): วัดความสามารถของแบบจำลองในการดึงข้อมูลประเภทนั้น ๆ ออกมาให้ครบถ้วน โดยเป็นอัตราส่วนระหว่าง True Positive และผลรวมของ True Positive และ False Negative ค่ายิ่งสูงแสดงถึงความสามารถของแบบจำลองในการจำแนกข้อมูลประเภทนั้นได้อย่างครบถ้วน
- **F1-score** เป็นค่าเฉลี่ยระหว่าง Precision และ Recall ช่วยให้เห็นภาพรวมของความแม่นยำและความครอบคลุมในการทำนายแต่ละประเภท ค่ายิ่งใกล้ 1 แสดงว่าแบบจำลองมีประสิทธิภาพในการจำแนกประเภทนั้นได้ดี

ผลการประเมินในแต่ละกลุ่มข้อมูล

ANTI INFECTIVES:

- Precision = 0.96, Recall = 0.99, F1-score = 0.98
- แสดงถึงความสามารถของแบบจำลองในการจำแนกประเภท ANTI INFECTIVES ได้อย่างแม่นยำสูงและครอบคลุม โดยมีค่า Recall สูงถึง 0.99 แสดงว่าแบบจำลองสามารถดึงกลับข้อมูลประเภทนี้ได้เกือบทั้งหมด

CARDIAC:

- Precision = 1.00, Recall = 0.99, F1-score = 0.99
- ผลลัพธ์แสดงว่าแบบจำลองมีความแม่นยำสูงสุดในกลุ่ม CARDIAC ด้วยค่า Precision ที่เท่ากับ 1.00 แสดงว่าทุกตัวอย่างที่แบบจำลองทำนายว่าเป็น CARDIAC นั้นถูกต้องทั้งหมด และค่า F1-score ที่สูงสะท้อนถึงความแม่นยำและครอบคลุมที่ดีมาก

DERMA:

- Precision = 0.97, Recall = 0.95, F1-score = 0.96
- แบบจำลองมีค่า Precision และ Recall ค่อนข้างสูงในกลุ่ม DERMA แต่ค่า Recall ที่ต่ำกว่าเล็กน้อยแสดงถึงความแม่นยำในการจำแนกประเภทนี้อยู่ในเกณฑ์ที่ดี แต่ยังมีความเป็นไปได้ที่บางตัวอย่างถูกจำแนกผิด

GASTRO INTESTINAL:

- Precision = 0.99, Recall = 0.92, F1-score = 0.95
- ค่า Precision อยู่ในระดับสูง (0.99) แสดงถึงความแม่นยำในการทำนายว่าเป็น GASTRO INTESTINAL อย่างไรก็ตาม ค่า Recall ที่ต่ำกว่าเล็กน้อย (0.92) แสดงว่าแบบจำลองอาจพลาดการดึงกลับตัวอย่างในกลุ่มนี้บางส่วน ซึ่งอาจต้องปรับปรุงในด้านนี้

NEURO CNS:

- Precision = 0.96, Recall = 0.99, F1-score = 0.97
- แบบจำลองมีประสิทธิภาพในการจำแนกประเภทนี้ดีมาก โดยมีค่า Precision และ Recall สูง ทำให้มั่นใจได้ว่าแบบจำลองสามารถระบุตัวอย่างในกลุ่มนี้ได้อย่างถูกต้องและครบถ้วน

OTHERS:

- Precision = 0.96, Recall = 0.76, F1-score = 0.85
- ค่า Recall ต่ำกว่ากลุ่มอื่น ๆ โดยอยู่ที่ 0.76 แสดงว่าแบบจำลองยังมีปัญหาในการจำแนกกลุ่ม OTHERS ให้ครบถ้วน อาจเป็นผลจากจำนวนตัวอย่างที่มีน้อย (99 ตัวอย่าง) หรือข้อมูลที่มีความหลากหลาย ส่งผลให้แบบจำลองขาดความแม่นยำในการแยกแยะตัวอย่างในกลุ่มนี้

PAIN ANALGESICS:

- Precision = 0.97, Recall = 0.99, F1-score = 0.98
- ค่า Precision และ Recall ที่สูงในกลุ่มนี้แสดงว่าแบบจำลองมีความสามารถในการจำแนกประเภทนี้ได้อย่างแม่นยำและครอบคลุม ซึ่งสะท้อนถึงประสิทธิภาพที่ดีในการระบุตัวอย่างในกลุ่ม PAIN ANALGESICS

RESPIRATORY:

- Precision = 0.99, Recall = 0.96, F1-score = 0.98
- ผลการทำนายในกลุ่มนี้มีประสิทธิภาพสูง โดยมีค่า Precision และ F1-score สูง ค่า Recall อยู่ที่ 0.96 แสดงถึงความสามารถของแบบจำลองในการจำแนกประเภท RESPIRATORY ได้อย่างแม่นยำ

การสรุปผลการประเมินโดยรวม

Accuracy

- แบบจำลองมีค่า Accuracy รวมที่ 0.97 ซึ่งหมายถึงแบบจำลองมีความถูกต้องในการจำแนกประเภทข้อมูลทั้งหมดถึง 97%

Macro Avg:

- Precision = 0.97, Recall = 0.94, F1-score = 0.96
- ค่าเฉลี่ย Macro ของ Precision และ Recall สะท้อนถึงความแม่นยำและความครอบคลุมของแบบจำลองในแต่ละประเภทข้อมูลแบบไม่ถ่วงน้ำหนัก ซึ่งให้ภาพรวมของประสิทธิภาพของแบบจำลองในแต่ละประเภท

Weighted Avg:

- Precision = 0.97, Recall = 0.97, F1-score = 0.97
- ค่าเฉลี่ยแบบถ่วงน้ำหนักนี้สะท้อนถึงประสิทธิภาพรวมของแบบจำลอง โดยคำนึงถึงจำนวนตัวอย่างในแต่ละประเภท ซึ่งแสดงให้เห็นว่าแบบจำลองมีประสิทธิภาพสูงในการจำแนกประเภทข้อมูลโดยรวม

ข้อเสนอแนะสำหรับการปรับปรุงเพิ่มเติม

แม้ว่าแบบจำลองจะมีความแม่นยำสูงในหลายกลุ่ม แต่ยังพบว่ามีข้อจำกัดในกลุ่มที่มีข้อมูลจำนวนน้อยและมีความหลากหลายสูง เช่น กลุ่ม OTHERS ซึ่งค่า Recall ค่อนข้างต่ำ อาจพิจารณาเพิ่มจำนวนข้อมูลฝึกอบรมในกลุ่มนี้ หรือปรับปรุงแบบจำลองให้มีการแยกแยะคุณลักษณะที่สำคัญเพิ่มเติมในกลุ่มนี้ นอกจากนี้ การใช้เทคนิคการเรียนรู้แบบเสริม (Ensemble Learning) หรือการปรับแต่งพารามิเตอร์เพิ่มเติมอาจช่วยให้แบบจำลองมีความสามารถในการจำแนกประเภทข้อมูลได้ดียิ่งขึ้น

6.1.3 การเปรียบเทียบประสิทธิภาพของโมเดล Naïve Bayes และ Deep learning

	precision	recall	f1-score	support
ANTI INFECTIVES	0.96	0.99	0.98	11441
CARDIAC	1.00	0.99	0.99	3634
DERMA	0.97	0.95	0.96	1977
GASTRO INTESTINAL	0.99	0.92	0.95	6713
NEURO CNS	0.96	0.99	0.97	4621
OTHERS	0.96	0.76	0.85	99
PAIN ANALGESICS	0.97	0.99	0.98	6614
RESPIRATORY	0.99	0.96	0.98	4605
accuracy			0.97	39704
macro avg	0.97	0.94	0.96	39704
weighted avg	0.97	0.97	0.97	39704

Classification Report:

	precision	recall	f1-score	support
ยาฆ่าเชื้อ	1.00	1.00	1.00	1
ยาระงับประสาท	1.00	0.67	0.80	3
ยาแก้ปวด	0.80	1.00	0.89	4
ยาแก้แพ้	1.00	1.00	1.00	6
ยาแก้ไอ ขับเสมหะ	1.00	1.00	1.00	1
accuracy			0.93	15
macro avg	0.96	0.93	0.94	15
weighted avg	0.95	0.93	0.93	15

รูปที่ 19 รูปเปรียบเทียบประสิทธิภาพของโมเดล Naïve Bayes และ Deep learning

1. ผลลัพธ์ของโมเดล Naïve Bayes

จากรายงานการจัดประเภทในรูปแรก โมเดล Naïve Bayes ถูกใช้สำหรับจำแนกข้อความออกเป็น 5 กลุ่ม (ยาฆ่าเชื้อ, ยาระงับประสาท, ยาแก้ปวด, ยาแก้แพ้, ยาแก้ไอ ขับเสมหะ) โดยมีจำนวนตัวอย่างทั้งหมด 15 ตัวอย่าง โดยผลลัพธ์มีดังนี้:

- ค่า precision อยู่ในช่วง 0.80 ถึง 1.00 ซึ่งบ่งบอกถึงความสามารถของโมเดลในการลดจำนวนผลลัพธ์ผิดพลาด
- ค่า recall อยู่ในช่วง 0.67 ถึง 1.00 โดยแสดงถึงความสามารถของโมเดลในการระบุข้อมูลกลุ่มเป้าหมายได้อย่างครบถ้วน
- ค่า f1-score อยู่ในช่วง 0.80 ถึง 1.00 ซึ่งเป็นตัววัดผลรวมของ precision และ recall
- ค่า accuracy ของโมเดลเท่ากับ 0.93 ซึ่งแสดงถึงความถูกต้องโดยรวม

2. ผลลัพธ์ของโมเดล Deep learning

จากรายงานในรูปที่สอง โมเดล LSTM ถูกนำมาใช้จำแนกข้อความออกเป็น 7 กลุ่ม (ANTI INFECTIVES, CARDIAC, DERMA, GASTRO INTESTINAL, NEURO CNS, OTHERS, PAIN ANALGESICS, RESPIRATORY) โดยมีจำนวนตัวอย่างทั้งหมด 39,704 ตัวอย่าง โดยผลลัพธ์มีดังนี้:

- ค่า precision อยู่ในช่วง 0.96 ถึง 1.00 ซึ่งแสดงถึงความแม่นยำสูงมากในการลดความผิดพลาด
- ค่า recall อยู่ในช่วง 0.76 ถึง 0.99 โดยมีข้อจำกัดเล็กน้อยในบางคลาส เช่น กลุ่ม OTHERS ที่ recall เท่ากับ 0.76
- ค่า f1-score อยู่ในช่วง 0.85 ถึง 0.99 ซึ่งแสดงถึงความสมดุลของ precision และ recall
- ค่า accuracy ของโมเดลสูงถึง 0.97 ซึ่งแสดงถึงความแม่นยำโดยรวมที่ดีกว่า Naïve Bayes

3. การเปรียบเทียบประสิทธิภาพระหว่าง 2 โมเดล

โมเดล Naïve Bayes ทำงานได้ดีในกรณีที่มีจำนวนน้อยกลุ่ม (5 กลุ่ม) และจำนวนข้อมูลที่น้อย (15 ตัวอย่าง) แต่ในภาพรวมค่า precision, recall, และ f1-score มีความผันผวนระหว่างกลุ่มต่าง ๆ โมเดล Deep learning มีประสิทธิภาพที่เหนือกว่า Naïve Bayes ในทุกตัววัด โดยเฉพาะอย่างยิ่งเมื่อจำแนกข้อมูลในกลุ่มที่มีขนาดใหญ่ (7 กลุ่ม และ 39,704 ตัวอย่าง) ค่า precision, recall, และ f1-score มีค่าที่สูงกว่าและเสถียรกว่า Deep learning มีความได้เปรียบในการประมวลผลลักษณะของข้อมูลแบบลำดับ (sequence data) เช่น ข้อความ ซึ่งช่วยให้สามารถดึงบริบทที่ซับซ้อนในข้อความได้ดีกว่า Naïve Bayes

สรุปโดยรวม

โมเดล Deep learning มีความเหมาะสมมากกว่า Naïve Bayes ในการจำแนกข้อความ โดยเฉพาะในกรณีที่มีข้อมูลจำนวนมากและมีความซับซ้อน อย่างไรก็ตาม Naïve Bayes เป็นโมเดลที่เรียบง่ายและเหมาะสมในกรณีที่ข้อมูลมีขนาดเล็กและไม่ซับซ้อนมากนัก

6.2 ข้อจำกัด

1. เมื่อต้องการจะใช้งานแอปพลิเคชัน จำเป็นต้องเชื่อมต่ออินเตอร์เน็ตเท่านั้น
2. ปัจจุบันแอพพลิเคชั่นรองรับแค่ระบบ Android เท่านั้น
3. ผู้ใช้ยังไม่สามารถรับรู้ วิธีการทานยาจากแอปพลิเคชันได้
4. ปัจจุบันการเทรนโมเดลด้วยภาษาไทยและภาษาอังกฤษยังไม่สามารถใช้โมเดลเดียวกันได้ ภาษาไทยใช้ Machine Learning ในการเทรน แต่ภาษาอังกฤษใช้ Deep Learning เนื่องจาก จำนวนข้อมูลของยาภาษาไทยไม่เพียงพอ
5. การนำกล่องยาที่มีสรรพคุณภาษาอังกฤษมาใช้เป็นข้อมูล บางชนิดหาพบได้ยากเนื่องจากเป็นยาที่ไม่นิยมในประเทศไทย

6.3 ข้อเสนอแนะ

1. แอปพลิเคชันสามารถบอกวิธีการกินยาได้ว่ากินวันละกี่เม็ดและเวลาในการกิน
2. แอปพลิเคชันสามารถเก็บประวัติของผลลัพธ์เอาไว้ดูเมื่ออยากใช้ยาตัวนั้นอีกครั้ง
3. เก็บข้อมูลภาษาไทยให้มากขึ้นและนำมาใช้กับโมเดลเดียวกัน

บรรณานุกรม

[1] amazon. (2567). “OCR (การรู้จำอักขระด้วยแสง),” [ออนไลน์]. เข้าถึงเมื่อ 20 พฤษภาคม 2567. เข้าถึงได้จาก https://aws.amazon.com/th/what-is/ocr/.

[2] aws.amazon. (2567). “NLP,” [ออนไลน์]. เข้าถึงเมื่อ 20 พฤษภาคม 2567. เข้าถึงได้จาก https://aws.amazon.com/th/what-is/nlp/.

[3] aws.amazon. (2567). “แมชชีนเลิร์นนิงคืออะไร,” . [ออนไลน์]. เข้าถึงเมื่อ 20 พฤษภาคม 2567. เข้าถึงได้จาก https://aws.amazon.com/th/what-is/machine-learning/.

[4] aws.amazon. (2567). “ดิปเลิร์นนิงคืออะไร,” . [ออนไลน์]. เข้าถึงเมื่อ 20 พฤษภาคม 2567. เข้าถึงได้จาก https://aws.amazon.com/th/what-is/deep-learning/.

[5] aws.amazon. “การจัดประเภทข้อความ,” [ออนไลน์]. เข้าถึงเมื่อ 20 พฤษภาคม 2567. เข้าถึงได้จาก https://aws.amazon.com/th/what-is/text-classification/.

[6] N. Soontranon. (2566). “nerd-data,” [ออนไลน์]. เข้าถึงเมื่อ 20 พฤษภาคม 2567. เข้าถึงได้จาก https://www.nerd-data.com/deep_learning_lstm/.

[7] H. Ketmaneechairat, K. Jongsiriworachot และ C. Thangtoon. (2561). “Mobile search engine for medicine information on smartphone,” *Mobile search engine for medicine information on smartphone,* 4 มิถุนายน 2561.

[8] S. E. Rifqi, A. Saiful และ A. Fitra. (2563). “Application Development for Searching Drugs Information,” *Application Development for Searching Drugs Information,* 14 พฤษภาคม 2563.

[9] thedigitaltips . (2566). “Digital Tips,” [ออนไลน์]. เข้าถึงเมื่อ 20 พฤษภาคม 2567. เข้าถึงได้จาก https://thedigitaltips.com/blog/google/what-is-google-lens/.

[10] “geeksforgeeks,” [ออนไลน์]. เข้าถึงเมื่อ 20 พฤษภาคม 2567. เข้าถึงได้จาก https://www.geeksforgeeks.org/naive-bayes-classifiers/.

ภาคผนวก ก ตัวอย่างโค้ดและวิธีการติดตั้ง

ตัวอย่างโค้ด

```
# ตั้งค่าคีย์ API ของ Google Cloud
os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = r'project-service.json'
client = vision_v1.ImageAnnotatorClient()

# ตั้งค่าพาธของไฟล์รูปภาพ
file_name = 'p.jpg'
image_folder = 'C:/Users/User/google_vision_ai_demo/'
image_path = os.path.join(image_folder, file_name)

# อ่านข้อมูลรูปภาพ
with io.open(image_path, 'rb') as image_file:
content = image_file.read()

image = vision_v1.types.Image(content=content)

# ระบุคำใบ้ภาษา (ภาษาไทยและภาษาอังกฤษ)
image_context = vision_v1.types.ImageContext(
language_hints=['th', 'en']
)

# ทำการตรวจจับข้อความในรูปภาพ
response = client.text_detection(image=image, image_context=image_context)
texts = response.text_annotations
```

โค้ดนี้ใช้ Google Cloud Vision API สำหรับการประมวลผลภาพโดยเฉพาะการทำ OCR (Optical Character Recognition) หรือการแปลงข้อความในภาพเป็นข้อมูลตัวอักษร โดยมีรายละเอียดดังนี้

1. การตั้งค่าคีย์ API ของ Google Cloud

- GOOGLE_APPLICATION_CREDENTIALS โดยชี้ไปยังไฟล์ JSON ที่มีคีย์ของโปรเจกต์ Google Cloud เพื่อให้สามารถเข้าถึง API ได้
- client เป็นอินสแตนซ์ของ ImageAnnotatorClient จากไลบรารี vision_v1 ที่ใช้ในการเรียก API ของ Google Cloud Vision สำหรับประมวลผลภาพ

2. การกำหนดรายละเอียดของไฟล์รูปภาพ

- file_name ระบุชื่อไฟล์รูปภาพที่ต้องการประมวลผล (ในที่นี้คือ 'p.jpg')
- image_folder ระบุโฟลเดอร์ที่เก็บรูปภาพ
- image_path คือพาธแบบเต็มของไฟล์รูปภาพที่ได้จากการรวมโฟลเดอร์และชื่อไฟล์เข้าด้วยกันผ่านฟังก์ชัน os.path.join

3. อ่านข้อมูลในรูปภาพ

- ใช้คำสั่ง with เพื่อเปิดไฟล์รูปภาพในโหมด 'rb' (read binary) ซึ่งเหมาะสำหรับการอ่านข้อมูลไบนารี เช่น รูปภาพ
- อ่านข้อมูลจากไฟล์และเก็บไว้ในตัวแปร content

4. สร้างออบเจ็กต์ของภาพ

- สร้างออบเจ็กต์ Image ของ Google Cloud Vision โดยใช้ข้อมูลรูปภาพ (content) ที่อ่านมาเป็นคอนเทนต์ของออบเจ็กต์นี้

5. ระบุบริบทของภาษา (ภาษาไทยและภาษาอังกฤษ)

- สร้างออบเจ็กต์ ImageContext เพื่อระบุบริบทภาษาที่ API จะใช้ในการประมวลผล OCR
- language_hints กำหนดภาษาที่คาดว่าจะพบในรูปภาพ เพื่อช่วยให้ API ตรวจจับตัวอักษรได้แม่นยำขึ้น ในที่นี้ใช้ ['th', 'en'] เพื่อบอกให้ API รู้ว่าภาพอาจมีทั้งภาษาไทยและภาษาอังกฤษ

```
# กรองและรวมข้อความจาก OCR
ocr_texts = [filter_alnum(text.description) for text in texts if
filter_alnum(text.description)]

# แปลงข้อความเป็น vectors โดยใช้ TfidfVectorizer
vectorizer = TfidfVectorizer()
vectors = vectorizer.fit_transform(data['name'].dropna().unique())
ocr_vector = vectorizer.transform([' '.join(ocr_texts)])

# คำนวณค่า cosine similarity ระหว่าง vectors
cosine_similarities = cosine_similarity(ocr_vector, vectors).flatten()
highest_similarity_index = cosine_similarities.argmax()
highest_similarity_value = cosine_similarities[highest_similarity_index]

# กำหนดค่า similarity ที่เหมาะสม
similarity_threshold = 0.5

if highest_similarity_value >= similarity_threshold:
    best_match = data['name'].dropna().unique()[highest_similarity_index]
    matches = data[data['name'].str.contains(best_match, case=False, na=False)]
    if not matches.empty:
        print(matches.iloc[0]
```

โค้ดนี้เป็นการใช้ TfidfVectorizer และ cosine_similarity ในการคำนวณความคล้ายคลึงระหว่างข้อความที่แปลงเป็นเวกเตอร์ ตัวโค้ดมีขั้นตอนดังนี้

1. **แปลงข้อความเป็นเวกเตอร์**โดยใช้ TfidfVectorizer TfidfVectorizer ถูกใช้เพื่อแปลงข้อความเป็นเวกเตอร์ที่มีการปรับน้ำหนักตามความถี่ของคำ TF-IDF (Term Frequency-Inverse Document Frequency) vectors จะเป็นเวกเตอร์ของข้อมูลทั้งหมดจากคอลัมน์ name ใน data หลังจากที่เอาค่า NaN ออกและเลือกค่าไม่ซ้ำ ocr_vector จะเป็นเวกเตอร์ของข้อความ OCR ที่ต่อกันเป็นสตริงเดียวจาก ocr_texts

2. **คำนวณค่า Cosine Similarity** ระหว่างเวกเตอร์ คำนวณค่า cosine_similarity ระหว่าง ocr_vector และ vectors เพื่อดูความคล้ายคลึงกันระหว่างข้อความ OCR กับข้อมูลที่มีใน data หาค่าที่มีความคล้ายสูงสุด (highest_similarity_index) และค่าของความคล้ายสูงสุด (highest_similarity_value)

3. **กำหนดค่า Similarity** ที่เหมาะสม กำหนดเกณฑ์ความคล้ายคลึง (similarity_threshold) ไว้ที่ 0.5 (ค่าที่สูงกว่าหรือเท่ากับนี้จะถือว่ามีความคล้ายคลึงกันมากพอ)

4. **ตรวจสอบว่าค่าความคล้ายสูงสุด**เกิน Threshold (ค่าจำกัดหรือเกณฑ์ที่กำหนดขึ้นเพื่อใช้ในการตัดสินใจหรือตรวจสอบว่าเงื่อนไขหรือข้อมูลที่ได้รับเข้ามานั้นควรจัดอยู่ในกลุ่มใด)ถ้าค่าความคล้ายสูงสุด (highest_similarity_value) มีค่ามากกว่าหรือเท่ากับ similarity_threshold ให้ค้นหาแถวใน data ที่มีข้อความคล้ายกันมากที่สุด (best_match)
ถ้ามีแถวที่พบว่าตรงกับ best_match ก็จะแสดงข้อมูลของแถวนั้นออกมาจาก Data set

```
import os
import io
import pickle
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import nltk
from google.cloud import vision_v1
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
from pythainlp import word_tokenize
```

รูปผนวกที่ ก.1 ไลบรารีสำหรับโมเดล machine learning

จากรูปผนวก ก.1.1 ไลบรารีและโมดูลในการใช้สำหรับการจำแนกสรรพคุณของยาจาก code ในรูปมีดังนี้

โมดูล

- os: ใช้สำหรับการจัดการและเข้าถึงระบบไฟล์ เช่น การสร้างพาธ การตรวจสอบไฟล์ในระบบ
- io: ใช้ในการจัดการการอ่านและเขียนข้อมูลในโหมดไบนารี ซึ่งเหมาะสำหรับการเปิดไฟล์ที่เป็นรูปภาพหรือไฟล์อื่น ๆ
- pickle: ใช้สำหรับการบันทึกและโหลดออบเจ็กต์ใน Python (เช่น โมเดลหรือข้อมูล) ในรูปแบบไฟล์ไบนารี ซึ่งสะดวกในการเก็บโมเดลที่ฝึกแล้วเพื่อนำไปใช้ในภายหลัง

การจัดการข้อมูลและการสร้างกราฟ

- pandas: ไลบรารีสำหรับจัดการและวิเคราะห์ข้อมูลที่มีโครงสร้างแบบตาราง เช่น ข้อมูลจากไฟล์ CSV การกรองข้อมูล การจัดการข้อมูลที่สูญหาย เป็นต้น
- matplotlib.pyplot: ไลบรารีสำหรับการสร้างกราฟและแผนภาพ โดยเฉพาะการสร้างแผนภูมิหรือการพล็อตข้อมูล
- matplotlib.font_manager: ใช้จัดการฟอนต์ที่ใช้ในกราฟที่สร้างด้วย matplotlib เช่น การเปลี่ยนฟอนต์เพื่อรองรับภาษาไทย
- seaborn: ไลบรารีสำหรับการสร้างกราฟที่มีความสวยงามและการออกแบบที่ดี โดยทำงานร่วมกับ matplotlib เพื่อสร้างกราฟที่ดูเรียบง่ายและน่าสนใจ

การประมวลผลภาษาธรรมชาติ (NLP)

- nltk: ไลบรารีที่นิยมใช้ในงานประมวลผลภาษาธรรมชาติ เช่น การตัดคำ การทำ stemming การสร้าง corpus หรือการจัดการกับ stop words
- pythainlp.word_tokenize: ฟังก์ชันจาก pythainlp ที่ใช้สำหรับการตัดคำภาษาไทย ทำให้ง่ายต่อการแยกข้อความภาษาไทยเป็นคำ ๆ

Google Cloud Vision API

- google.cloud.vision_v1: ไลบรารีที่ใช้ในการเข้าถึง Google Cloud Vision API โดยสามารถใช้ทำ OCR (Optical Character Recognition) ซึ่งเป็นการแปลงข้อความในภาพให้กลายเป็นตัวอักษร นอกจากนี้ยังสามารถใช้ในงานอื่น ๆ ที่เกี่ยวกับการประมวลผลภาพ เช่น การตรวจจับวัตถุ การจำแนกประเภทของภาพ ฯลฯ

การแปลงข้อความและการสร้างคุณลักษณะสำหรับ Machine Learning

- CountVectorizer จาก sklearn.feature_extraction.text ใช้แปลงข้อความเป็นเวกเตอร์ของจำนวนครั้งที่คำแต่ละคำปรากฏในข้อความ เหมาะสำหรับการเตรียมข้อมูลข้อความก่อนนำไปสร้างโมเดล เช่น การสร้างโมเดลที่เกี่ยวกับการจำแนกข้อความหรือการคาดการณ์เนื้อหา

การสร้างโมเดล Machine Learning

- MultinomialNB จาก sklearn.naive_bayes อัลกอริทึม Naive Bayes แบบ Multinomial เหมาะสำหรับงานจำแนกประเภทข้อความ โดยจะใช้ในการวิเคราะห์การกระจายของข้อมูลและคาดการณ์จากข้อมูล
- Pipeline จาก sklearn.pipeline ใช้ในการรวมขั้นตอนต่าง ๆ ในกระบวนการของ Machine Learning ให้เป็นกระบวนการเดียวกัน ช่วยลดขั้นตอนการเขียนโค้ดและทำให้การประมวลผลข้อมูลเป็นระบบมากขึ้น
- train_test_split จาก sklearn.model_selection ใช้สำหรับแบ่งข้อมูลออกเป็นชุดการฝึกและชุดการทดสอบ เพื่อให้สามารถประเมินประสิทธิภาพของโมเดลได้

การประเมินผลโมเดล

- classification_report จาก sklearn.metrics ใช้สำหรับสร้างรายงานผลการประเมินโมเดล โดยให้ข้อมูลเกี่ยวกับ precision, recall, F1-score ของแต่ละคลาส
- confusion_matrix จาก sklearn.metrics ใช้สร้างตารางแสดงผลการจำแนกของโมเดลในรูปแบบของ confusion matrix ช่วยให้สามารถเห็นได้ว่าโมเดลสามารถจำแนกแต่ละคลาสได้ถูกต้องหรือไม่

```
data
```

	name	description	type	use
0	Claritin	ช่วยลดอาการเช่น คันตา น้ำมูกไหล และจามที่เกิดจ...	ยาแก้แพ้	อาการแพ้ทั่วไปที่ไม่ทำให้ง่วงนอน
1	Telfast	ลดอาการคันและผื่นที่เกิดจากการแพ้	ยาแก้แพ้	อาการแพ้ที่ไม่ทำให้ง่วงนอน
2	Aerius	ใช้เพื่อลดอาการแพ้ เช่น น้ำมูกไหล, จาม, คันในจ...	ยาแก้แพ้	อาการแพ้ที่ต้องการผลข้างเคียงน้อย
3	Clarinase	บรรเทาอาการหวัดภูมิแพ้เช่น คัดจมูก, น้ำมูกไหล,...	ยาแก้แพ้	อาการแพ้ในจมูกที่รุนแรง
4	Allernix	บรรเทาอาการหวัดภูมิแพ้เช่น คัดจมูก, น้ำมูกไหล,...	ยาแก้แพ้	อาการแพ้ทั่วไปที่ไม่ทำให้ง่วงนอน
...	...	...	...	...
68	Depilox	ใช้ในการรักษาอาการซึมเศร้าและโรควิตกกังวลทั่ว...	ยาระงับประสาท	ใช้ในการรักษาอาการซึมเศร้าและโรควิตกกังวลทั่ว...
69	Neopam	ใช้เพื่อรักษาอาการวิตกกังวล (anxiety), อาการแพ...	ยาระงับประสาท	ใช้เพื่อรักษาอาการวิตกกังวล (anxiety), อาการแพ...
70	Megacetam	ใช้เพื่อส่งเสริมการเพิ่มการไหลเวียนของเลือดไปย...	ยาระงับประสาท	ใช้เพื่อส่งเสริมการเพิ่มการไหลเวียนของเลือดไปย...
71	Paro CR	ชัรักษาโรคซึมเศร้า ,โรควิตกกังวลทั่วไป ,โรคแพน...	ยาระงับประสาท	ชัรักษาโรคซึมเศร้า ,โรควิตกกังวลทั่วไป ,โรคแพน...
72	prexaron 500 tablet	ใช้เพื่อส่งเสริมสุขภาพสมองและระบบประสาท โดยเฉพ...	ยาระงับประสาท	ใช้เพื่อส่งเสริมสุขภาพสมองและระบบประสาท ในกรณี...

รูปผนวกที่ ก.2 ข้อมูลของตัวยาภาษาไทย

```
X_train, X_test, y_train, y_test = train_test_split(df['description'], df['type'], test_size=0.2, random_state=42)
    model = Pipeline([
        ('vectorizer', CountVectorizer()),
        ('classifier', MultinomialNB())
    ])
    history = model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

โค้ดนี้ใช้สำหรับสร้างโมเดลการจำแนกประเภทโดยใช้ Naive Bayes กับข้อมูลข้อความ โดยผ่านการแปลงข้อความเป็นเวกเตอร์และการทดสอบโมเดล

รายละเอียดโค้ด

แบ่งข้อมูลการฝึกและการทดสอบ:

บรรทัดนี้ใช้ฟังก์ชัน train_test_split เพื่อแบ่งข้อมูลใน DataFrame ชื่อ df ออกเป็นชุดข้อมูลฝึก (X_train, y_train) และชุดข้อมูลทดสอบ (X_test, y_test) โดย

- X_train และ X_test คือข้อมูลข้อความในคอลัมน์ description
- y_train และ y_test คือประเภทหรือคลาสของข้อมูลในคอลัมน์ type
- test_size=0.2 แบ่งข้อมูล 20% สำหรับการทดสอบ และ 80% สำหรับการฝึกโมเดล
- random_state=42 กำหนดค่า seed เพื่อให้การแบ่งข้อมูลเป็นไปอย่างสม่ำเสมอเมื่อรันโค้ดซ้ำ

สร้าง Pipeline ของโมเดล:

ส่วนนี้สร้าง Pipeline เพื่อกำหนดขั้นตอนในการแปลงและจำแนกข้อมูล ดังนี้:

- vectorizer ใช้ CountVectorizer() ในการแปลงข้อความเป็นเวกเตอร์ของการนับคำ (Bag of Words) โดยสร้างฟีเจอร์จากความถี่ของคำที่ปรากฏในข้อความ
- classifier ใช้ MultinomialNB() เป็นโมเดล Naive Bayes สำหรับการจำแนกประเภท ซึ่งเหมาะสำหรับการจัดการกับข้อมูลข้อความที่แปลงเป็นจำนวนเต็มตามความถี่

ฝึกโมเดล:

บรรทัดนี้เรียกใช้เมธอด fit เพื่อฝึกโมเดลโดยใช้ข้อมูล X_train และ y_train ซึ่งจะทำให้โมเดลเรียนรู้คุณลักษณะของข้อมูลที่แปลงด้วย CountVectorizer และจำแนกประเภทตาม MultinomialNB

ทำนายผลการทดสอบ:

บรรทัดนี้ใช้เมธอด predict ในการทำนายผลลัพธ์ประเภท (y_pred) ของข้อมูลทดสอบ (X_test) โดยโมเดลที่ผ่านการฝึกมาแล้ว

โค้ดนี้ทำการสร้าง Pipeline สำหรับการแปลงข้อมูลข้อความเป็นเวกเตอร์ของคำด้วย CountVectorizer และทำการจำแนกประเภทข้อความด้วยโมเดล MultinomialNB ของ Naive Bayes โดยใช้ข้อมูลฝึกและทดสอบที่แบ่งจาก DataFrame

```
Predicted type using loaded model: ยาแก้ไอ ขับเสมหะ

Matching data from CSV using loaded model:
Type: ยาแก้ไอ ขับเสมหะ, Use: ใช้สำหรับผู้ที่มีอาการไอและเสมหะในระบบทางเดินหายใจ
```

รูปผนวกที่ ก.3 ผลลัพธ์การ predict ของโมเดล naive bayes

```python
import os
import io
import pandas as pd
import numpy as np
from google.cloud import vision_v1
from pythainlp import word_tokenize
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, Bidirectional
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.callbacks import EarlyStopping
import re
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, recall_score, confusion_matrix, f1_score
import seaborn as sns
import difflib
from sklearn.metrics import classification_report, log_loss
from sklearn.metrics import precision_score
```

รูปผนวกที่ ก.4 ไลบรารีสำหรับโมเดล Long short-term memmory

จากรูปผนวก ก.1.4 ในโค้ดนี้เป็นส่วนของไลบรารีที่ใช้ในการทำงานของโมเดล มีการนำเข้าไลบรารีและโมดูลต่าง ๆ ซึ่งมีบทบาทสำคัญในกระบวนการประมวลผลข้อมูล การเตรียมข้อมูลสำหรับการวิเคราะห์ และการสร้างโมเดลด้วยเทคนิค Machine Learning และ Deep Learning ดังต่อไปนี้

1. โมดูลสำหรับการจัดการไฟล์และข้อความ

- os: ไลบรารีที่ช่วยในการจัดการกับระบบไฟล์ เช่น การระบุเส้นทางไปยังไฟล์หรือโฟลเดอร์
- io: ใช้ในการอ่านและเขียนข้อมูลในรูปแบบไบนารี ซึ่งเป็นประโยชน์ในการจัดการข้อมูลจากไฟล์ที่เป็น binary
- re: ไลบรารีสำหรับการทำงานกับ Regular Expressions เพื่อค้นหาและจัดการข้อความที่มีรูปแบบเฉพาะ

2. ไลบรารีสำหรับการจัดการข้อมูล

- pandas as pd ไลบรารีที่ช่วยในการจัดการข้อมูลในรูปแบบตารางหรือ DataFrame ใช้ในการโหลดและประมวลผลข้อมูล เช่น ข้อมูลในไฟล์ CSV
- numpy as np: ไลบรารีสำหรับการประมวลผลและคำนวณค่าทางคณิตศาสตร์ โดยเฉพาะการจัดการอาร์เรย์ (arrays) และเมทริกซ์

3. ไลบรารีสำหรับการประมวลผลภาษาธรรมชาติ (Natural Language Processing)

- google.cloud import vision_v1 ไลบรารีที่เชื่อมต่อกับ Google Cloud Vision API ใช้ในการวิเคราะห์และประมวลผลภาพ เช่น การทำ OCR
- pythainlp.word_tokenize: ฟังก์ชันจาก pythainlp สำหรับการตัดคำภาษาไทย ช่วยในการแยกข้อความภาษาไทยออกเป็นคำ ๆ เพื่อการวิเคราะห์ข้อความ

4. การเตรียมข้อมูลสำหรับการประมวลผลข้อความ

- sklearn.model_selection import train_test_split ฟังก์ชันสำหรับแบ่งข้อมูลออกเป็นชุดการฝึกและชุดทดสอบเพื่อใช้ในการประเมินประสิทธิภาพของโมเดล
- sklearn.preprocessing import LabelEncoder ใช้แปลงข้อมูลป้ายกำกับ (labels) ให้อยู่ในรูปแบบของตัวเลข
- tensorflow.keras.preprocessing.text import Tokenizer ใช้ในการแปลงข้อความเป็นลำดับของตัวเลขหรือ tokens เพื่อเตรียมข้อมูลสำหรับโมเดล deep learning
- tensorflow.keras.preprocessing.sequence import pad_sequences ใช้สำหรับการเติมเต็มลำดับข้อมูล (padding) เพื่อให้ลำดับข้อมูลทั้งหมดมีความยาวเท่ากัน ซึ่งจำเป็นสำหรับการใช้ในโมเดล RNN/LSTM

5. การสร้างโมเดลด้วย TensorFlow/Keras

- tensorflow.keras.models import Sequential, load_model Sequential ใช้สำหรับสร้างโมเดลแบบ Sequential และ load_model ใช้สำหรับโหลดโมเดลที่ฝึกแล้ว
- tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, Bidirectional
    - Embedding: ใช้แปลงคำหรือ tokens ให้เป็นเวกเตอร์ตัวเลข
    - LSTM: ชั้นของ Recurrent Neural Network ที่ใช้สำหรับประมวลผลข้อมูลที่มีลำดับ เช่น ข้อความ
    - Dense: ชั้นของ Neural Network ที่เป็น Fully Connected Layer
    - Dropout: ใช้เพื่อป้องกันการ overfitting โดยการสุ่มปิดบาง neurons
    - Bidirectional: ใช้ในการสร้าง LSTM แบบสองทิศทาง (bidirectional) เพื่อเพิ่มความแม่นยำในการประมวลผลลำดับ
- tensorflow.keras.utils import to_categorical ใช้แปลงข้อมูลเป้าหมายให้อยู่ในรูปแบบ one-hot encoding
- tensorflow.keras.callbacks import EarlyStopping ใช้หยุดการฝึกโมเดลโดยอัตโนมัติเมื่อการสูญเสีย (loss) ไม่ลดลง เพื่อป้องกันการ overfitting

6. การสร้างกราฟและการแสดงผลข้อมูล

- matplotlib.pyplot as plt ไลบรารีที่ใช้สำหรับการสร้างกราฟและแผนภูมิต่าง ๆ เช่น line plot, scatter plot, และ bar chart
- seaborn as sns: ไลบรารีที่ใช้ในการสร้างกราฟเชิงสถิติที่มีความสวยงามและเป็นมิตรกับผู้ใช้งาน เช่น heatmap และ pairplot

7. การประเมินผลการทำงานของโมเดล

- sklearn.metrics import accuracy_score, recall_score, confusion_matrix, f1_score, classification_report, log_loss, precision_score
    - accuracy_score คำนวณความแม่นยำของการทำนาย
    - recall_score คำนวณค่าความสมบูรณ์ของการดึงข้อมูลที่ทำนายได้ถูกต้อง
    - confusion_matrix แสดงตารางที่สรุปผลการทำนายทั้งที่ถูกต้องและไม่ถูกต้อง
    - f1_score คำนวณค่าเฉลี่ยระหว่าง precision และ recall
    - classification_report แสดงผลการประเมินค่า precision, recall, และ F1-score
    - log_loss คำนวณค่าความสูญเสียของการทำนายแบบโลจิสติกส์ (log loss)
    - precision_score คำนวณความแม่นยำของการทำนายที่ถูกต้อง

```
df1 = pd.DataFrame(data, columns=['use0', 'Therapeutic Class', 'sideEffect1'])
df1
```

	use0	Therapeutic Class	sideEffect1
0	Treatment of Bacterial infections	ANTI INFECTIVES	Nausea
1	Treatment of Bacterial infections	ANTI INFECTIVES	Nausea
2	Treatment of Bacterial infections	ANTI INFECTIVES	Nausea
3	Treatment of Bacterial infections	ANTI INFECTIVES	Nausea
4	Treatment of Resistance Tuberculosis (TB)	ANTI INFECTIVES	Vomiting
...	...	...	...
198511	Treatment of Cough with mucus	RESPIRATORY	Vomiting
198512	Chronic obstructive pulmonary disease (COPD)	RESPIRATORY	Headache
198513	Treatment of Common cold	RESPIRATORY	Rash
198514	Treatment of Allergic conditions	RESPIRATORY	Dizziness
198515	Treatment of Anxiety	RESPIRATORY	Nausea

รูปผนวกที่ ก.5 ข้อมูลของยาภาษาอังกฤษ

```
# สร้าง dataframe สำหรับการประมวลผลโมเดล
df = pd.DataFrame(data, columns=['use0', 'Therapeutic Class', 'sideEffect1'])
df['use0'] = df['use0'].apply(lambda x: ','.join(str(x).split()))
```

```
df
```

	use0	Therapeutic Class	sideEffect1
0	Treatment,of,Bacterial,infections	ANTI INFECTIVES	Nausea
1	Treatment,of,Bacterial,infections	ANTI INFECTIVES	Nausea
2	Treatment,of,Bacterial,infections	ANTI INFECTIVES	Nausea
3	Treatment,of,Bacterial,infections	ANTI INFECTIVES	Nausea
4	Treatment,of,Resistance,Tuberculosis,(TB)	ANTI INFECTIVES	Vomiting
...	...	...	...
198511	Treatment,of,Cough,with,mucus	RESPIRATORY	Vomiting
198512	Chronic,obstructive,pulmonary,disease,(COPD)	RESPIRATORY	Headache
198513	Treatment,of,Common,cold	RESPIRATORY	Rash
198514	Treatment,of,Allergic,conditions	RESPIRATORY	Dizziness
198515	Treatment,of,Anxiety	RESPIRATORY	Nausea

รูปผนวกที่ ก.6 ข้อมูลของยาภาษาอังกฤษที่แบ่งคำแล้ว

จากรูปผนวก ก.6 โค้ดนี้เป็นการจัดการข้อมูล (Data Processing) หรือการเตรียมข้อมูล (Data Preparation)ในโค้ดนี้ การจัดการข้อมูลมีลักษณะดังนี้

1. **การสร้าง DataFrame**

   สร้างตารางข้อมูลในรูปแบบ DataFrame โดยกำหนดชื่อคอลัมน์เป็น use0,Therapeutic Class, และ sideEffect1 การสร้าง DataFrame นี้ช่วยในการจัดการข้อมูลให้อยู่ในโครงสร้างที่ง่ายต่อการทำงานและการประมวลผลในขั้นตอนถัดไป

2. **การแปลงรูปแบบข้อความในคอลัมน์ use0**

มีการปรับรูปแบบข้อมูลในคอลัมน์ use0 โดยใช้ฟังก์ชัน apply() ร่วมกับ lambda เพื่อเปลี่ยนให้แต่ละแถวของข้อมูลในคอลัมน์ use0 มีการคั่นด้วยเครื่องหมายจุลภาค (,) แทนช่องว่าง (space) ซึ่งสามารถแบ่งขั้นตอนนี้ได้ดังนี้

- การแปลงข้อมูลเป็นสตริงและการแยกคำใช้ str(x).split() เพื่อแยกคำในแต่ละแถวออกเป็นรายการย่อยโดยแยกตามช่องว่าง
- การรวมข้อมูลด้วยเครื่องหมายจุลภาคใช้ ',.join(...) เพื่อนำคำที่แยกแล้วมารวมกันอีกครั้งโดยมีเครื่องหมายจุลภาคเป็นตัวคั่น

ขั้นตอนนี้เป็นการทำความสะอาดข้อมูล (Data Cleaning) และการแปลงข้อมูล (Data Transformation) ซึ่งช่วยให้ข้อมูลมีความสอดคล้องกันในรูปแบบที่เหมาะสมต่อการประมวลผลหรือนำไปใช้ในการวิเคราะห์ต่อไป

3. **การแสดงผลข้อมูล**

การเรียก df เพื่อแสดงผล DataFrame ซึ่งเป็นขั้นตอนสุดท้ายเพื่อให้มั่นใจว่าข้อมูลทั้งหมดถูกจัดการและอยู่ในรูปแบบที่ต้องการ

```
# ดึงข้อความ OCR
full_text = ',.join([text.description.replace(',', ' ') for text in texts])
filtered_text = process_text(full_text)
cleaned_text = filtered_text.replace("  ", " ").replace(" ,", ",").replace(", ", ",")
cleaned_text1 = re.sub(r',+', ',', cleaned_text)
cleaned_text2 = re.sub(r'\.+', ' ', cleaned_text1)
```

โค้ดนี้เป็นการจัดการข้อความที่ได้จากกระบวนการ OCR (Optical Character Recognition) เพื่อนำมาทำความสะอาดและแปลงให้อยู่ในรูปแบบที่เหมาะสม โดยกระบวนการในแต่ละบรรทัดทำงานดังนี้

1.รวบรวมข้อความทั้งหมดจากผลลัพธ์ OCR

- ใช้ join() เพื่อรวมข้อความจากแต่ละ text.description ในรายการ texts เข้าเป็นข้อความเดียว โดยคั่นด้วยช่องว่าง (' ').

- ใช้ replace(',', ' ') เพื่อแทนที่เครื่องหมายจุลภาค (,) ด้วยช่องว่าง ช่วยทำให้ข้อมูลไม่มีสัญลักษณ์ที่ไม่ต้องการ

2.กรองข้อความผ่านฟังก์ชัน process_text

- ฟังก์ชัน process_text (full_text) มีหน้าที่กรองหรือทำความสะอาดข้อความเพิ่มเติม ตามการตั้งค่าที่อาจกำหนดในฟังก์ชันนั้น

3.แทนที่และจัดรูปแบบข้อความ

- แทนที่ช่องว่างสองช่อง (" ") ด้วยช่องว่างหนึ่งช่อง (" ").
- แทนที่ช่องว่างก่อนจุด (" .) ด้วยจุด (".") ให้ถูกต้องตามหลักไวยากรณ์.
- แทนที่เครื่องหมายจุลภาคซ้อนกัน (", ,") ด้วยเครื่องหมายจุลภาคหนึ่งตัว (",").

4.การใช้ Regular Expression เพื่อลบช่องว่างหรือจุดซ้อนกัน

- ใช้ re.sub() กับแพทเทิร์น \s+ เพื่อแทนที่ช่องว่างที่ซ้ำกันหลาย ๆ ช่องด้วยช่องว่างเพียงหนึ่งช่อง

5.การลบจุดซ้อนกันหลายจุดให้เหลือเพียงจุดเดียว

- ใช้ re.sub() กับแพทเทิร์น \.+ เพื่อแทนที่จุด (".") ที่ซ้ำกันหลาย ๆ จุดด้วยจุดเพียงหนึ่งจุด

```
# แบ่งข้อมูลเป็น train และ test set
X_train, X_test, y_train, y_test = train_test_split(df['use0'], df['Therapeutic Class'], test_size=0.2,
random_state=42)

# ใช้ Tokenizer เพื่อแปลงข้อความเป็นลำดับตัวเลข
tokenizer = Tokenizer(num_words=7000)
tokenizer.fit_on_texts(X_train)

# แปลงข้อความเป็นลำดับและทำ padding
X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)
```

โค้ดนี้เป็นการเตรียมและจัดการข้อมูลสำหรับการสร้างโมเดลที่ประมวลผลข้อความ ซึ่งประกอบไปด้วยขั้นตอนดังนี้

1. **แบ่งข้อมูลเป็น train และ test set**
   - ใช้ฟังก์ชัน train_test_split จาก sklearn เพื่อแบ่งข้อมูลใน df (DataFrame) เป็นชุดข้อมูลฝึก (train) และทดสอบ (test)
   - df [use0] คือคอลัมน์ข้อมูลที่ใช้เป็น feature (ตัวแปรอิสระ)
   - df [Therapeutic Class] คือคอลัมน์ข้อมูลที่ใช้เป็น target (ตัวแปรตาม)
   - กำหนด test_size=0.2 หมายถึงแบ่งข้อมูลส่วน 20% สำหรับชุดทดสอบ และ 80% สำหรับชุดฝึก
   - random_state=42 เพื่อให้การแบ่งข้อมูลสุ่มเป็นแบบเดิมทุกครั้งที่รัน

2. **ใช้ Tokenizer เพื่อแปลงข้อความให้เป็นลำดับตัวเลข**

- สร้าง Tokenizer โดยกำหนดให้ใช้งานคำสูงสุดไม่เกิน 7000 คำ (num_words=7000)
- fit_on_texts (X_train) คือการฝึก Tokenizer กับข้อมูล X_train เพื่อให้ Tokenizer จดจำและกำหนดหมายเลขประจำคำที่พบใน X_train

3. **แปลงข้อความใน X_train และ X_test ให้เป็นลำดับของตัวเลข**

- แปลงข้อความใน X_train และ X_test ให้กลายเป็นลำดับของตัวเลข (sequences) ตามที่ Tokenizer ได้สร้างไว้
- ลำดับตัวเลขนี้สามารถนำไปใช้กับโมเดลที่ประมวลผลข้อมูลแบบลำดับ (sequence)
- โดยแปลงโดยการใช้ความถี่ของคำมาแปลงคำให้เป็น ลำดับตัวเลข โดยใช้ความถี่ของการเกิดคำมาเป็นตัวกำหนดอันดับในลำดับการแปลง

```
# กำหนดค่าความยาวของลำดับข้อความ
max_length = 150
X_train_padded = pad_sequences(X_train_seq, maxlen=max_length, padding='post')
X_test_padded = pad_sequences(X_test_seq, maxlen=max_length, padding='post')
```

โค้ดนี้ทำหน้าที่ กำหนดความยาวของลำดับข้อมูลข้อความ โดยใช้การเติมข้อมูล (padding) เพื่อให้ลำดับข้อมูลมีขนาดเท่ากัน มีวิธีได้ดังนี้

1. กำหนดความยาวสูงสุดของลำดับข้อความ

- กำหนดความยาวของลำดับข้อความเป็น 150 ตัว โดยใช้ค่าตัวแปร max_length สำหรับใช้ในการเติมข้อมูล
- หากลำดับข้อความมีความยาวน้อยกว่า 150 ตัว โค้ดจะเติมข้อมูลเพิ่มเติมเพื่อให้ได้ความยาวที่กำหนดไว้

2. การเติมข้อมูลในชุดฝึก (X_train_padded)

- ใช้ฟังก์ชัน pad_sequences จาก TensorFlow/Keras เพื่อเติมข้อมูลในลำดับ X_train_seq omaxlen=max_length กำหนดความยาวของลำดับข้อความให้เป็น 150 ตัว opadding ='post' ระบุให้เติมข้อมูลที่ตำแหน่งท้ายลำดับ (post-padding) เพื่อให้ความยาวของแต่ละลำดับเท่ากับ max_length

3. การเติมข้อมูลในชุดทดสอบ (X_test_padded)

- ทำงานเช่นเดียวกับในชุดฝึก โดยเติมข้อมูลใน X_test_seq ให้มีความยาวเท่ากับ max_length ที่ 150 ตัว
- การเติมข้อมูลจะเกิดขึ้นที่ท้ายลำดับ (post-padding) เช่นเดียวกันโค้ดนี้ใช้การ padding เพื่อปรับความยาวของลำดับข้อความในชุดข้อมูลให้เท่ากัน โดยการกำหนดค่าความยาวสูงสุด (max_length=150) และเติมข้อมูลที่ตำแหน่งท้ายลำดับ (padding='post') ซึ่งเป็นวิธีที่ช่วยให้ข้อมูลพร้อมสำหรับการนำเข้าโมเดลที่ต้องการอินพุตแบบลำดับที่มีขนาดคงที่

```
# สร้างตัวแปลง LabelEncoder
label_encoder = LabelEncoder()

# แปลง y_train และ y_test เป็นตัวเลข
y_train = label_encoder.fit_transform(y_train)
y_test = label_encoder.transform(y_test)

# แปลง y_train และ y_test เป็น One-Hot Encoding
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
# แก้ไขชั้นสุดท้ายของโมเดลเป็น softmax
```

โค้ดนี้ใช้สำหรับการแปลง labels ของข้อมูลที่เป็นข้อความให้เป็นค่าตัวเลข และจากนั้นทำการแปลงค่าตัวเลขเหล่านี้ให้เป็น One-Hot Encoding เพื่อเตรียมข้อมูลสำหรับการใช้งานในโมเดลที่ใช้ softmax เป็นฟังก์ชันในขั้นตอนการทำนาย สามารถอธิบายได้ดังนี้

รายละเอียดของโค้ด

1. สร้างตัวแปลง LabelEncoder

- สร้าง INSTANCE ของ LabelEncoder ซึ่งเป็นตัวแปลงจาก sklearn สำหรับ (label) ที่เป็นข้อความให้เป็นค่าตัวเลข

2. แปลง y_train และ y_test ให้เป็นค่าตัวเลข

- fit_transform(y_train) เป็นการฝึก LabelEncoder บนข้อมูล y_train โดยการแปลงป้ายกำกับที่เป็นข้อความให้เป็นตัวเลข เช่น ถ้ามีคลาส ['A', 'B', 'C'] อาจถูกแปลงเป็น [0, 1, 2]
- transform(y_test) ทำการแปลง y_test ให้เป็นค่าตัวเลข โดยใช้ค่าที่ฝึกไว้จาก y_train เพื่อให้ค่าป้ายกำกับใน y_test สอดคล้องกับค่าที่กำหนดใน y_train

3. แปลง y_train และ y_test เป็น One-Hot Encoding

- ใช้ฟังก์ชัน to_categorical จาก TensorFlow/Keras เพื่อแปลงข้อมูล y_train และ y_test ให้เป็น One-Hot Encoding
- One-Hot Encoding จะทำให้ข้อมูลอยู่ในรูปแบบเมทริกซ์ที่มีคอลัมน์เท่ากับจำนวนคลาสทั้งหมด เช่น หากมี 3 คลาส [0, 1, 2] ค่าจะถูกแปลงเป็นรูปแบบ [1, 0, 0], [0, 1, 0], และ [0, 0, 1] ตามลำดับโค้ดนี้ทำการแปลง labels ที่เป็นข้อความในชุดข้อมูลให้เป็นค่าตัวเลข (โดยใช้ LabelEncoder) และแปลงค่าตัวเลขเหล่านั้นเป็น One-Hot Encoding (โดยใช้ to_categorical) เพื่อให้เหมาะสมกับการใช้งานในโมเดลที่มีการทำนายหลายคลาส

```
model = Sequential()
model.add(Embedding(input_dim=5000, output_dim=256, input_length=max_length))
model.add(Bidirectional(LSTM(256, return_sequences=False)))
model.add(Dropout(0.7))
model.add(Dense(128, activation='relu'))
model.add(Dense(len(label_encoder.classes_), activation='softmax'))
```

โค้ดนี้เป็นการสร้างโมเดล (Neural Network) โดยใช้ Keras และกำหนดสถาปัตยกรรมแบบ Sequential สำหรับการจัดลำดับ layer ซึ่งโค้ดนี้น่าจะใช้สำหรับการจัดหมวดหมู่ข้อความ (Text Classification) โดยมีคำอธิบายของแต่ละบรรทัดดังนี้

1. การสร้างโมเดลแบบ Sequential

การสร้างโมเดลแบบ Sequential ซึ่งใช้สำหรับการวาง layer แบบเรียงลำดับทีละชั้น

2. Embedding

Layer เป็น Embedding Layer ซึ่งแปลงคำแต่ละคำในอินพุต (ที่เป็นตัวเลขแทนคำ) ให้อยู่ในรูปแบบเวกเตอร์ที่มีความหนาแน่น

- input_dim=5000 กำหนดขนาดของคำศัพท์ (vocabulary size) โดยในที่นี้มีขนาดเท่ากับ 5,000 คำ ซึ่งหมายความว่าโมเดลนี้สามารถรับคำได้สูงสุด 5,000 คำที่แตกต่างกัน
- output_dim=256 กำหนดขนาดของเวกเตอร์ที่ได้จากการฝัง (embedding) ซึ่งในที่นี้จะได้เวกเตอร์ขนาด 256 มิติสำหรับแต่ละคำ
- input_length=max_length กำหนดความยาวสูงสุดของลำดับอินพุต

3. **Bidirectional LSTM Layer**

Layer นี้เป็น LSTM แบบสองทิศทาง (Bidirectional LSTM) ซึ่งช่วยให้โมเดลสามารถเรียนรู้ข้อมูลจากลำดับทั้งในทิศทางข้างหน้าและข้างหลังได้

- LSTM(256) หมายถึงการใช้ LSTM ที่มีหน่วยประมวลผลขนาด 256 หน่วย
- return_sequences=False กำหนดให้ส่งออกเฉพาะผลลัพธ์สุดท้ายของลำดับเท่านั้น ซึ่งมักใช้สำหรับงานการจัดหมวดหมู่ลำดับ (Sequence Classification)

4. Dropout Layer

Layer นี้เป็น Dropout Layer ซึ่งใช้เพื่อลดการเกิดการจดจำข้อมูลแบบเกินจริง (Overfitting) โดยจะสุ่มปิดหน่วยประมวลผลบางส่วนในระหว่างการฝึก โดยในที่นี้ 0.7 หมายถึงปิด 70% ของหน่วยประมวลผล

5. Dense Layer (Fully Connected Layer)

Layer นี้เป็น Dense Layer หรือแบบ Fully Connected ที่มีหน่วยประมวลผลจำนวน 128 หน่วยและใช้ฟังก์ชันกระตุ้นแบบ ReLU (Rectified Linear Unit) ซึ่งช่วยให้โมเดลสามารถเรียนรู้ความสัมพันธ์ที่ซับซ้อนได้มากขึ้น

6. Output Layer

layer นี้เป็นเอาต์พุตที่ใช้สำหรับการจัดหมวดหมู่แบบหลายคลาส (Multiclass Classification)

- len(label_encoder.classes_) กำหนดจำนวนของหน่วยประมวลผลเท่ากับจำนวนคลาสทั้งหมด (ตามที่ได้จาก Label Encoder)
- activation='softmax' ใช้ฟังก์ชันกระตุ้นแบบ Softmax เพื่อให้ผลลัพธ์เป็นความน่าจะเป็นของแต่ละคลาส

```
# เพิ่ม Early Stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
```

```
# เทรนโมเดล
history = model.fit(X_train_padded, y_train, epochs=100, batch_size=32, validation_data=(X_test_padded, y_test), callbacks=[early_stopping])
```

```
Epoch 1/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 1017s 205ms/step - accuracy: 0.9206 - loss: 0.2664 - val_accuracy: 0.9713 - val_loss: 0.1009
Epoch 2/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 976s 197ms/step - accuracy: 0.9701 - loss: 0.1083 - val_accuracy: 0.9728 - val_loss: 0.0991
Epoch 3/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 882s 178ms/step - accuracy: 0.9711 - loss: 0.1039 - val_accuracy: 0.9720 - val_loss: 0.0990
Epoch 4/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 889s 179ms/step - accuracy: 0.9713 - loss: 0.1026 - val_accuracy: 0.9707 - val_loss: 0.0969
Epoch 5/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 896s 181ms/step - accuracy: 0.9717 - loss: 0.0994 - val_accuracy: 0.9731 - val_loss: 0.0961
Epoch 6/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 896s 181ms/step - accuracy: 0.9721 - loss: 0.0995 - val_accuracy: 0.9714 - val_loss: 0.0964
Epoch 7/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 886s 178ms/step - accuracy: 0.9726 - loss: 0.0973 - val_accuracy: 0.9731 - val_loss: 0.0955
Epoch 8/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 890s 179ms/step - accuracy: 0.9719 - loss: 0.0981 - val_accuracy: 0.9731 - val_loss: 0.0964
Epoch 9/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 887s 179ms/step - accuracy: 0.9722 - loss: 0.0982 - val_accuracy: 0.9726 - val_loss: 0.0975
Epoch 10/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 878s 177ms/step - accuracy: 0.9725 - loss: 0.0969 - val_accuracy: 0.9735 - val_loss: 0.0968
Epoch 11/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 884s 178ms/step - accuracy: 0.9716 - loss: 0.0984 - val_accuracy: 0.9732 - val_loss: 0.0958
Epoch 12/100
4963/4963 ━━━━━━━━━━━━━━━━━━━━ 974s 196ms/step - accuracy: 0.9718 - loss: 0.0981 - val_accuracy: 0.9722 - val_loss: 0.0977
```

รูปผนวกที่ ก.7 โค้ดนี้เป็นการฝึกโมเดลโดยใช้ EarlyStopping เพื่อหยุดการฝึกโมเดลหากไม่เกิดการพัฒนาเพิ่มเติม

จากรูปผนวก ก.1.7 โค้ดนี้เป็นการสร้างขั้นตอนการฝึกโมเดลโครงข่ายประสาทเทียม (Neural Network) โดยใช้เทคนิค Early Stopping เพื่อเพิ่มประสิทธิภาพของการฝึกโมเดล

1. การกำหนด Early Stopping

สร้างตัวแปร early_stopping ซึ่งเป็นเทคนิคการหยุดการฝึกแบบอัตโนมัติเมื่อการปรับปรุงประสิทธิภาพของโมเดลเริ่มคงที่

- monitor='val_loss' เป็นการกำหนดให้ติดตามค่าวัดผลการสูญเสีย (loss) ของข้อมูลชุดทดสอบ (validation loss) เพื่อใช้เป็นเกณฑ์ในการหยุดการฝึก
- patience=5 กำหนดให้หยุดการฝึกเมื่อค่าวัดผลของชุดทดสอบไม่ดีขึ้นติดต่อกัน 5 รอบ (epochs)
- restore_best_weights=True กำหนดให้คืนค่าถ่วงน้ำหนัก (weights) ของโมเดลที่ดีที่สุดก่อนจะหยุดการฝึก เพื่อให้โมเดลอยู่ในสภาวะที่มีประสิทธิภาพสูงสุด

2. การฝึกโมเดล

การฝึกโมเดลโดยใช้ข้อมูลการฝึกและการทดสอบ โดยมีพารามิเตอร์การฝึกดังนี้

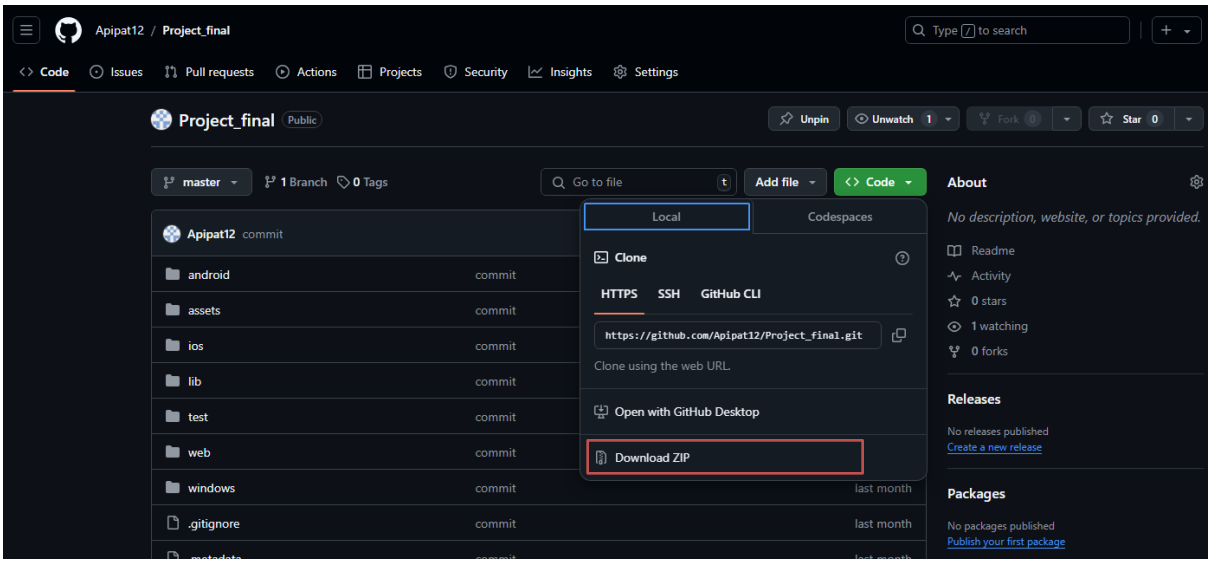
- X_train_padded และ y_train คือข้อมูลการฝึก (อินพุตและผลลัพธ์ที่แท้จริง) ที่ผ่านการปรับความยาวให้เท่ากัน (padding) แล้ว
- epochs=100 กำหนดจำนวนรอบสูงสุดในการฝึกโมเดลเท่ากับ 100 รอบ
- batch_size=32 กำหนดให้โมเดลอัปเดตค่าถ่วงน้ำหนักทีละ 32 ตัวอย่างในแต่ละรอบ
- validation_data=(X_test_padded, y_test) กำหนดข้อมูลชุดทดสอบเพื่อใช้วัดผลการทำงานของโมเดลในแต่ละรอบของการฝึก
- callbacks=[early_stopping] เป็นการเรียกใช้งาน Early Stopping เพื่อหยุดการฝึกเมื่อเงื่อนไขการหยุดที่กำหนดไว้เป็นจริง

3. ผลลัพธ์ที่ได้จากการฝึกโมเดล

ในส่วนของผลลัพธ์การฝึกแสดงให้เห็นถึงการเปลี่ยนแปลงของค่าความแม่นยำ (accuracy) และค่าการสูญเสีย (loss) ทั้งของข้อมูลฝึก (accuracy และ loss) และข้อมูลชุดทดสอบ (val_accuracy และ val_loss) ในแต่ละรอบของการฝึก โมเดลจะหยุดการฝึกเมื่อการสูญเสียในชุดทดสอบ (val_loss) หยุดลดลงติดต่อกัน 5 รอบตามเงื่อนไขที่กำหนดใน Early Stopping

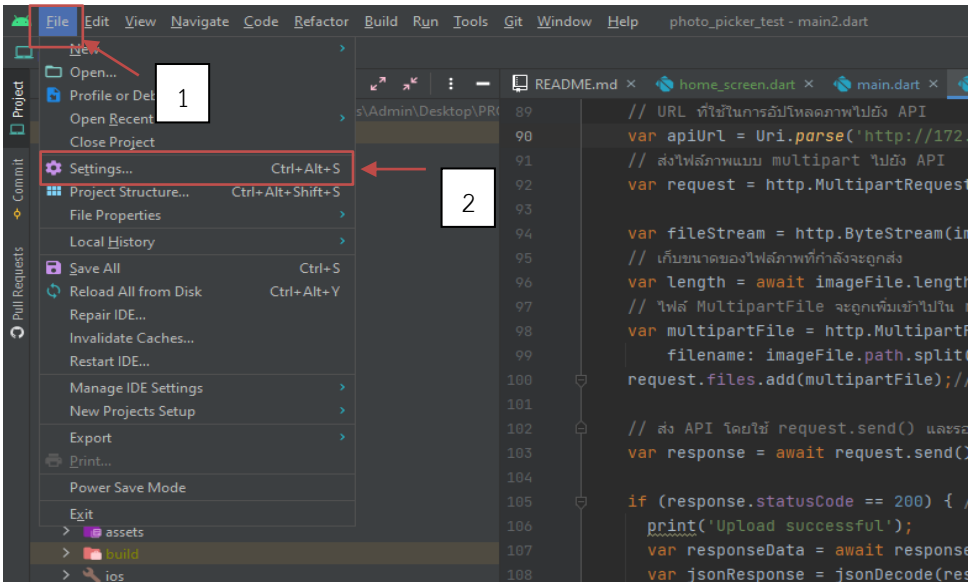
วิธีการติดตั้งแอปพลิเคชัน

1. Download Zip ไฟล์และทำการแตกไฟล์ จากนั้นเปิดโฟลเดอร์ด้วยโปรแกรม Android Studio ดังรูปผนวกที่ ก.2.1



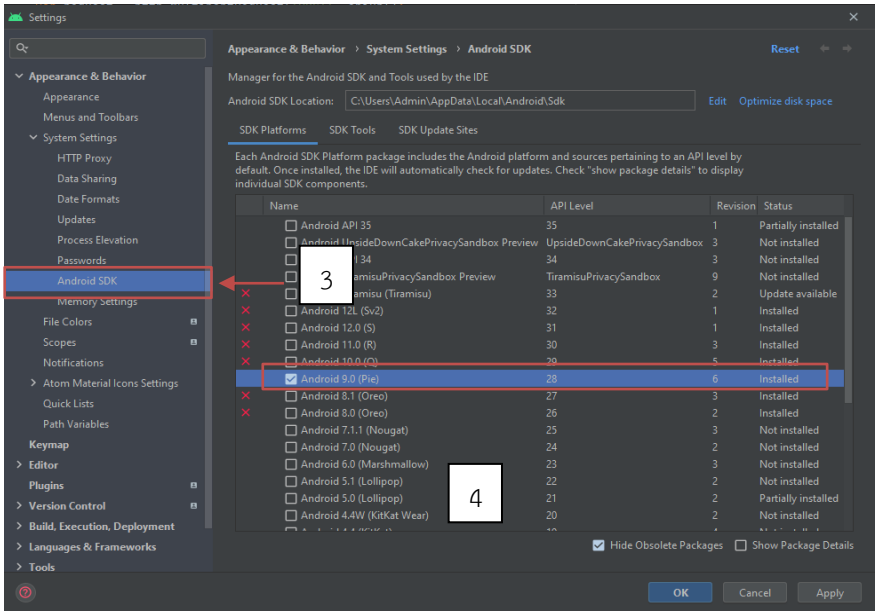
รูปผนวกที่ ก.8 รูป Download Zip ไฟล์ของ Application

2. เมื่อเปิดโฟลเดอร์สำเร็จแล้ว ทำการเปิดเมนู File เลือก Settings ดังรูปผนวกที่ ก.2.2



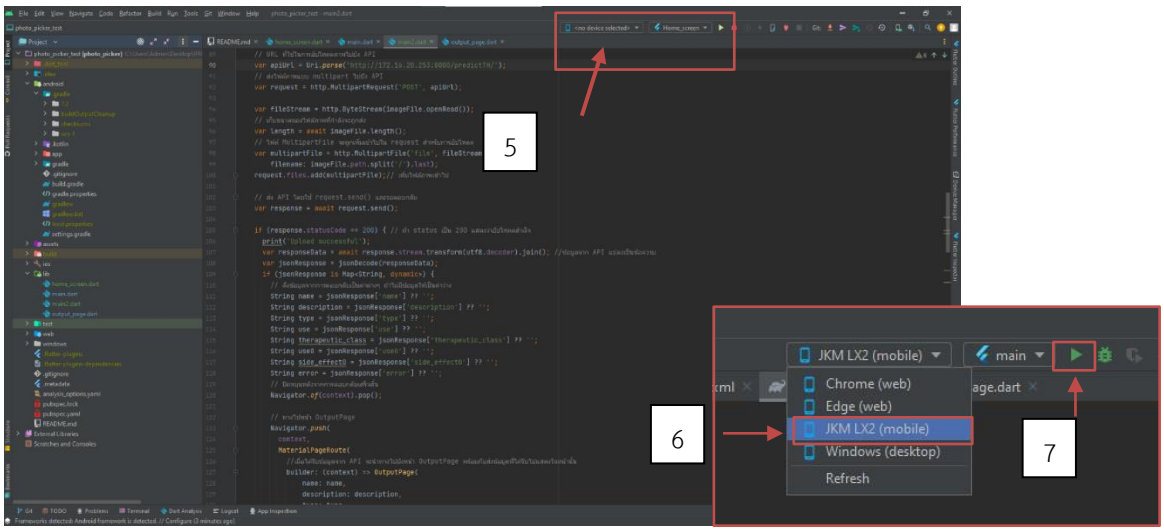
รูปผนวกที่ ก.9 รูปเปิดเมนู File เลือก Settings ใน Android Studio

3. เลือกหัวข้อ Android SDK เพื่อทำการตั้งค่า Android Version ให้สอดคล้องกับอุปกรณ์โทรศัพท์มือถือติดตั้งให้เรียบร้อย แล้วกด OK ดังรูปผนวกที่ ก.2.3



รูปผนวกที่ ก.10 รูปการตั้งค่า Android Version

4. เชื่อมต่อกับอุปกรณ์โทรศัพท์มือถือ จากนั้นเลือก devices เป็นอุปกรณ์ (mobile) ที่เชื่อมต่อและกดปุ่ม Run Application เป็นการติดตั้ง Application บนอุปกรณ์โทรศัพท์มือถือดังรูปผนวกที่ ก.2.4

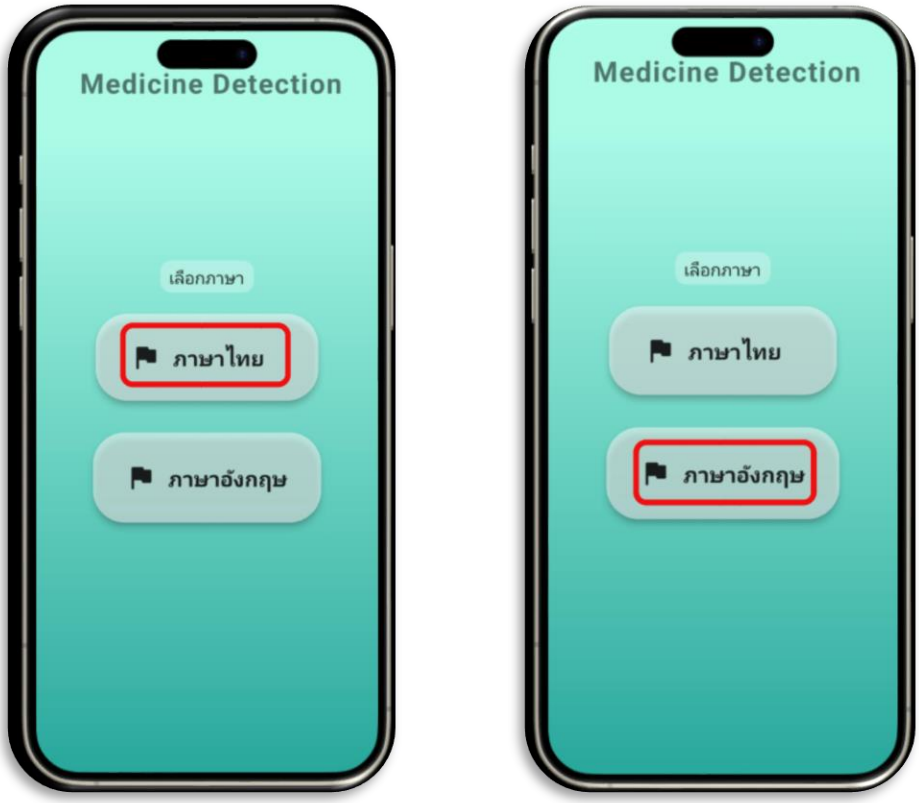


รูปผนวกที่ ก.11 รูปเลือก devices และกดปุ่ม Run Application เป็นการติดตั้ง Application

ภาคผนวก ข คู่มือการใช้งาน

หน้าเลือกภาษาของแอปพลิเคชัน

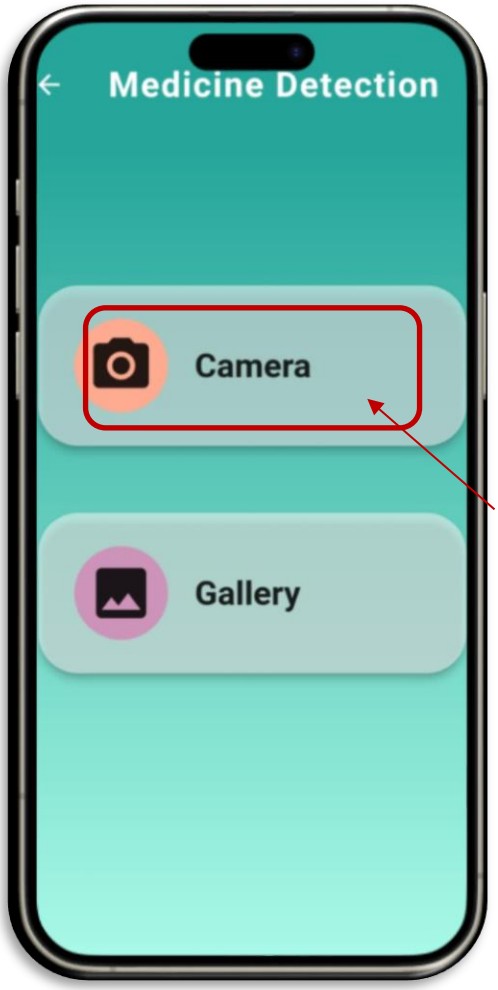
ในหน้านี้จะให้ผู้ใช้เลือกเมนูภาษาในการใช้งานระหว่างการทำนายด้วยชื่อยาภาษาไทย หรือการทำนายด้วยชื่อยาภาษาอังกฤษ



รูปผนวกที่ ข.1 หน้าเลือกภาษาของแอปพลิเคชั่น

หน้าเลือกฟังก์ชันของแอปพลิเคชัน

มีปุ่มในการใช้งานให้เลือก 2 ปุ่ม คือ ปุ่มเปิดกล้องเพื่อถ่ายภาพ และ ปุ่มเลือกรูปภาพจาก Gallery



รูปผนวกที่ ข.2 หน้าเลือกฟังก์ชันของ
แอปพลิเคชั่น กดปุ่มเปิดกล้องเพื่อถ่ายภาพ

เมื่อเลือกเมนูเปิดกล้องเพื่อถ่ายภาพ

แอปพลิเคชันจะทำการเรียกฟังก์ชันเปิดกล้องถ่ายรูป ให้ทำการถ่ายรูปหน้าฉลากยาหรือกล่องยา จากนั้นแอปพลิเคชันจะทำการ OCR (Optical Character Recognition) จำแนกตัวอักษรจากภาพถ่ายเพื่อดึงชื่อยาออกมาและนำไปค้นหาชื่อยาตรงกันใน Data Sets

รูปผนวกที่ ข.3 ฟังก์ชั่นเปิดกล้องถ่ายรูป ให้ทำการถ่ายรูปหน้าฉลากยา
หรือกล่องยา

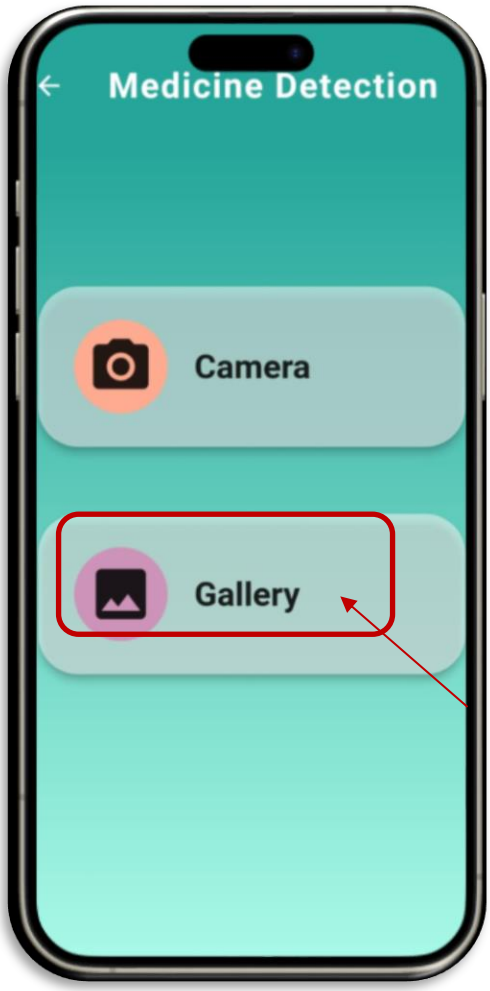
หน้าแสดงผลของแอปพลิเคชันเมื่อพบข้อมูล

เมื่อพบข้อมูลชื่อยาที่ตรงกับ Data Sets แอปพลิเคชันจะแสดงผลลัพธ์ออกมาเป็นข้อมูลของยาชนิดนั้นมีหัวข้อดังต่อไปนี้ ชื่อยา สรรพคุณยา และประเภทของยา

รูปผนวกที่ ข.4 หน้าแสดงผลของแอปพลิเคชั่นเมื่อพบข้อมูล

หน้าเลือกฟังก์ชันของแอปพลิเคชัน

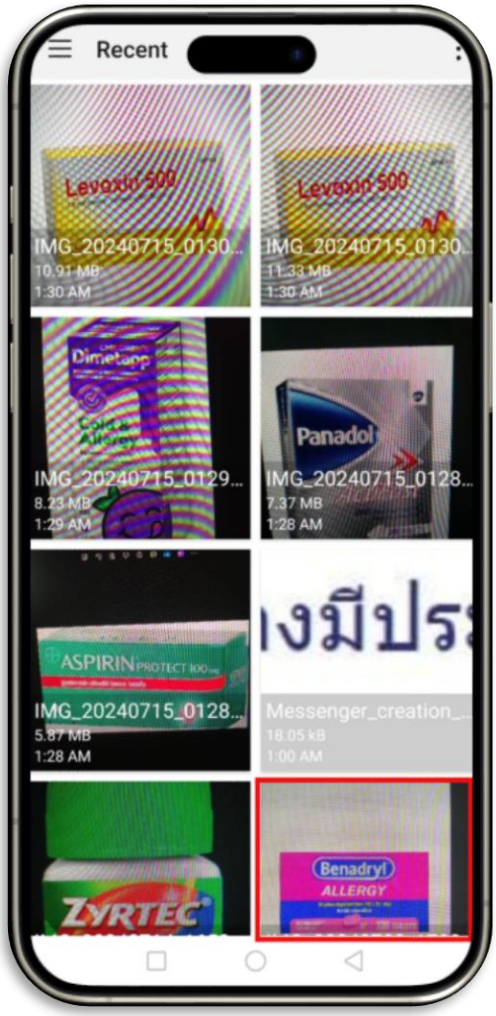
เมื่อกดปุ่มเลือกรูปภาพจาก Gallery เพื่อดึงชื่อยาออกมา



รูปผนวกที่ ข.5 หน้าเลือกฟังก์ชันของแอปพลิเคชั่น กดปุ่มเลือกรูปภาพจาก Gallery

เมื่อเลือกเมนูเลือกรูปภาพจาก Gallery

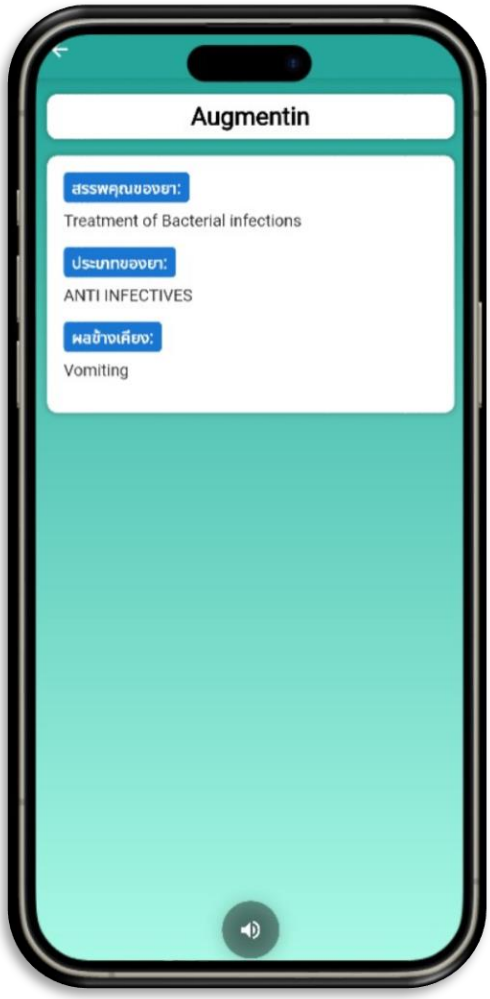
แอปพลิเคชันจะทำการเรียกฟังก์ชันเปิด Gallery ในโทรศัพท์มือถือ เพื่อทำการเลือกรูปหน้าฉลากยาหรือกล่องยา จากนั้นแอปพลิเคชันจะทำการ OCR (Optical Character Recognition) จำแนกตัวอักษรจากรูปภาพใน Gallery เพื่อดึงชื่อยาออกมาและนำไปค้นหาชื่อยาตรงกันใน Data Sets



รูปผนวกที่ ข.6 ฟังก์ชั่นเปิด Gallery ในโทรศัพท์มือถือ เพื่อทำการเลือกรูปหน้าฉลากยา

หน้าแสดงผลของแอปพลิเคชันเมื่อพบข้อมูล

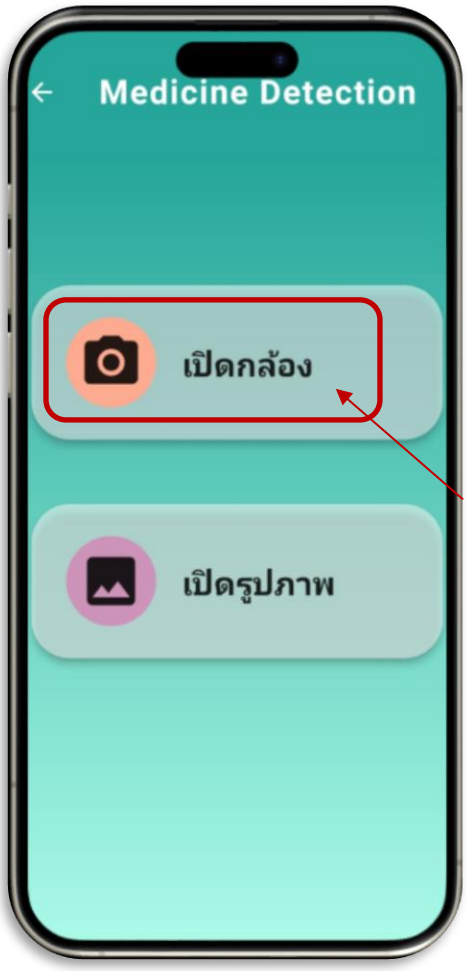
เมื่อพบข้อมูลชื่อยาที่ตรงกับ Data Sets แอปพลิเคชันจะแสดงผลลัพธ์ออกมาเป็นข้อมูลของยาชนิดนั้นมีหัวข้อดังต่อไปนี้ ชื่อยา สรรพคุณยา และประเภทของยา



รูปผนวกที่ ข.7 หน้าแสดงผลของแอปพลิเคชั่นเมื่อพบข้อมูล

ในกรณีที่ไม่พบชื่อยี่ห้อยาใน Data sets จะจำแนกจากสรรพคุณ

แอปพลิเคชันยังสามารถทำการจำแนกประเภทของยาจากสรรพคุณโดยการสแกน Text จากสรรพคุณของยาโดยการถ่ายรูปภาพ



รูปผนวกที่ ข.8 หน้าเลือกฟังก์ชันของแอปพลิเคชั่น ในกรณีที่ไม่พบชื่อยี่ห้อยาใน Data sets จะจำแนกจากสรรพคุณ

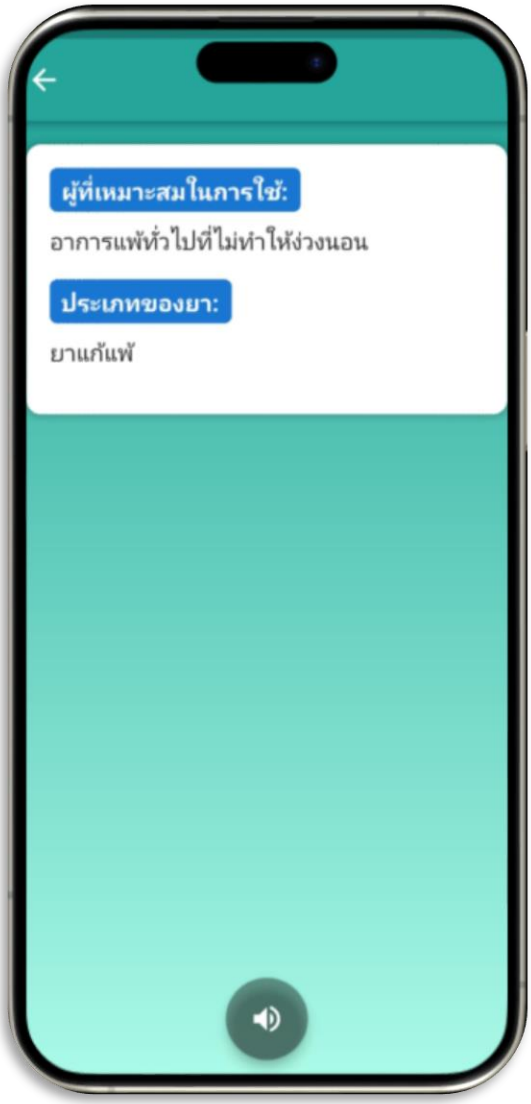
เมื่อเลือกเมนูเปิดกล้องเพื่อถ่ายภาพ ในกรณีจำแนกจากสรรพคุณ

แอปพลิเคชันจะทำการเรียกฟังก์ชันเปิดกล้องถ่ายรูป ให้ทำการถ่ายรูปสรรพคุณของยา จากนั้นแอปพลิเคชันจะทำการ OCR (Optical Character Recognition) ดึงตัวอักษรจากภาพถ่ายเพื่อจำแนกสรรพคุณยาออกมาโดยจับคำที่ใกล้เคียงที่สุดกับสรรพคุณใน Data Sets เช่น “แก้ปวด” ,“ลดไข้” และจะจำแนกว่าสรรพคุณยานั้นเป็นยาประเภทใด

รูปผนวกที่ ข.9 หน้าเลือกเมนูเปิดกล้องเพื่อถ่ายภาพ ในกรณีจำแนกจากสรรพคุณ

หน้าแสดงผลของแอปพลิเคชันในกรณีจำแนกจากสรรพคุณ

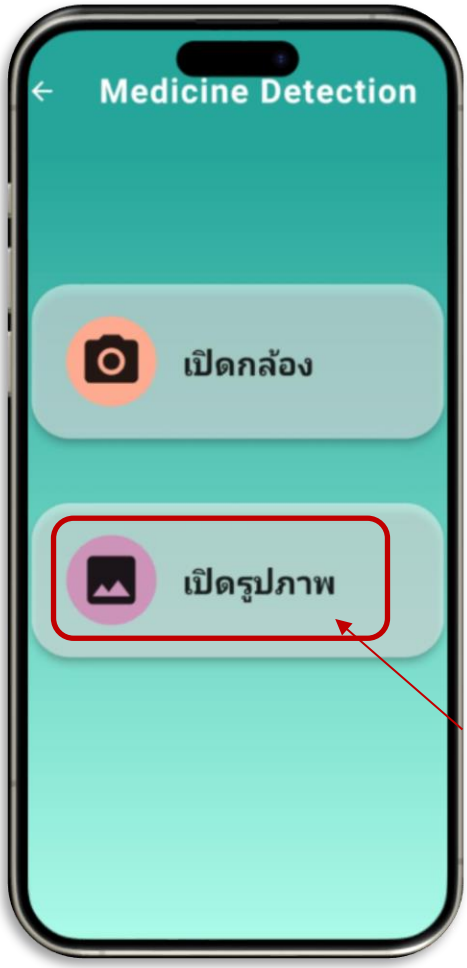
แอปพลิเคชันจะแสดงผลลัพธ์ออกมาเป็นประเภทของยาชนิดนั้นและใช้สำหรับผู้ป่วยประเภทใด



รูปผนวกที่ ข.10 หน้าแสดงผลของแอปพลิเคชั่นในกรณีจำแนกจากสรรพคุณ

ในกรณีที่ไม่พบชื่อยี่ห้อยาใน Data sets จะจำแนกจากสรรพคุณ

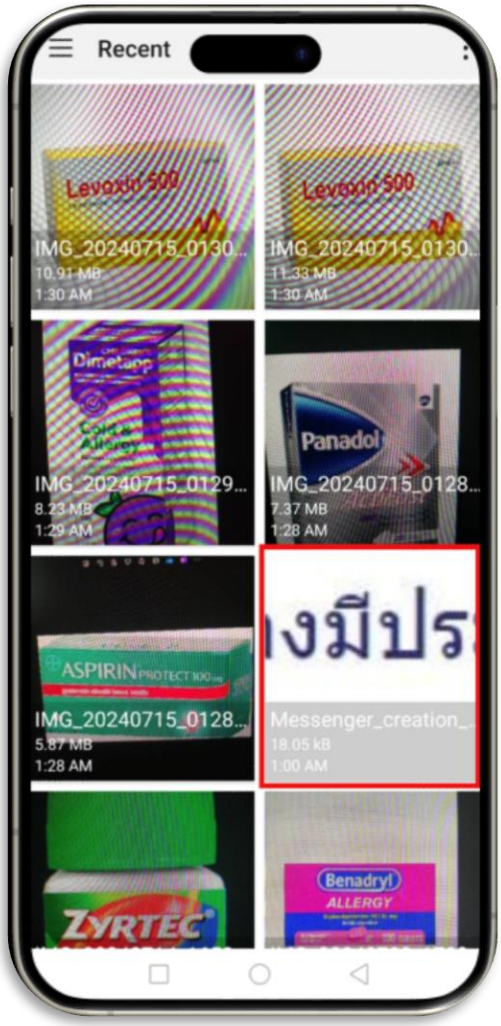
แอปพลิเคชันยังสามารถทำการจำแนกประเภทของยาจากสรรพคุณโดยการสแกน Text จากสรรพคุณของยาโดยการเลือกภาพภ่ายใน Gallery



รูปผนวกที่ ข.11 หน้าเลือกฟังก์ชันของแอปพลิเคชัน กดปุ่มเลือกรูปภาพจาก Gallery

เมื่อเลือกเมนูเลือกรูปภาพจาก Gallery ในกรณีจำแนกจากสรรพคุณ

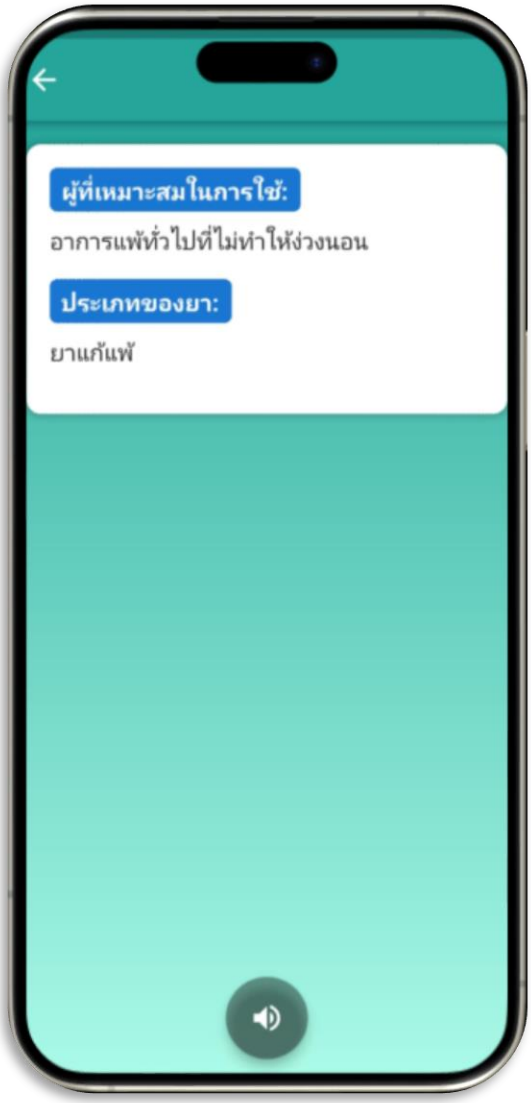
แอปพลิเคชันจะทำการเรียกฟังก์ชันเปิด Gallery ในโทรศัพท์มือถือ เพื่อทำการเลือกรูปหน้าสรรพคุณยาที่ถ่ายไว้ จากนั้นแอปพลิเคชันจะทำการ OCR (Optical Character Recognition) ดึงตัวอักษรจากภาพถ่ายเพื่อจำแนกสรรพคุณยาออกมาโดยจับคำที่ใกล้เคียงที่สุดกับสรรพคุณใน Data Sets เช่น “แก้ปวด” “ลดไข้” และจะจำแนกว่าสรรพคุณยานั้นเป็นยาประเภทใด



รูปผนวกที่ ข.12 หน้าเมนูเลือกรูปภาพจาก Gallery ในกรณีจำแนกจากสรรพคุณ

หน้าแสดงผลของแอปพลิเคชันในกรณีจำแนกจากสรรพคุณ

แอปพลิเคชันจะแสดงผลลัพธ์ออกมาเป็นประเภทของยาชนิดนั้นและใช้สำหรับผู้ป่วยประเภทใด



รูปผนวกที่ ข.13 หน้าแสดงผลของแอปพลิเคชั่นในกรณีจำแนกจากสรรพคุณ

หน้าแสดงผลของแอปพลิเคชันในกรณีไม่พบข้อมูลยา

ในกรณีที่ไม่พบข้อความหรือไม่สามารถจำแนกได้จะแสดงแจ้งเตือนว่า”ไม่พบข้อมูลยา”

รูปผนวกที่ ข.14 หน้าแสดงผลของแอปพลิเคชันในกรณีไม่พบข้อมูลยา